

Test Review

John Urbanic

Parallel Computing Scientist
Pittsburgh Supercomputing Center

Distinguished Service Professor
Carnegie Mellon University

Exam Question 1

1) (3 points) Given this MySQL table named **Customer**

Field	Type	Null	Key	Default	Extra
<u>CustomerId</u>	int	NO		0	
FirstName	<u>varchar(50)</u>	YES		NULL	
LastName	<u>varchar(50)</u>	YES		NULL	
AddressLine1	<u>varchar(100)</u>	YES		NULL	
AddressLine2	<u>varchar(100)</u>	YES		NULL	
City	<u>varchar(50)</u>	YES		NULL	
State	<u>varchar(50)</u>	YES		NULL	
<u>PostalCode</u>	<u>varchar(15)</u>	YES		NULL	
Country	<u>varchar(50)</u>	YES		NULL	
<u>CreditLimit</u>	<u>varchar(50)</u>	YES		NULL	

Write the SQL statement to show the first and last names of all customers in Postal Code 32405.

```
SELECT FirstName, LastName
FROM Customer
WHERE PostalCode = "32405";
```

ORDER OF OPERATIONS

One counter intuitive, but central notion, to SQL is that the listed order of the specified commands does not correspond to the order in which they are executed. There is a mandated order to the evaluation of the clauses.

1. FROM and JOINS

The FROM clause, and subsequent JOINS are first executed to determine the set of data that is being queried. This includes subqueries in this clause, and can cause temporary tables to be created.

2. WHERE

Then any WHERE constraints are applied to the individual rows, and rows that do not satisfy the constraint are discarded. Each of the constraints can only access columns directly from the tables requested in the FROM clause. Aliases in the SELECT part of the query are not accessible since they may include expressions dependent on parts of the query that have not yet executed.

3. GROUP BY

Remaining rows after the WHERE constraints are applied are then grouped based on common values in the column specified in the GROUP BY clause. As a result of the grouping, there will only be as many rows as there are unique values in that column. This means that you should only use this when you have aggregate functions in your query.

4. HAVING

If the query has a GROUP BY clause, then any constraints of a HAVING clause are applied to the grouped rows. Like the WHERE clause, aliases may also not be accessible from this step.

5. SELECT

Any expressions in the SELECT part of the query are finally computed.

6. DISTINCT

Of the remaining rows, rows with duplicate values in the column marked as DISTINCT will be discarded.

7. ORDER BY

If an ORDER BY is specified, the rows are then sorted by the specified data in either ascending or descending order. Since all the expressions in the SELECT part of the query have been computed, you can reference aliases at this point.

8. LIMIT / OFFSET

Last, the rows that fall outside the range specified by the LIMIT and OFFSET are discarded.

Exam Question 2

2) (1 point) List all the Fields from **Customer** that could be a primary key.

Field	Type	Null	Key	Default
<u>CustomerId</u>	int	NO		0
FirstName	<u>varchar(50)</u>	YES		NULL
LastName	<u>varchar(50)</u>	YES		NULL
AddressLine1	<u>varchar(100)</u>	YES		NULL
AddressLine2	<u>varchar(100)</u>	YES		NULL
City	<u>varchar(50)</u>	YES		NULL
State	<u>varchar(50)</u>	YES		NULL
<u>PostalCode</u>	<u>varchar(15)</u>	YES		NULL
Country	<u>varchar(50)</u>	YES		NULL
<u>CreditLimit</u>	<u>varchar(50)</u>	YES		NULL

Keys

If we are trying to quickly equate things from two tables, you might imagine that the organization of those tables might have a major effect on performance. Indeed, the correct selection of keys for each table is the most important consideration.

There are a variety of key types. Two are very important.

Primary Key

A column (*or possibly combination of columns!*) with unique values.

Foreign Key

A column whose values point to a Primary Key in a different table.

There are other terms for keys that are less important to know. *Candidate Keys* are any keys that could be a Primary Key. A *Unique Key* could have a single NULL value (which is not allowed for a Primary Key). A *Composite Key* is a key created from multiple columns, etc.

Keys

Primary Keys are very important as the database can use that as an index to allow us to quickly find a record. This is usually via a good *hashing algorithm*.

When we are doing a join, this allows us to quickly find any two items we are wishing to compare.

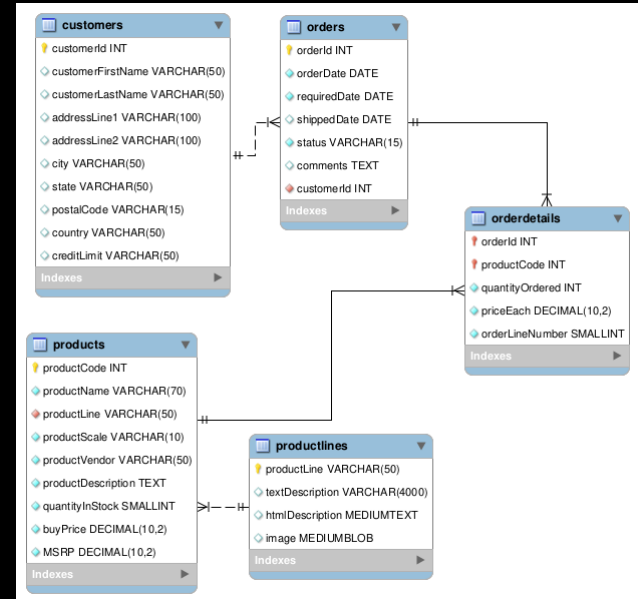
This is why we really prefer our joins to use the assigned table keys if possible.

Keys can also aid greatly in ensuring data integrity.

If it is the case that every record should be unique (order #s, for example), then using that as the primary key will enforce that condition.

A necessary relationship between data in different tables can be enforced with foreign keys. If an Order table uses a customer ID as a foreign key, they will ensure that a matching customer exists in a Customer data table.

A common default Primary Key is simply an integer that might be auto-incremented as each new record is added. In *Pandas* we always have a row number.

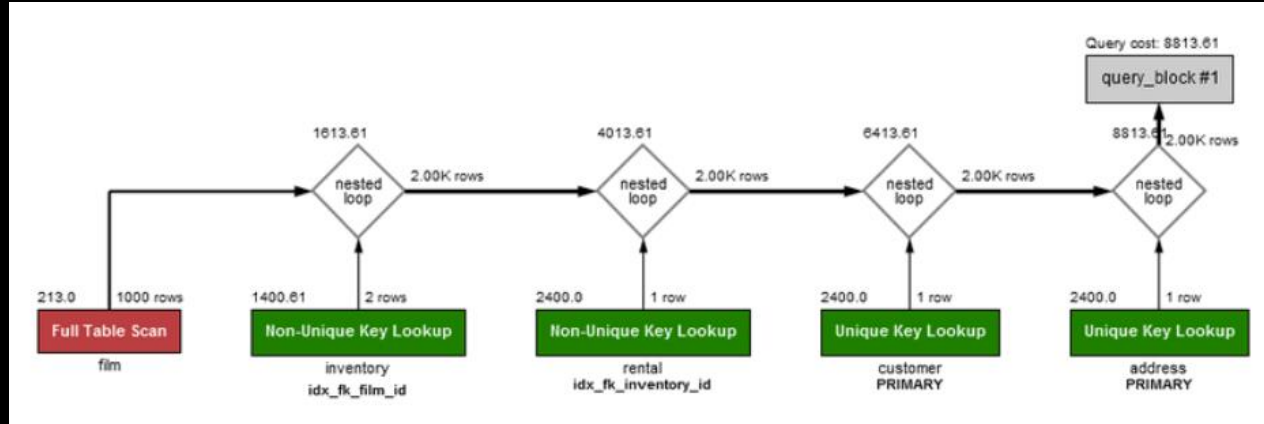


Joins are Loops

You notice how we loop through the keys as we manually create our joins. This is what our database must do as well. Nested joins turn into nested loops. Here is a typical query from a classic film rental database.

```
SELECT CONCAT(customer.last_name, ', ', customer.first_name)
  AS customer, address.phone, film.title FROM rental
INNER JOIN customer ON rental.customer_id = customer.customer_id
INNER JOIN address ON customer.address_id = address.address_id
INNER JOIN inventory ON rental.inventory_id = inventory.inventory_id
INNER JOIN film ON inventory.film_id = film.film_id
WHERE rental.return_date IS NULL
AND rental_date + INTERVAL film.rental_duration DAY < CURRENT_DATE()
LIMIT 5;
```

This is how MySQL workbench explains the operations.



3) (2 points) If the following code is run on 4 PEs, what will be the output?

```
from mpi4py import MPI
my_pe_num = MPI.COMM_WORLD.Get_rank()

message = 2

if my_pe_num == 0:
    MPI.COMM_WORLD.send(message, dest=1)

message_received = MPI.COMM_WORLD.recv(source=0)

if message_received == my_pe_num:
    print("PE ", my_pe_num, " got the message.")
else:
    print("I did not get the message.")
```

I did not get the message.

Analysis of “Synchronization”

The best way to make sure that you understand what is happening in the code above is to look at things from the perspective of each PE in turn. THIS IS THE WAY TO DEBUG ANY MESSAGE-PASSING CODE.

Follow from the top to the bottom of the code as PE 0, and do likewise for PE 1. See exactly where one PE is dependent on another to proceed. Look at each PEs progress as though it is 100 times faster or slower than the other nodes. Would this affect the final program flow? It shouldn't unless you made assumptions that are not always valid.

Exam Question 4

4) (3 points) Given the following two RDDs:

```
>>> table1.collect()
```

```
Out: [('A', 1), ('B', 2), ('C', 3), ('D', 4)]
```

```
>>> table2.collect()
```

```
Out: [('A', 11), ('B', 22), ('X', 33), ('Y', 44)]
```

and we join them with

```
>>> table3 = table1.join(table2)
```

What will be the result of the following?

```
table3.collect()
```

```
table1.collect()
```

```
[('A', 1), ('B', 2), ('C', 3), ('D', 4)]
```

```
table2.collect()
```

```
[('A', 11), ('B', 22), ('X', 33), ('Y', 44)]
```

```
table3 = table1.join(table2)
```

```
table3.collect()
```

```
[('B', (2, 22)), ('A', (1, 11))]
```

Exam Question 5

5) (1 point) Given the following Spark code, which reads in lines from a movie script:

```
lines_rdd = sc.textFile("alien.txt")  
parse_rdd = lines_rdd.map(lambda x: x.split())
```

parse_rdd will have

a) more

b) fewer

c) equal

d) can't tell

number of elements as lines_rdd. Circle your answer.

Common Transformations

Transformation	Result	
map(func)	Return a new RDD by passing each element through <i>func</i> .	Same Size
filter(func)	Return a new RDD by selecting the elements for which <i>func</i> returns true.	Fewer Elements
flatMap(func)	<i>func</i> can return multiple items, and generate a sequence, allowing us to “flatten” nested entries (JSON) into a list.	More Elements
distinct()	Return an RDD with only distinct entries.	
sample(...)	Various options to create a subset of the RDD.	
union(RDD)	Return a union of the RDDs.	
intersection(RDD)	Return an intersection of the RDDs.	
subtract(RDD)	Remove argument RDD from other.	
cartesian(RDD)	Cartesian product of the RDDs.	
parallelize(list)	Create an RDD from this (Python) list (using a spark context).	

Full list at <http://spark.apache.org/docs/latest/api/python/reference/api/pyspark.RDD.html#pyspark.RDD>

6) (2 points) What is the difference between the Linux commands *cd*, *cp* and *scp*? Very briefly describe all three.

cd	change directory
cp	copy
scp	copy between different computers

Unix commands

You can get very far with a relatively small number of *Unix* (the umbrella term for the family of operating systems that includes *Linux*) commands. Our philosophy will be to introduce them as needed.

You will also find countless help at your fingertips online should you feel the need. Google "how do I do x in linux" if you have any doubts.

The handful of commands that will do most of the work are

<code>ls</code>	list the directory contents
<code>cd</code>	change the directory you are in
<code>cp</code>	cp a file
<code>rm</code>	delete a file
<code>pwd</code>	show the current directory
<code>chmod</code>	change file permissions
<code>mkdir/rmdir</code>	make/remove directory

If you are put off by the cryptic names, I can't blame you. They do have a somewhat logical etymology, but I won't bore you with it here. As you will ultimately need so few commands, it won't really be a problem.

Online Question

Log onto Bridges from your laptop and grab some interactive PEs. Write an MPI code that will run on any number of PEs, and

- 1) Prints the sum of all of the PE numbers. On 4 PEs this would be $0+1+2+3 = 6$.
- 2) Takes the square root of that sum and has the PE closest to that number announce itself. That would be $\sqrt{6} = 2.44$, or PE 2 in this case.

It should function like the following example:

```
mpirun -n 10 python exam.py
```

```
The sum of PEs is: 45
```

```
I am PE 7 and I am closest to the square root of the sum
```

One of infinite solutions.

```
from mpi4py import MPI
import math

comm = MPI.COMM_WORLD
my_PE_num = comm.Get_rank()

sum = comm.allreduce( my_PE_num, op=MPI.SUM )

if my_PE_num == round(math.sqrt(sum)):
    print("The sum of PEs is:", sum)
    print("I am PE", my_PE_num, "and I am closest to the square root of the sum")
```

AI (at least GPT-5, Gemini and Claud) kept creating silly and needlessly elaborate solutions.

Deep Learning

Unprecedented Disruption

In the history of science, I defy you to find a similarly quick paradigm shift.

12 years ago

“Neural nets will enable real time ray tracing.”

“Neural nets will do protein folding.”

Science Fiction.

Word salad.

8 years ago

“Neural nets will do CFD.”

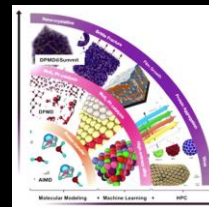
Well, maybe someday, but not soon.

Today

Neural net enabled algorithms are the best way to do protein folding.

Tomorrow

Skynet will kill us all. Or at least steal our jobs.



2024 Nobel in Chemistry

2024 Nobel in Physics
was AI too!

Why Now?

The ideas have been around for decades. Two components came together in the past 15 years to enable astounding progress:

Widespread parallel computing (GPUs)



Big data training sets



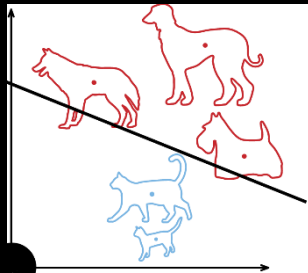
Two Perspectives

There are really two common ways to view the fundamentals of deep learning.

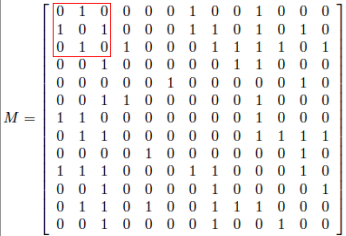
- Inspired by biological models.



- An evolution of classic ML techniques (the perceptron).

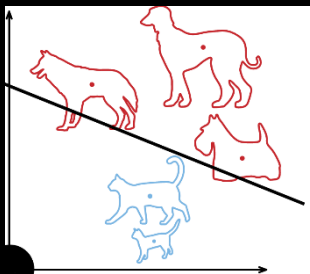


They are both fair and useful. We'll give each a thin slice of our attention before we move on to the actual implementation. You can decide which perspective works for you.



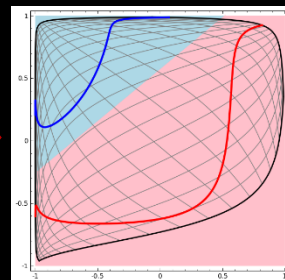
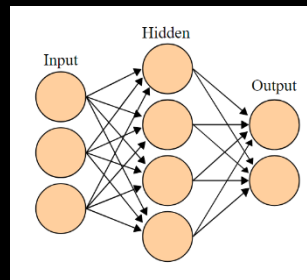
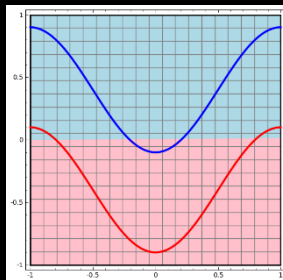
As a Highly Dimensional Non-linear Classifier

Perceptron



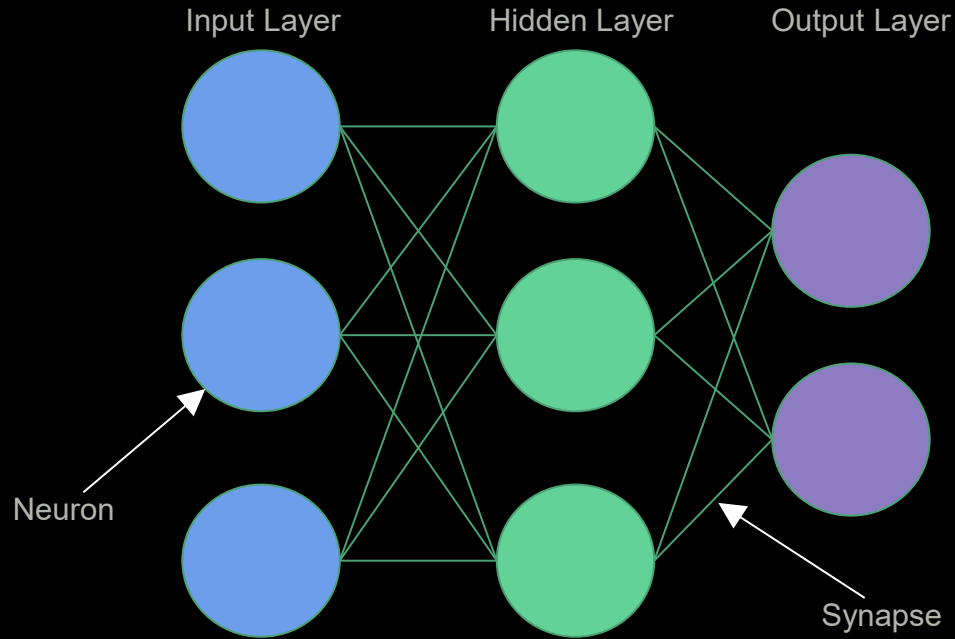
No Hidden Layer
Linear

Network



Hidden Layers
Nonlinear

Basic NN Architecture



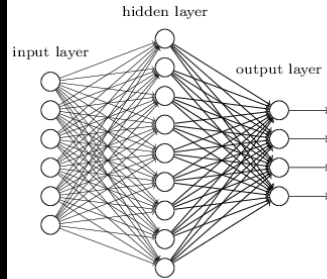
In Practice

How many inputs?

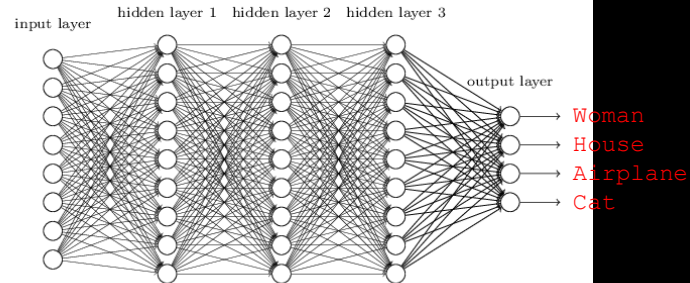


For an image it could be one (or 3) per pixel.

"Non-deep" feedforward neural network



Deep neural network



How deep?

100+ layers have become common.

How many outputs?

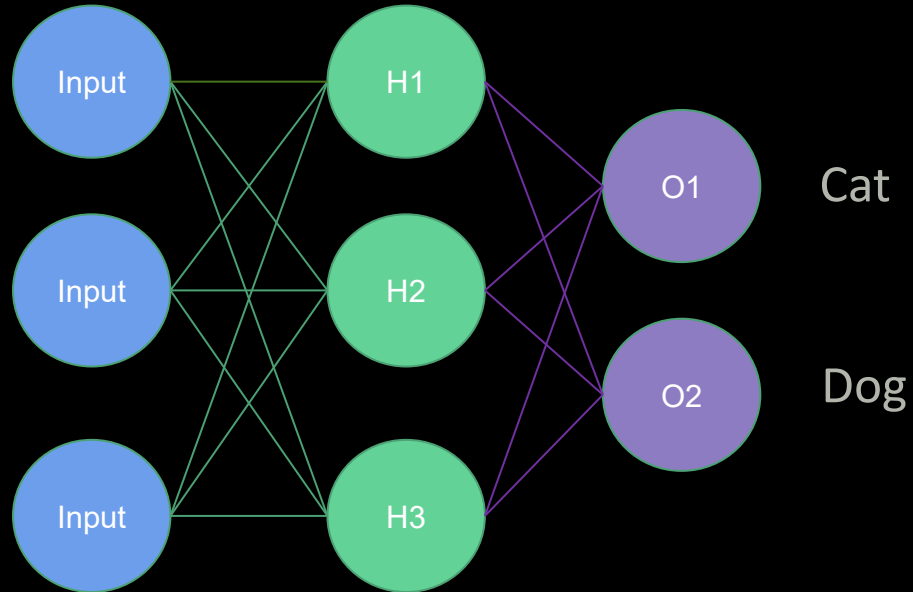


Might be an entire image.

Or could be discreet set of classification possibilities.

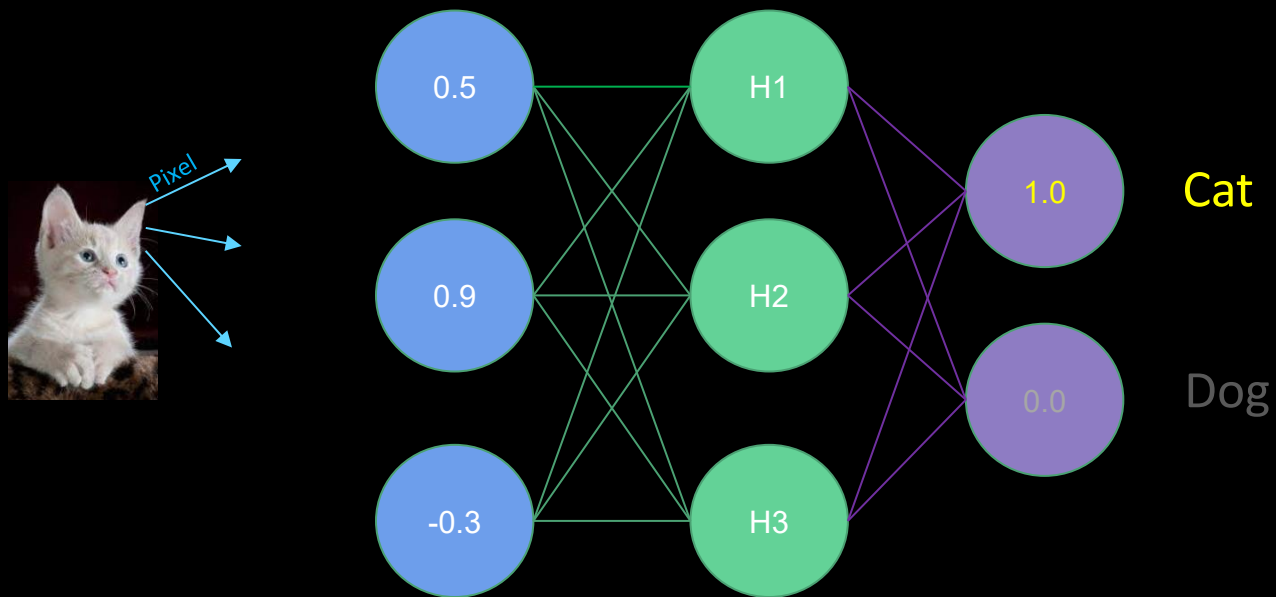
Inference

The "forward" or thinking step



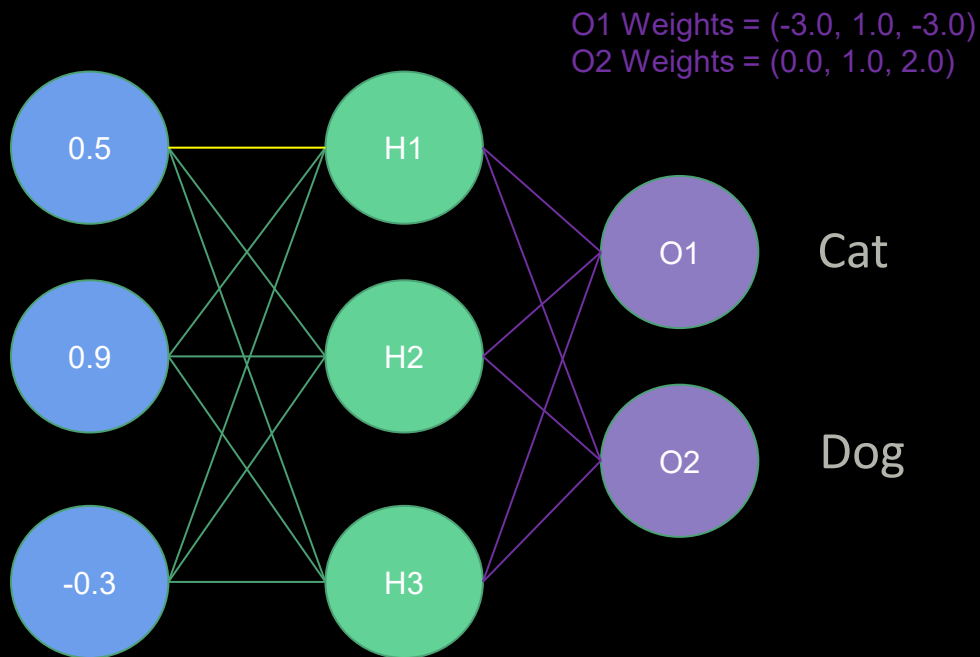
Inference

Input and Output Layers



Inference

Weights or Parameters

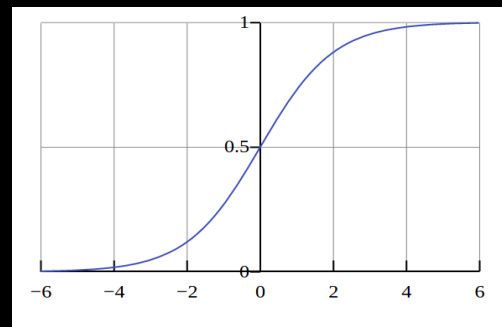


H1 Weights = (1.0, -2.0, 2.0)
H2 Weights = (2.0, 1.0, -4.0)
H3 Weights = (1.0, -1.0, 0.0)

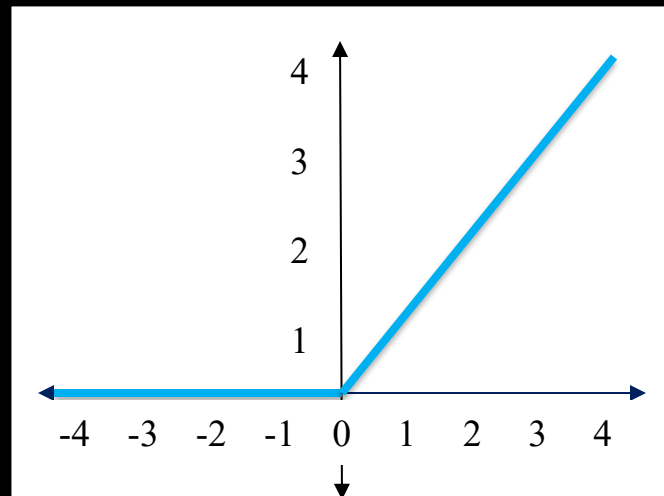
Activation Function

Neurons apply activation functions at these summed inputs. Activation functions are typically non-linear. There are countless possibilities. In reality, there are really only a few popular families:

- The **Sigmoid** function produces a value between 0 and 1, so it is intuitive when a probability is desired, and was almost standard for many years.
- The **Rectified Linear** activation function is zero when the input is negative and is equal to the input when the input is positive. Rectified Linear activation functions are currently the most popular activation function as they are more efficient than the sigmoid or hyperbolic tangent.
 - Sparse activation: In a randomly initialized network, only 50% of hidden units are active.
 - Better gradient propagation: Fewer vanishing gradient problems compared to sigmoidal activation functions that saturate in both directions.
 - Efficient computation: Only comparison, maybe addition and multiplication for variants.
 - There are **Leaky** and **Noisy** variants.

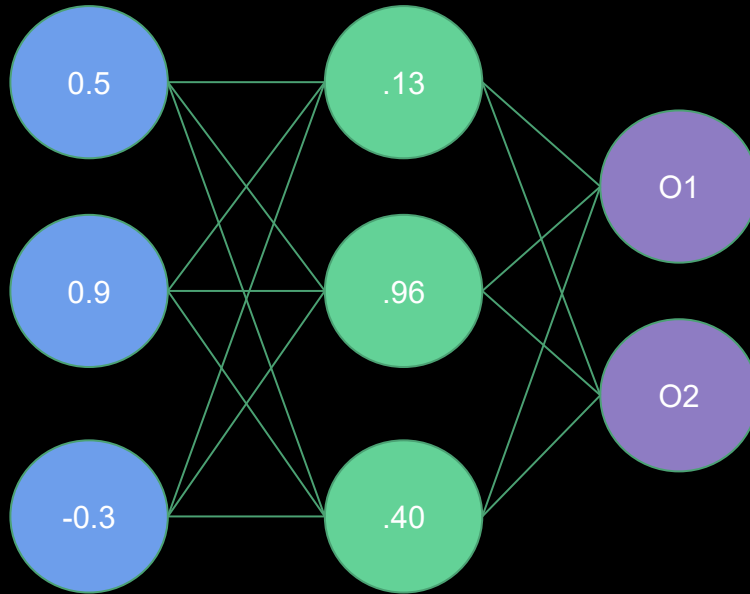


$$S(t) = \frac{1}{1 + e^{-t}}$$



Inference

Multiply, Add, do something non-linear.



H1 Weights = (1.0, -2.0, 2.0)

H2 Weights = (2.0, 1.0, -4.0)

H3 Weights = (1.0, -1.0, 0.0)

O1 Weights = (-3.0, 1.0, -3.0)

O2 Weights = (0.0, 1.0, 2.0)

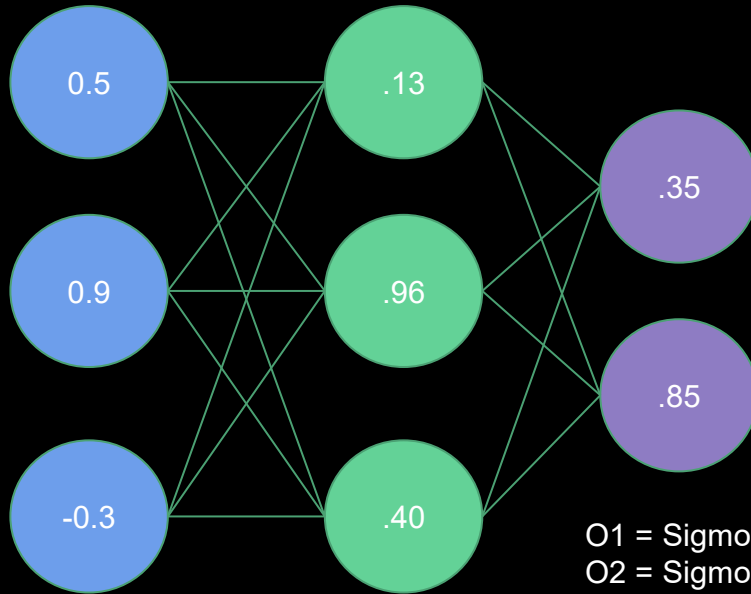
$$H1 = \text{Sigmoid}(0.5 * 1.0 + 0.9 * -2.0 + -0.3 * 2.0) = \text{Sigmoid}(-1.9) = .13$$

$$H2 = \text{Sigmoid}(0.5 * 2.0 + 0.9 * 1.0 + -0.3 * -4.0) = \text{Sigmoid}(3.1) = .96$$

$$H3 = \text{Sigmoid}(0.5 * 1.0 + 0.9 * -1.0 + -0.3 * 0.0) = \text{Sigmoid}(-0.4) = .40$$

Inference

Then do it again.



H1 Weights = (1.0, -2.0, 2.0)

H2 Weights = (2.0, 1.0, -4.0)

H3 Weights = (1.0, -1.0, 0.0)

O1 Weights = (-3.0, 1.0, -3.0)

O2 Weights = (0.0, 1.0, 2.0)

$$O1 = \text{Sigmoid}(.13 * -3.0 + .96 * 1.0 + .40 * -3.0) = \text{Sigmoid}(-.63) = .35$$

$$O2 = \text{Sigmoid}(.13 * 0.0 + .96 * 1.0 + .40 * 2.0) = \text{Sigmoid}(1.76) = .85$$

As A Matrix Operation

H1 Weights = (1.0, -2.0, 2.0)

H2 Weights = (2.0, 1.0, -4.0)

H3 Weights = (1.0, -1.0, 0.0)

$$\text{Sig}\left(\begin{array}{|c|c|c|} \hline & \text{Hidden Layer Weights} & \\ \hline 1.0 & -2.0 & 2.0 \\ \hline 2.0 & 1.0 & -4.0 \\ \hline 1.0 & -1.0 & 0.0 \\ \hline \end{array} * \begin{array}{|c|} \hline \text{Inputs} \\ \hline 0.5 \\ \hline 0.9 \\ \hline -0.3 \\ \hline \end{array} \right) = \text{Sig}\left(\begin{array}{|c|c|c|} \hline -1.9 & 3.1 & -0.4 \\ \hline \end{array} \right) = \begin{array}{|c|c|c|} \hline \text{Hidden Layer Outputs} \\ \hline .13 & .96 & 0.4 \\ \hline \end{array}$$

Now this looks like something that we can pump through a GPU.

Biases

It is also very useful to be able to offset our inputs by some constant. You can think of this as centering the activation function, or translating the solution (next slide). We will call this constant the *bias*, and it there will often be one value per layer.

Our math for the previously calculated layer now looks like this with *bias=0.1*:

$$\text{Sig}\left(\begin{array}{|c|c|c|} \hline \text{Hidden Layer Weights} & & \\ \hline 1.0 & -2.0 & 2.0 \\ \hline 2.0 & 1.0 & -4.0 \\ \hline 1.0 & -1.0 & 0.0 \\ \hline \end{array} * \begin{array}{|c|} \hline \text{Inputs} \\ \hline 0.5 \\ \hline 0.9 \\ \hline -0.3 \\ \hline \end{array} + \begin{array}{|c|} \hline \text{Bias} \\ \hline 0.1 \\ \hline 0.1 \\ \hline 0.1 \\ \hline \end{array} \right) = \text{Sig}\left(\begin{array}{|c|c|c|} \hline -1.8 & 3.2 & -0.3 \\ \hline \end{array} \right) = \begin{array}{|c|c|c|} \hline \text{Hidden Layer Outputs} \\ \hline .14 & .96 & 0.4 \\ \hline \end{array}$$

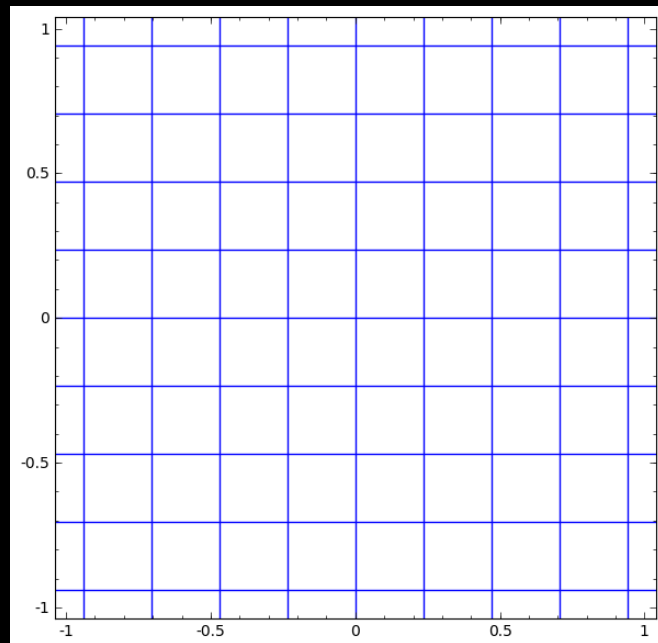
Linear + Nonlinear

The magic formula for a neural net is that, at each layer, we apply linear operations (which look naturally like linear algebra matrix operations) and then pipe the final result through some kind of final nonlinear **activation function**. The combination of the two allows us to do very general transforms.

The matrix multiply provides the *skew*, *rotation* and *scale*.

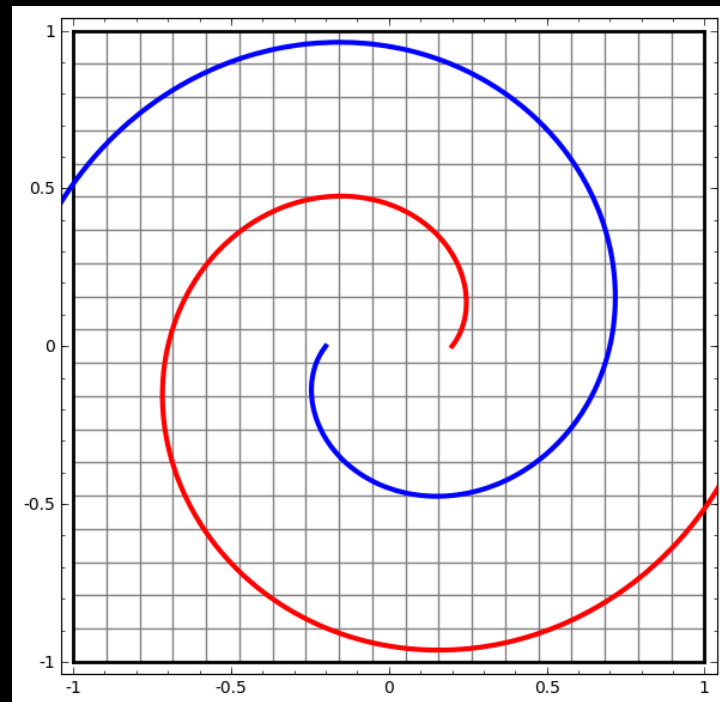
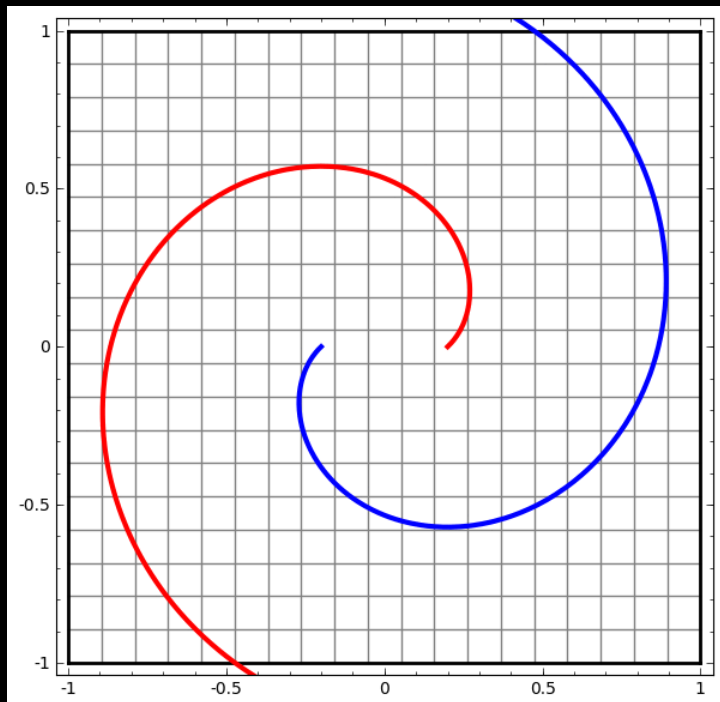
The bias provides the *translation*.

The activation function provides the *warp*.



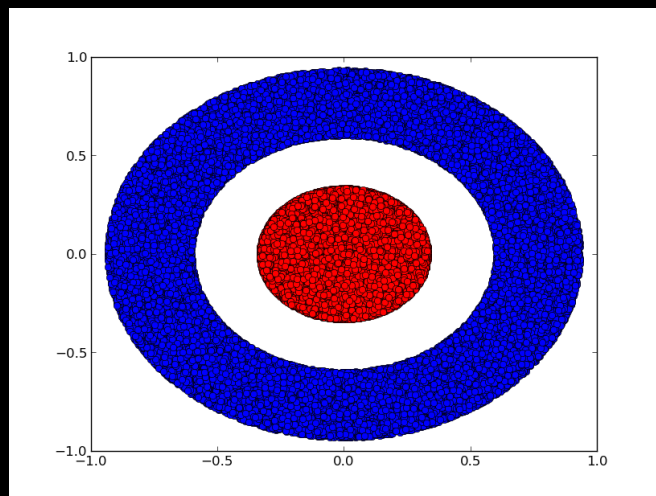
Linear + Nonlinear

These are two very simple networks untangling spirals. Note that the second does not succeed. With more substantial networks these would both be trivial.



Width of Network

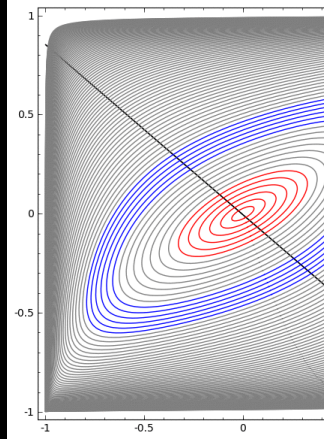
A very underappreciated fact about networks is that the width of any layer determines how many dimensions it can work in. This is valuable even for lower dimension problems. How about trying to classify (separate) this dataset:



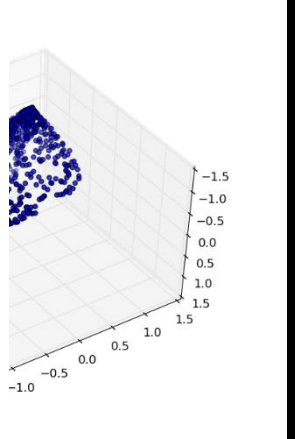
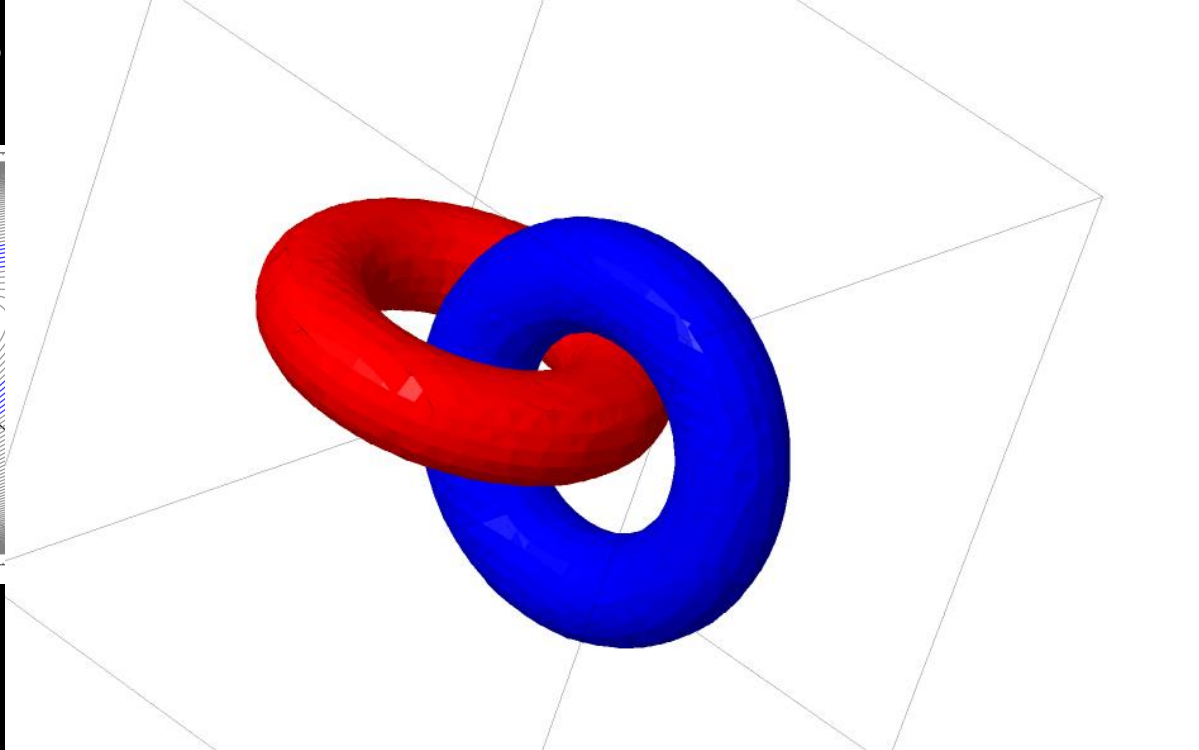
Can a neural net do this with twisting and deforming? What good does it do to have more than two dimensions with a 2D dataset?

Working In Higher Dimensions

It takes at least 3



Trying



s in 3D

Greater depth allows us to stack these operations, and can be very effective. The gains from depth are harder to characterize.

Theoretically

Universal Approximation Theorem: A 1-hidden-layer feedforward network of this type can approximate any function¹, given enough width².

Not really that useful as:

- Width could be enormous.
- Doesn't tell us how to find the correct weights.

1) Borel measurable. Basically, mostly continuous and bounded.

2) Could be exponential number of hidden units, with one unit required for each distinguishable input configuration.

Deep Learning

John Urbanic

Parallel Computing Scientist
Pittsburgh Supercomputing Center

Distinguished Service Professor
Carnegie Mellon University

But First:

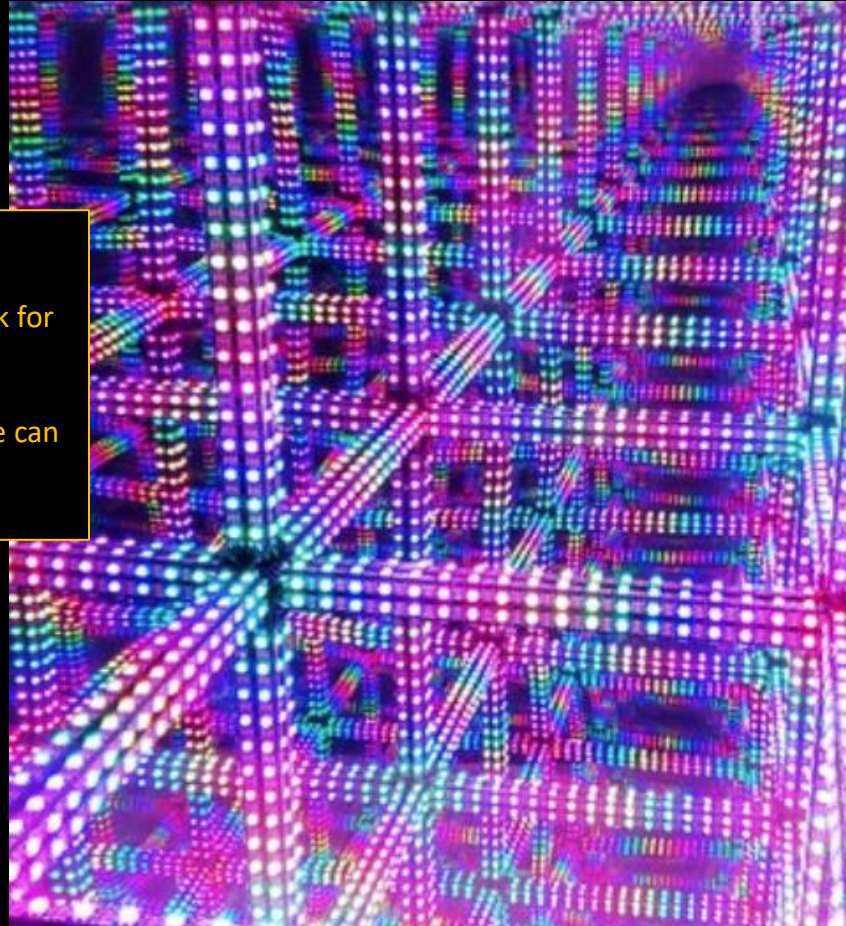
A Few Alternative Assignment #4 Solutions

The Big Reveal: What was in our data?

Oh yeah, it's 6 dimensional.

We are going to have to work for it.

Let's dive in and see what we can peel apart.



First dive: How many "things" are there?

```
input_rdd = sc.textFile('space.dat').map(lambda x:x.split(' ')).map(lambda x: [float(x[0]),float(x[1]),float(x[2]),float(x[3]),  
float(x[4]),float(x[5])])
```

```
from pyspark.mllib.clustering import KMeans
```

```
for clusters in range(1,12):  
    model = KMeans.train(input_rdd, clusters)  
    print (model.computeCost(input_rdd))
```

```
41842175.37560895  
16378806.48358392  
8717197.09771713  
825460.43393024  
657602.42742217  
561272.63826808  
450556.17011665  
446049.61204343  
346930.93273397  
336121.05005578  
287024.81939476
```

Looks like probably 4
clusters.

What can we say about these four objects?

```
model = KMeans.train(input_rdd, 4)
centers = model.clusterCenters
print(centers)
```

```
[array([60.01009462, 60.00480883, 60.00208445, 60.00042102, 59.99751545, 60.01716693]),
 array([30.01678989, 29.99626696, 29.98402263, 30.02258272, 30.0312735 , 30.00558741]),
 array([20.00492953, 29.99535658, 40.00856975, 50.00785294, 59.99358006, 70.00478914]),
 array([82.33292803, 82.33292803, 82.33292803, 82.33292803, 82.33292803, 82.33292803])]
```

Oddly nice
coordinates. Can't be a
coincidence.

```
pred = model.predict(input_rdd)
clusters = input_rdd.zip(pred)
```

```
clusters.take(3)
```

```
[([57.34269207756787, 58.64539091444486, 59.79934641688323, 60.17343791292141, 60.96727212965308, 56.828910332940936], 0),
 ([19.05241890791453, 29.58328818061184, 39.44842044075715, 49.90320343211805, 59.93659696752646, 70.51107954647642], 2),
 ([30.04646069048952, 34.13199328972533, 32.09774274747985, 30.78441105101962, 23.97459984798748, 31.068679127951953], 1)]
```

PCA might tell me

```
from pyspark.ml.feature import VectorAssembler
from pyspark.ml.feature import PCA

# For each of the labeled clusters: get a count, center cluster and do PCA
for cluster_num in range(4):
    cluster = clusters.filter(lambda x: x[1]==cluster_num).map(lambda x: x[0])
    print("Cluster ",cluster_num," has count ",cluster.count())
    center = [float(x) for x in centers[cluster_num]]
    centered_cluster = cluster.map(lambda x: [x[0]-center[0],x[1]-center[1],x[2]-center[2],x[3]-center[3],x[4]-center[4],x[5]-center[5]])
    df = spark.createDataFrame(centered_cluster)
    assembler = VectorAssembler(inputCols=['_1', '_2', '_3', '_4', '_5', '_6'], outputCol='feat_vec')
    ndf = assembler.transform(df)
    pca = PCA(k=6, inputCol="feat_vec", outputCol="pca_features")
    model = pca.fit(ndf)
    print("Cluster: ",cluster_num," explainedVariance: ", model.explainedVariance)
```

```
Cluster 0 has count 4000
Cluster: 0 explainedVariance: [0.94113732918, 0.058868670817, 2.22657241405e-16, 2.9772933143e-17, 2.267209539e-17, 8.054333993e-18]
Cluster 1 has count 8000
Cluster: 1 explainedVariance: [0.33939903609, 0.335704999885, 0.324901151697553, 1.8946473678e-16, 1.684557658e-16, 4.804174054e-17]
Cluster 2 has count 7000
Cluster: 2 explainedVariance: [0.17241654581, 0.169181472622, 0.167480619270236, 0.16498932332964, 0.1638456433717, 0.1620864448797]
Cluster 3 has count 2000
Cluster: 3 explainedVariance: [0.9999999999999999,6.515852244396576e-17,0.0,0.0,0.0,0.0]
```

Why DataFrames?

The DataFrame PCA offered me some useful functionality that I would have had to write myself for the RDD version of PCA, notably the explained variance.

As DataFrame APIs are picky about datatypes, I just went with the VectorAssembler helper routine to create my inputs.

We just include all our columns here, but you should know about this for when you are constructing more selective input features in your own complex applications.

Again, these cluster sizes aren't a coincidence. The instructor is really leaving some guideposts!

What do we know so far?

```
Cluster 0 has count 4000
Cluster: 0 explainedVariance: [0.94113732918, 0.058868670817, 2.22657241405e-16, 2.9772933143e-17, 2.267209539e-17, 8.054333993e-18]
Cluster 1 has count 8000
Cluster: 1 explainedVariance: [0.33939903609, 0.335704999885, 0.324901151697553, 1.8946473678e-16, 1.684557658e-16, 4.804174054e-17]
Cluster 2 has count 7000
Cluster: 2 explainedVariance: [0.17241654581, 0.169181472622, 0.167480619270236, 0.16498932332964, 0.1638456433717, 0.1620864448797]
Cluster 3 has count 2000
Cluster: 3 explainedVariance: [0.9999999999999999, 6.515852244396576e-17, 0.0, 0.0, 0.0, 0.0]
```

Looks like cluster 0 is 4000 points in a 2 dimensional shape, roughly twice as long as wide. Rectangle? The letter "I"? An ellipsoid?

Looks like cluster 1 is 8000 points in a roughly equal 3 dimensional shape (ball, cube, dodecahedron?).

Looks like cluster 2 is 7000 points in a 6 dimensional shape with roughly equal dimensions.

Looks like cluster 3 is 2000 points in a very, very straight line.

What is there to say about a line?

```
cluster_num = 3
cluster = clusters.filter(lambda x: x[1]==cluster_num).map(lambda x: x[0])
start0 = cluster.map(lambda x: x[0]).reduce(lambda x,y: min(x,y))
end0    = cluster.map(lambda x: x[0]).reduce(lambda x,y: max(x,y))
print(start0,end0)
```

75.02536789608955 89.99528182722815

```
start1 = cluster.map(lambda x: x[1]).reduce(lambda x,y: min(x,y))
end1    = cluster.map(lambda x: x[1]).reduce(lambda x,y: max(x,y))
print(start1,end1)
```

75.02536789608955 89.99528182722815

...
...

It appears we have a line that goes from 75,75,75,75,75,75, to 90,90,90,90,90,90.

John really is playing nice. The round numbers for coordinates are unlikely to be accidents.

How about that 2 dimensional shape?

```
cluster_num = 0
cluster = clusters.filter(lambda x: x[1]==cluster_num).map(lambda x: x[0])
center = [float(x) for x in centers[cluster_num]]
centered_cluster = cluster.map(lambda x: [x[0]-center[0],x[1]-center[1],x[2]-center[2],x[3]-center[3],x[4]-center[4],x[5]-center[5]] )
df = spark.createDataFrame(centered_cluster)
assembler = VectorAssembler(inputCols=['_1', '_2', '_3', '_4', '_5', '_6'], outputCol='feat_vec')
ndf = assembler.transform(df)
```

```
ndf.show(truncate=False)
```

_1	_2	_3	_4	_5	_6	feat_vec
-2.667402544629951	-1.3594179128526207	-0.20273802912222294	0.17301689495227635	0.9697566771136508	-3.1882565983907867	[-2.667402544629951,-1.3594179128526207,-0.20273802912222294,0.17301689495227635,0.9697566771136508,-3.1882565983907867]
2.3685883962073078	0.8804974029660286	1.4612366472554044	0.8927642649829011	0.2919308435854404	7.792897830801728	[2.3685883962073078,0.8804974029660286,1.4612366472554044,0.8927642649829011,0.2919308435854404,7.792897830801728]
-1.3827101918784095	-0.30643110565547005	-1.6672412311842706	-1.1861605873030996	-0.903190153077702	-7.702508954953906	[-1.3827101918784095,-0.30643110565547005,-1.6672412311842706,-1.1861605873030996,-0.903190153077702,-7.702508954953906]
-1.0745983537008286	-0.3467567745294957	-0.8697128479149683	-0.5739091198855277	-0.3185307273439406	-4.336300242969045	[-1.0745983537008286,-0.3467567745294957,-0.8697128479149683,-0.5739091198855277,-0.3185307273439406,-4.336300242969045]
1.626737719286389	0.7282889358990801	0.5188842108003229	0.21728963155509717	-0.23570749914510003	3.475060226905775	[1.626737719286389,0.7282889358990801,0.5188842108003229,0.21728963155509717,-0.23570749914510003,3.475060226905775]
-0.7318569103384576	-0.29461954423625514	-0.3630098714907035	-0.20357849144730977	-0.010564175427703049	-2.0651865739678428	[-0.7318569103384576,-0.29461954423625514,-0.3630098714907035,-0.20357849144730977,-0.010564175427703049,-2.0651865739678428]
0.756446508977838	0.4248084735222084	-0.09702110408785813	-0.1752626401912991	-0.4140717858418981	0.3057558865608314	[0.756446508977838,0.4248084735222084,-0.09702110408785813,-0.1752626401912991,-0.4140717858418981,0.3057558865608314]
-0.06894297939096965	-0.17046674466611478	0.5255916004093848	0.43801646209432477	0.5027975301546164	1.9733717866283413	[-0.06894297939096965,-0.17046674466611478,0.5255916004093848,0.43801646209432477,0.5027975301546164,1.9733717866283413]

```
pca = PCA(k=2, inputCol="feat_vec", outputCol="pca_features")
model = pca.fit(ndf)
```

```
x = model.transform(ndf)
```

```
x.show()
```

_1	_2	_3	_4	_5	_6	feat_vec	pca_features
-2.667402544629951	-1.3594179128526207	-0.20273802912222294	0.17301689495227635	0.9697566771136508	-3.1882565983907867	[-2.667402544629951,-1.3594179128526207,-0.20273802912222294,0.17301689495227635,0.9697566771136508,-3.1882565983907867]	[3.99792783599977...]
2.3685883962073078	0.8804974029660286	1.4612366472554044	0.8927642649829011	0.2919308435854404	7.792897830801728	[2.3685883962073078,0.8804974029660286,1.4612366472554044,0.8927642649829011,0.2919308435854404,7.792897830801728]	[-8.3544351044819...]
-1.3827101918784095	-0.30643110565547005	-1.6672412311842706	-1.1861605873030996	-0.903190153077702	-7.702508954953906	[-1.3827101918784095,-0.30643110565547005,-1.6672412311842706,-1.1861605873030996,-0.903190153077702,-7.702508954953906]	[7.93026920130143...]
-1.0745983537008286	-0.3467567745294957	-0.8697128479149683	-0.5739091198855277	-0.3185307273439406	-4.336300242969045	[-1.0745983537008286,-0.3467567745294957,-0.8697128479149683,-0.5739091198855277,-0.3185307273439406,-4.336300242969045]	[4.565655499918546...]
1.626737719286389	0.7282889358990801	0.5188842108003229	0.21728963155509717	-0.23570749914510003	3.475060226905775	[1.626737719286389,0.7282889358990801,0.5188842108003229,0.21728963155509717,-0.23570749914510003,3.475060226905775]	[-3.9202788241248...]
-0.7318569103384576	-0.29461954423625514	-0.3630098714907035	-0.20357849144730977	-0.010564175427703049	-2.0651865739678428	[-0.7318569103384576,-0.29461954423625514,-0.3630098714907035,-0.20357849144730977,-0.010564175427703049,-2.0651865739678428]	[2.24956614887102...]

Just keep the transformed 2D data.

```
x = x.select('pca_features')
```

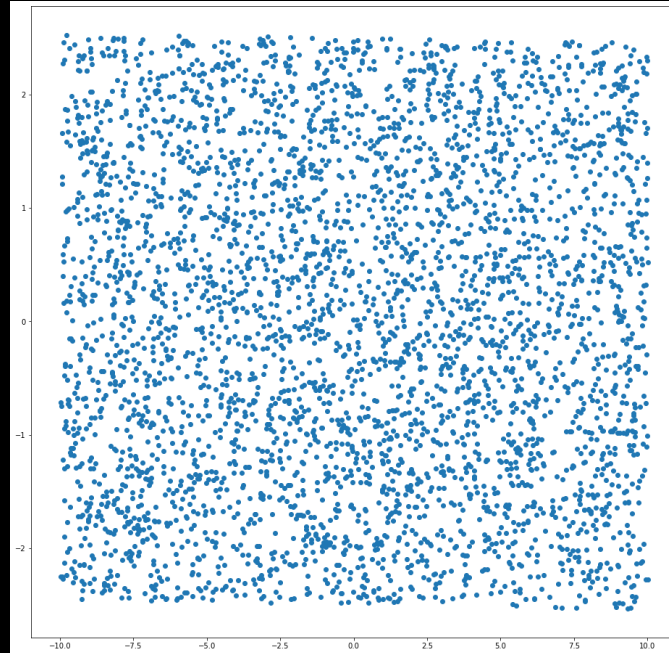
```
x.show(truncate=False)
```

```
+-----+
|pca_features|
+-----+
|[3.9979278359997767,2.038644677083587]|
|[-8.354435104481915,0.5793285278160654]|
|[7.9302692013014315,-1.8567942286966903]|
|[4.5656554991854685,-0.648479361558971]|
|[-3.9202788241248387,-0.5061141537196381]|
|[2.2495661488710215,-0.013961052571191623]|
|[-0.5543562542431459,-0.8663254982276529]|
|[-1.8872177194743192,1.0427501346465065]|
|[-4.486915242397211,-0.3468807523941083]|
|[-7.4052736944350634,-1.299952738245504]|
|[-0.14049532274150506,-1.978418672243316]|
|[7.726104669073028,-0.6729323819945661]|
|[-0.40348081955605264,-0.8689748431092859]|
|[0.7140708237672961,-0.9241435716199484]|
|[-2.2122075987007794,-0.13456773254652282]|
|[9.485710385357992,-1.4292877327108169]|
|[-0.8079428124435215,-1.7489314190287952]|
|[-8.419878570714548,0.8877420911700715]|
|[7.817855870622145,-1.7863909060630516]|
|[-0.6296300260742906,1.690450580612303]|
+-----+
```

only showing top 20 rows

```
y = x.rdd.map(lambda x: [x[0][0],x[0][1]])
z = y.map(lambda x: str(x[0])+', '+str(x[1]))
z = z.repartition(1)
z.saveAsTextFile('2D.dat')
```

A little matplotlib



Looks like a rectangle to me - if you pay attention to the axis scale, that is. So, putting it together, we have:

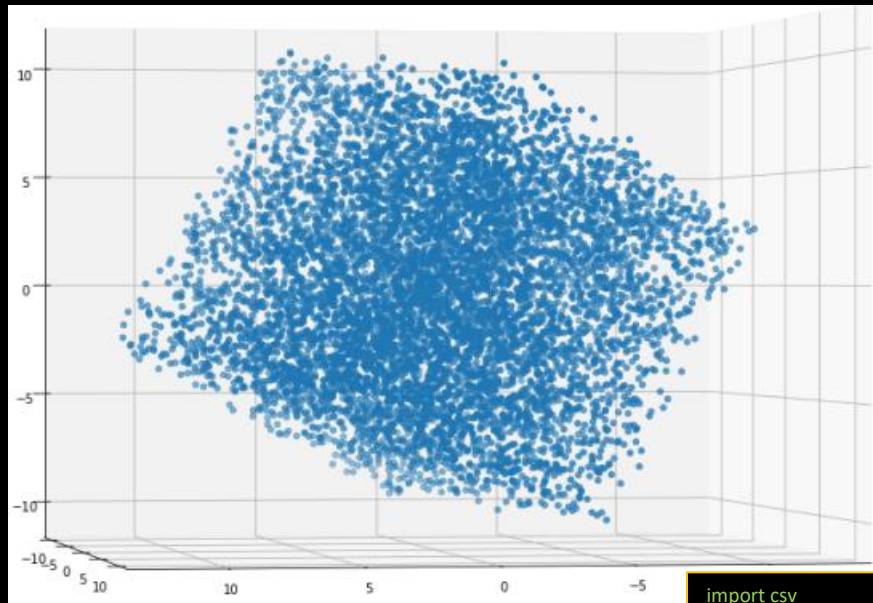
- 4000 points
- Rectangle
- About 5x20
- Centered at 60,60,60,60,60,60

we could even get the orientation from PCA (but visualizing a 6D rotation might not be useful to use here).

Let's try the same with the 3D cluster.

```
cluster_num = 1
cluster = clusters.filter(lambda x: x[1]==cluster_num).map(lambda x: x[0])
center = [float(x) for x in centers[cluster_num]]
centered_cluster = cluster.map(lambda x: [x[0]-center[0],x[1]-center[1],x[2]-center[2],x[3]-center[3],x[4]-center[4],x[5]-center[5]] )
df = spark.createDataFrame(centered_cluster)
assembler = VectorAssembler(inputCols=['_1', '_2', '_3', '_4', '_5', '_6'], outputCol='feat_vec')
ndf = assembler.transform(df)
pca = PCA(k=3, inputCol="feat_vec", outputCol="pca_features")
model = pca.fit(ndf)
x = model.transform(ndf).select('pca_features')
y = x.rdd.map(lambda x: [x[0][0],x[0][1],x[0][2]])
z = y.map(lambda x: str(x[0])+', '+str(x[1])+', '+str(x[2]))
z = z.repartition(1)
z.saveAsTextFile('3D.dat')
```

A little more matplotlib



This is a cube. I rotated the perspective a bit to convince myself. Putting

- 8000 points
- Cube
- About 15x15x15
- Centered at 30,30,30,30,30,30

Once again, we could even get the orientation if we wanted to.

```
import csv
```

My matplotlib

```
points = []
with open('3d.dat', newline='') as csvfile:
    spamreader = csv.reader(csvfile)
    for row in spamreader:
        points.append(( float(row[0]),float(row[1]),float(row[2]) ))
    print(row)
```

```
import matplotlib.pyplot as plt
```

```
fig = plt.figure(figsize=(16,16))
ax = fig.add_subplot(projection='3d')
ax.scatter([x[0] for x in points], [x[1] for x in points], [x[2] for x in points])
ax.view_init(elev=0,azim=80)
```

How about that 6 dimensional cluster?

We can't visualize. We can note that the explained variance here

0.1724, 0.1691, 0.1674, 0.1649, 0.1638, 0.1620

is similar to than that for our cube

0.3393, 0.3357, 0.3249

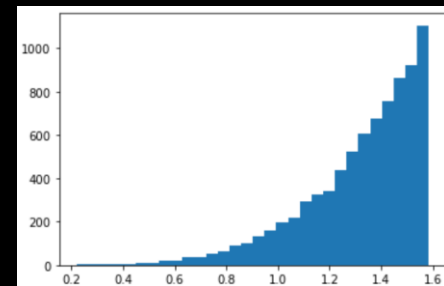
We might guess that this is rounder. So maybe it is a ball. But could it be a polyhedron? Or maybe a hollow 5D sphere or a gaussian distribution? Or maybe the "curse of dimensionality" has this effect on variance.

I did a histogram of point distances from the cluster center. We might hope that it would reveal the shape. In this case we would find a it follows a 5th power curve closely. That corresponds to a 6D ball. And we would find the cutoff/radius (15) as well.

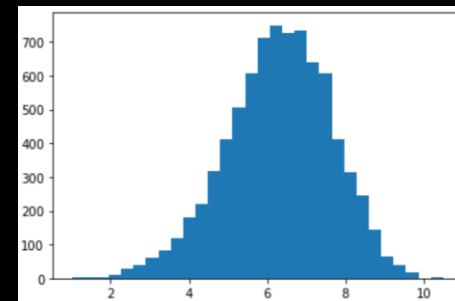
I don't expect that any of you went that far. If you guessed:

- 7000 points
- "Roundish" shape
- Centered at 20,30,40,50,60,70

Then congratulations. I think you did very well.



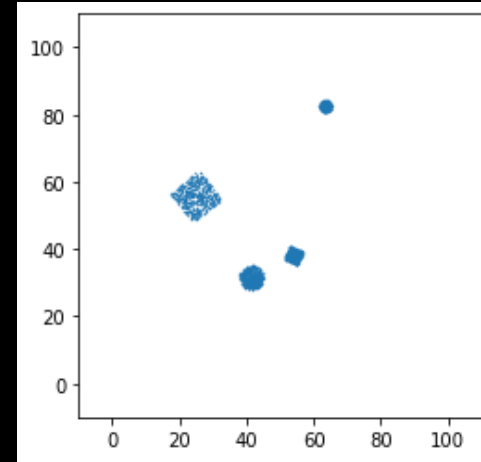
6D Ball



6D Cube

A quick discussion about noise and outliers.

KMeans (and most clustering approaches) have no notion of outliers. Every point gets assigned to a cluster. This introduces issues when faced with noise. Here is a simple dataset.

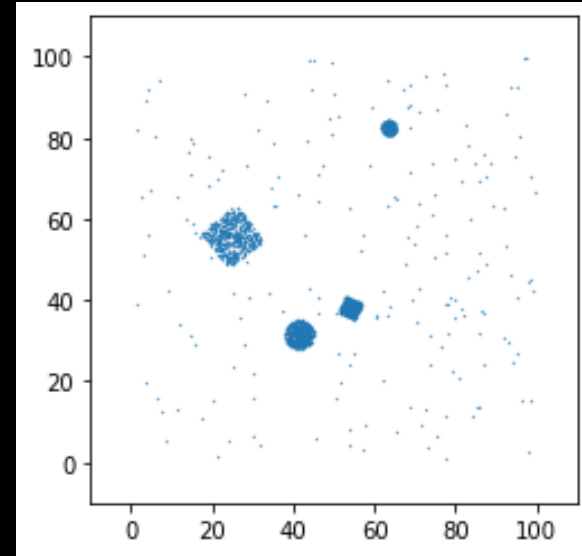


KMeans does well and gives us a sensible clustering.

Cost	Clusters
1212077	(45 , 52)
384888	(63 , 82) (40 , 41)
65392	(47 , 34) (24 , 55) (63 , 82)
15430	(24 , 55) (63 , 82) (53 , 38) (41 , 31)
11707	(53 , 38) (22 , 57) (63 , 82) (41 , 31) (26 , 54)
9401	(25 , 59) (63 , 82) (53 , 38) (27 , 53) (41 , 31) (21 , 55)
8313	(63 , 82) (53 , 38) (23 , 58) (40 , 30) (22 , 53) (28 , 55) (42 , 32)
7412	(22 , 53) (63 , 82) (53 , 38) (39 , 30) (23 , 58) (41 , 33) (28 , 55) (43 , 30)
7081	(63 , 82) (53 , 38) (28 , 56) (39 , 32) (24 , 51) (22 , 57) (43 , 30) (40 , 29) (41 , 33)

A little noise.

Add in a little noise (10%) and it is a little less obvious where our cost "elbow" is. But perhaps we are still OK.

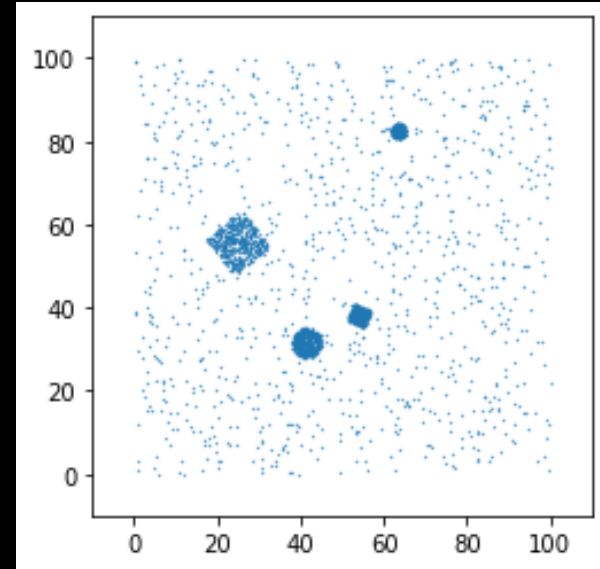


The cluster centers are still reasonable too.

Cost	Clusters
1542624	(46 , 51)
616768	(64 , 81) (40 , 41)
224474	(24 , 56) (48 , 34) (64 , 81)
145791	(40 , 30) (64 , 82) (24 , 56) (56 , 37)
104353	(24 , 56) (64 , 82) (54 , 38) (83 , 32) (40 , 30)
89475	(40 , 30) (63 , 82) (84 , 55) (24 , 56) (54 , 38) (79 , 17)
77963	(24 , 55) (63 , 82) (54 , 38) (40 , 30) (16 , 78) (84 , 55) (78 , 17)
68325	(41 , 31) (63 , 82) (21 , 60) (54 , 38) (25 , 54) (85 , 70) (17 , 16) (83 , 26)
63991	(54 , 38) (63 , 82) (24 , 56) (83 , 40) (17 , 10) (18 , 34) (41 , 31) (74 , 11) (87 , 75)

A little more noise.

If we increase the noise to 33% it becomes hard to be confident about the cluster count. My best guess might be 3...



And the cluster centers are no longer reliable.

Cost	clusters
2911714	(47 , 51)
1515577	(62 , 80) (40 , 38)
844979	(50 , 31) (66 , 80) (22 , 57)
638983	(45 , 32) (65 , 81) (22 , 58) (81 , 27)
492621	(23 , 60) (65 , 81) (19 , 17) (48 , 34) (81 , 26)
398572	(81 , 26) (66 , 81) (24 , 55) (19 , 85) (20 , 16) (48 , 34)
438669	(45 , 32) (79 , 17) (22 , 52) (65 , 83) (18 , 85) (53 , 62) (86 , 58)
301994	(43 , 11) (62 , 81) (13 , 20) (48 , 35) (87 , 73) (81 , 24) (24 , 55) (18 , 85)
289378	(23 , 60) (61 , 82) (41 , 31) (10 , 29) (82 , 49) (27 , 9) (54 , 38) (88 , 84) (78 , 14)

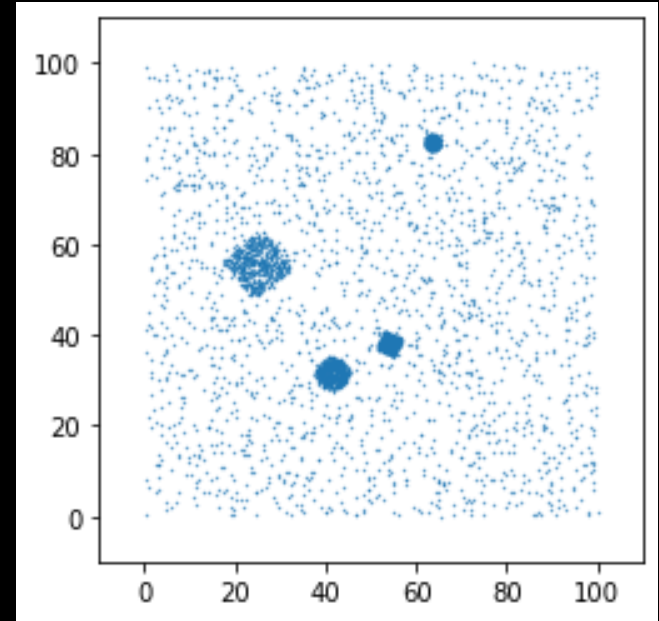
Significant noise.

As the S/N ratio hits 1, it becomes very hard to find any elbow.

The real world involves noise. Be aware of this limitation.

If there was an easy fix ("import super-k-means"), we would be using it.
There are solutions, but no magic bullets.

Even if we cheat and look at 4 clusters, the cluster centers are problematic.



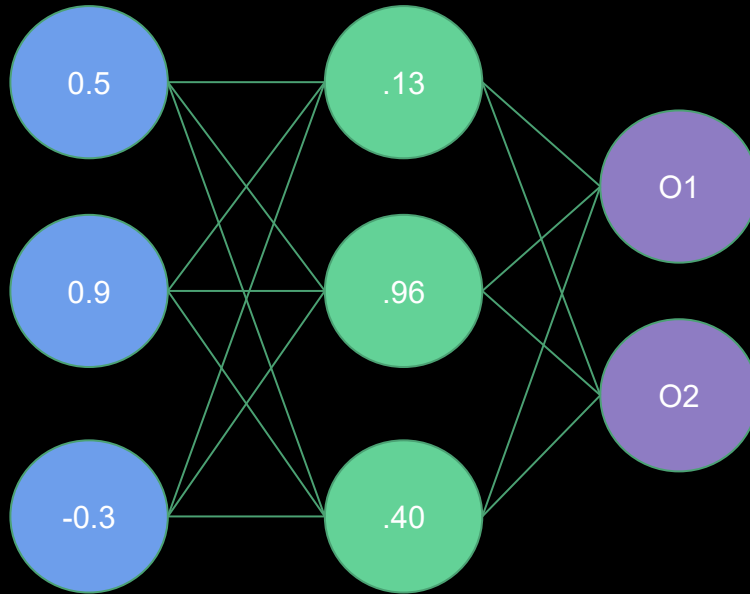
Cost	Clusters
4532592	(48 , 50)
2646052	(42 , 36) (60 , 79)
1553843	(21 , 58) (52 , 29) (67 , 79)
1122661	(22 , 61) (66 , 80) (80 , 26) (43 , 31)
981406	(79 , 54) (22 , 61) (43 , 30) (63 , 83) (79 , 15)
741845	(22 , 62) (80 , 18) (48 , 35) (60 , 82) (20 , 17) (84 , 67)
811270	(63 , 31) (22 , 54) (87 , 67) (35 , 26) (53 , 60) (20 , 84) (63 , 84)
586240	(22 , 54) (86 , 68) (55 , 39) (64 , 82) (12 , 83) (36 , 25) (81 , 17) (40 , 81)
462873	(48 , 35) (15 , 19) (47 , 10) (61 , 82) (83 , 15) (20 , 84) (23 , 55) (87 , 81) (78 , 49)

Back on track...



Inference

Multiply, Add, do something non-linear.



H1 Weights = (1.0, -2.0, 2.0)

H2 Weights = (2.0, 1.0, -4.0)

H3 Weights = (1.0, -1.0, 0.0)

O1 Weights = (-3.0, 1.0, -3.0)

O2 Weights = (0.0, 1.0, 2.0)

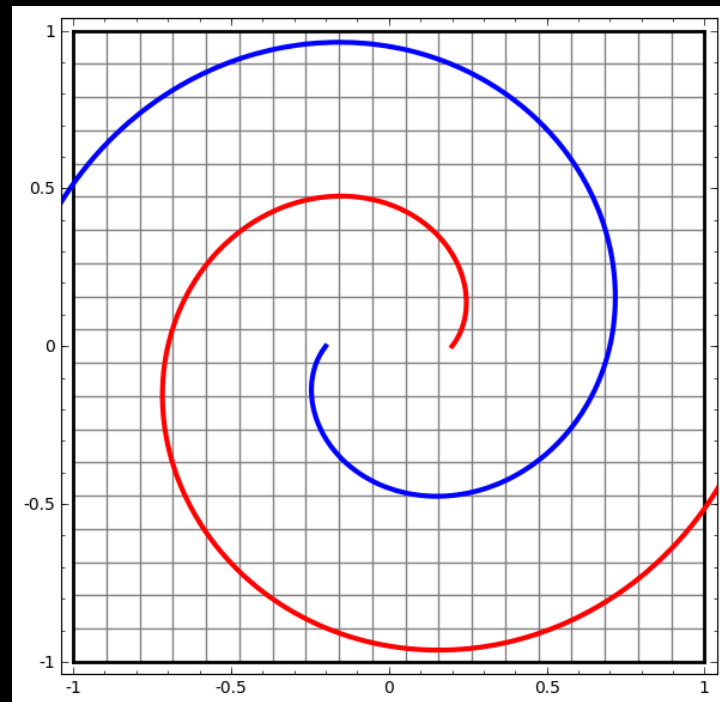
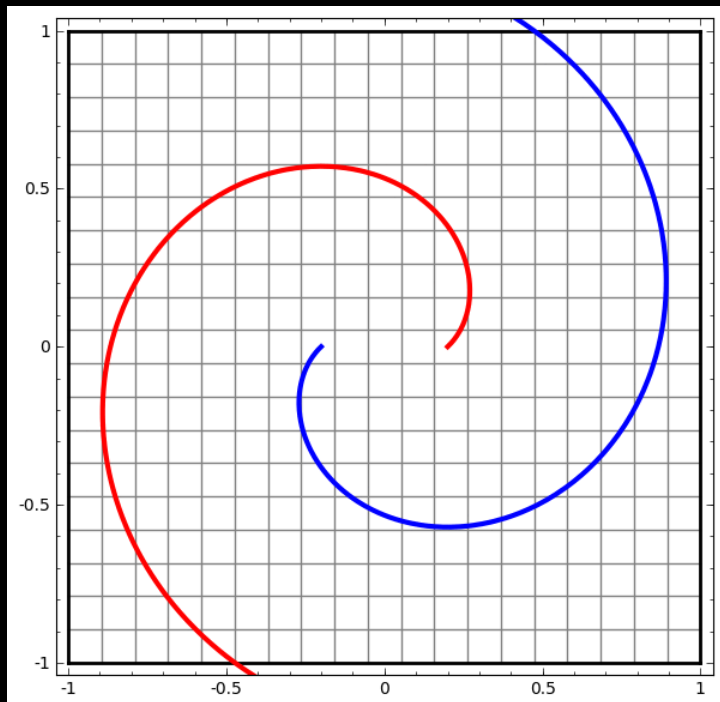
$$H1 = \text{Sigmoid}(0.5 * 1.0 + 0.9 * -2.0 + -0.3 * 2.0) = \text{Sigmoid}(-1.9) = .13$$

$$H2 = \text{Sigmoid}(0.5 * 2.0 + 0.9 * 1.0 + -0.3 * -4.0) = \text{Sigmoid}(3.1) = .96$$

$$H3 = \text{Sigmoid}(0.5 * 1.0 + 0.9 * -1.0 + -0.3 * 0.0) = \text{Sigmoid}(-0.4) = .40$$

Linear + Nonlinear

These are two very simple networks untangling spirals. Note that the second does not succeed. With more substantial networks these would both be trivial.



Theoretically

Universal Approximation Theorem: A 1-hidden-layer feedforward network of this type can approximate any function¹, given enough width².

Not really that useful as:

- Width could be enormous.
- Doesn't tell us how to find the correct weights.

1) Borel measurable. Basically, mostly continuous and bounded.

2) Could be exponential number of hidden units, with one unit required for each distinguishable input configuration.

How Do We Find These Magic Weights? Training!

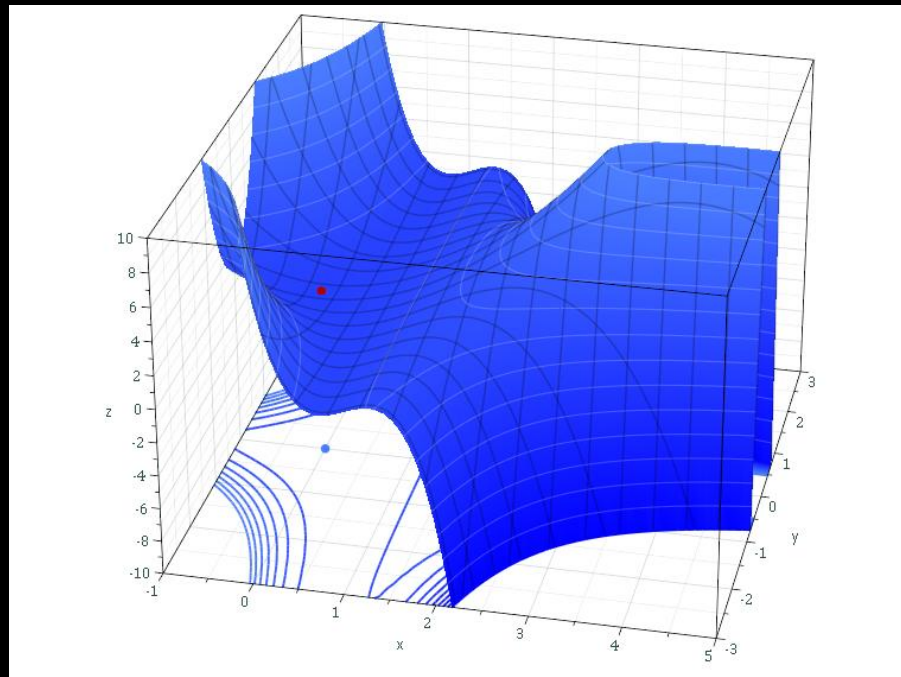
Our objective, mathematically, is really quite simple to state. *Given the loss as a function of weights, find the weight values that minimize the loss.*

If we had only two weights in our whole network, the picture would look like this.

We want to find the lowest point in our landscape.

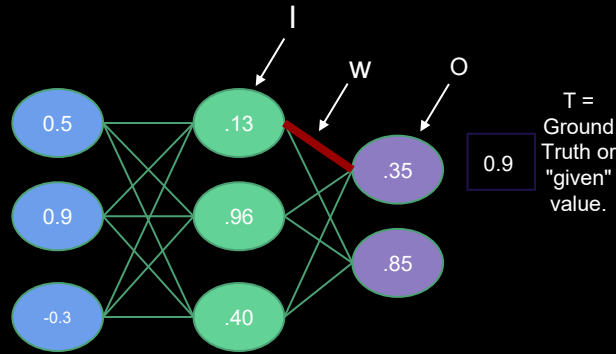
One obvious first step would be to descend the gradient from our current values. This should at least lead us to a better loss.

To do this gradient descent, we need some method of finding the gradients, $\frac{\partial E}{\partial w}$



Finding the gradients

For a tiny network, we could just write out the equations and use a little calculus. From our previous example we have all the math we need to find $\frac{\partial E}{\partial w}$ for one of the final weights.



$$\frac{\partial E}{\partial w} = I \cdot (O - T) \cdot O \cdot (1 - O)$$

$$\frac{\partial E}{\partial w} = .13 \cdot (.35 - .9) \cdot .35 \cdot (1 - .35)$$

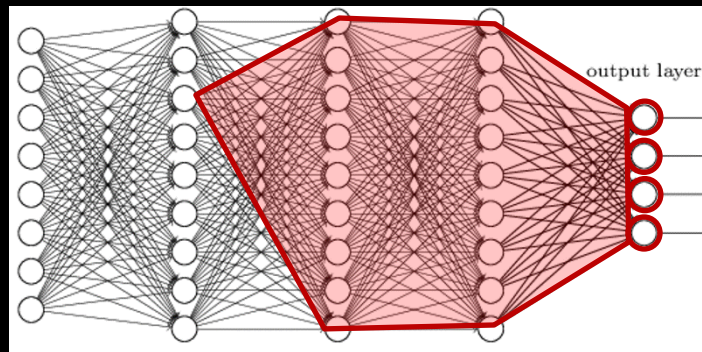
Using our Sigmoid activation function

For a tiny network, we could just write out the equations and use a little calculus. From our previous example we have all the math we need to find $\frac{\partial E}{\partial w}$ for one of the final weights.

With just this information, we have some idea of which way to adjust w . But, when we get deeper into a more realistic network, the expressions get more complex...

Back-Propagation

In a useful network, the chain rule results in a lot of factors for any given weight adjustment.



There are a lot of dependencies going on here. It isn't obvious that there is a viable way to do this in very large networks.

From the fantastic *Deep Learning*, Goodfellow, Bengio and Courville.

$$\frac{\partial u^{(n)}}{\partial u^{(j)}} = \sum_{\text{path}(u^{(\pi_1)}, u^{(\pi_2)}, \dots, u^{(\pi_t)}) \text{ from } \pi_1=j \text{ to } \pi_t=n} \prod_{k=2}^t \frac{\partial u^{(\pi_k)}}{\partial u^{(\pi_{k-1})}}.$$

Not as scary as it looks. There are really just two pieces. The sum represents all the possible paths, and the product operator is because each of the pieces along any path multiply each other.

Since the number of paths from one node to a distant node can grow exponentially in the length of these paths, the number of terms in the above sum, which is the number of such paths, can grow exponentially with depth. A large cost would be incurred because the same computation for the subfactors would be redone many times. To avoid such recomputation, back-propagation works as a table-filling algorithm that stores intermediate results and avoids repeating many common subexpressions. This is a version of *dynamic programming*, which you may have heard of.

Back-propagation Full Story

If you have 30 minutes, and remember freshman calculus, you can understand the complete details of the algorithm. I heartily recommend one of these.

An elegant perspective on this can be found from Chris Olah at
<http://colah.github.io/posts/2015-08-Backprop> .

With basic calculus you can readily work through the details. You can find an excellent explanation from the renowned *3Blue1Brown* at
<https://www.youtube.com/watch?v=Ilg3gGewQ5U> .

To be honest, many people are happy to leave the details to TensorFlow, or whatever package they are using. Just don't think it is beyond your understanding.

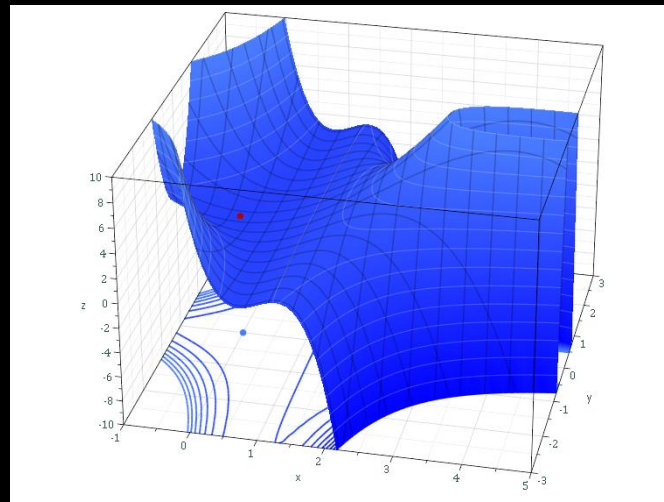
Gradient Descent

We now have gradients to push us in the right direction, and we can use the well-known numerical technique of *gradient descent*.

This is still a challenging situation, as our landscape has many local minimums that we could get trapped in on our way to finding a better one. We don't even hope to find the ideal global minimum.

Many variations have arisen that perform better for deep learning applications. TensorFlow will allow us to use these interchangeably; and we will.

Most interesting recent methods incorporate *momentum* to help get over a local minimum. Momentum and *step size* (or *learning rate*) are the two *hyperparameters* we will encounter later.



Again, just two weights.

We could/should find the current error for all the training data before updating the weights (an *epoch*). However it is usually much more efficient to use a *stochastic* approach, sampling a random subset of the data, updating the weights, and then repeating with another *mini-batch*. This is called *Stochastic Gradient Descent*.

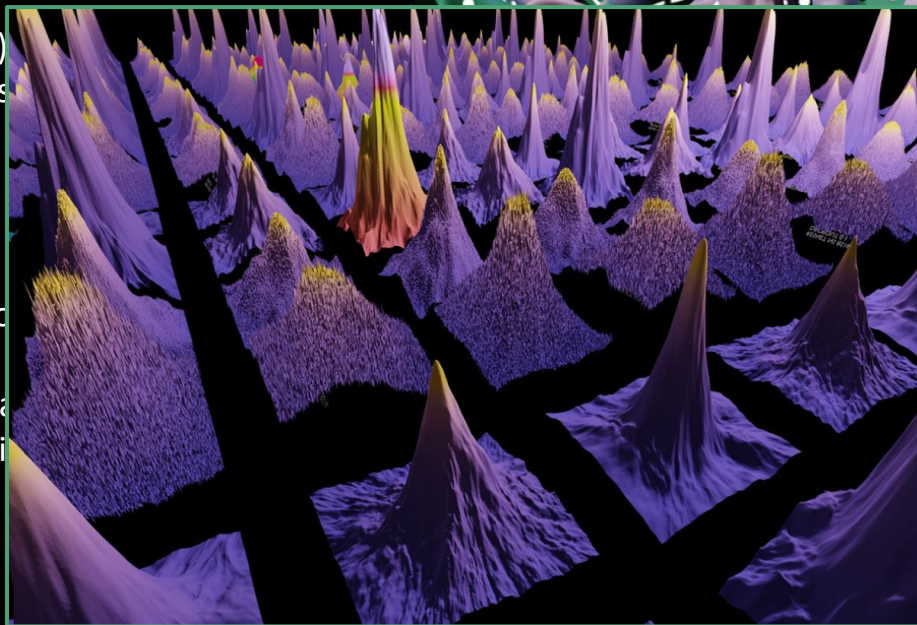
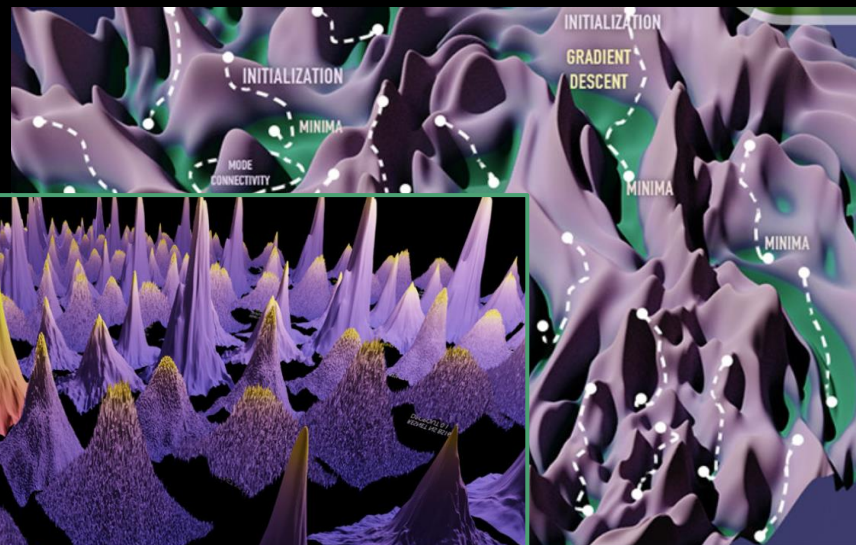
Problem Solved?

You should expect to struggle with these hyperparameters to get good performance. Why? Because these landscapes have many local minimums. For a long time this was considered an unsolvable obstacle.

For once, adding in more (many more!) helps us. There become many more "easy" valleys.

For the topologically inclined, we have more peaks than pits because we only need the Hessian to be negative, instead of all points.

Mini-batches also have an impact, as each batch explores a different landscape, shaking things up.



Visuals by Javier Ideami @ losslandscape.com

Go there!

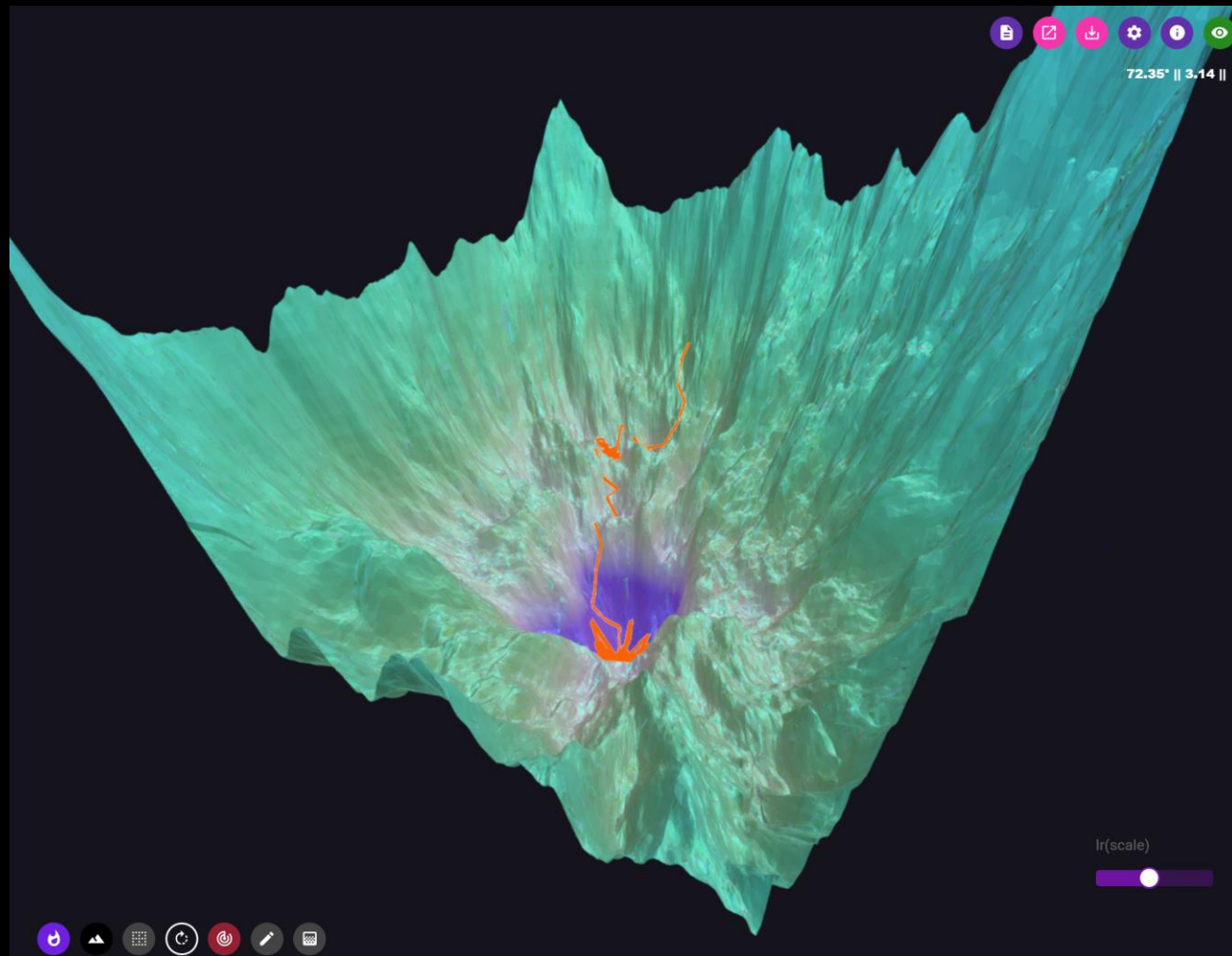
* This is not a rigorous assertion. A counterexample is protein folding. We know a very good approximate energy function, and it has many degrees of freedom, but you simply cannot obtain the folded structure by doing gradient descent.

Incredible Demo

We're going to try a live demo here, but you should visit this site and play around with it yourself. There is a lot of insight to be had.

<https://losslandscape.com/explorer>

Again, from the awesome
Javier Ideami @ losslandscape.com



Going To Play Along?

Make sure you are on a GPU node:

```
bridges2-login014% interact -gpu  
v001%
```

Load the TensorFlow 2 Container:

```
v001% singularity shell --nv /ocean/containers/ngc/tensorflow/tensorflow_23.04-tf2-py3.sif
```

And start TensorFlow:

```
Singularity> python  
Python 3.8.10 (default, Mar 13 2023, 10:26:41)  
[GCC 9.4.0] on linux  
Type "help", "copyright", "credits" or "license"  
>>> import tensorflow  
>>> ...some congratulatory noise...  
>>>
```

Two Other Ways To Play Along

From inside the container, and in the right example directory, run the python programs from the command line:

```
Singularity> python CNN_Dropout.py
```

or invoke them from within the python shell:

```
>>> exec(open("./CNN_Dropout.py").read())
```

TensorFlow or PyTorch or...?



TensorFlow

Install Learn API Resources More

TensorFlow Core v2.1.0

Overview Python JavaScript C++ Java

Input
Model
Sequential
activations
applications
backend
callbacks
constraints
datasets
estimator
experimental
initializers
layers

Overview
AbstractRNNCell
Activation
ActivityRegularization
Add
add
AdditiveAttention
AlphaDropout
Attention
Average
average
AveragePooling1D
AveragePooling2D
AveragePooling3D
BatchNormalization
Bidirectional
Concatenate
concatenate
Conv1D
Conv2D
Conv2DTranspose
Conv3D
Conv3DTranspose
ConvLSTM2D
Cropping1D
Cropping2D
Cropping3D
Dense
DenseFeatures
DepthwiseConv2D
deserialize
Dot
dot

Missed TensorFlow Dev Summit? Check out the video playlist. Watch recordings

TensorFlow > API > TensorFlow Core v2.1.0 > Python

☆☆☆☆

tf.keras.layers.Conv2D

✓ See Stable See Nightly

TensorFlow 1 version View source on GitHub

2D convolution layer (e.g. spatial convolution over images).

+ View aliases

```
tf.keras.layers.Conv2D(
    filters, kernel_size, strides=(1, 1), padding='valid', data_format=None,
    dilation_rate=(1, 1), activation=None, use_bias=True,
    kernel_initializer='glorot_uniform', bias_initializer='zeros',
    kernel_regularizer=None, bias_regularizer=None, activity_regularizer=None,
    kernel_constraint=None, bias_constraint=None, **kwargs
)
```

Used in the notebooks

Used in the guide	Used in the tutorials
The Keras functional API Migrate your TensorFlow 1 code to TensorFlow 2 Eager execution Train and evaluate with Keras Better performance with tf.function and AutoGraph	Custom layers Image classification Pix2Pix Convolutional Neural Network (CNN) Custom training with tf.distribute.Strategy

This layer creates a convolution kernel that is convolved with the layer input to produce a tensor of outputs. If `use_bias` is True, a bias vector is created and added to the outputs. Finally, if `activation` is not `None`, it is applied to the outputs as well.

When using this layer as the first layer in a model, provide the keyword argument `input_shape` (tuple of integers, does not include the sample axis), e.g. `input_shape=(128, 128, 3)` for

Documentation

The API is well documented.

That is terribly unusual.

Take advantage and keep a browser open as you develop.

MNIST

We now know enough to attempt a problem. Only because the TensorFlow framework, and the Keras API, fills in a lot of the details that we have glossed over. That is one of its functions.

Our problem will be character recognition. We will learn to read handwritten digits by training on a large set of 28x28 greyscale samples.



First we'll do this with the simplest possible model just to show how the TensorFlow framework functions. Then we will gradually implement our way to a quite sophisticated and accurate convolutional neural network for this same problem.

Getting Into MNIST

```
import tensorflow as tf

mnist = tf.keras.datasets.mnist
(train_images, train_labels), (test_images, test_labels) = mnist.load_data()

train_images = train_images.reshape(60000, 784)
test_images = test_images.reshape(10000, 784)

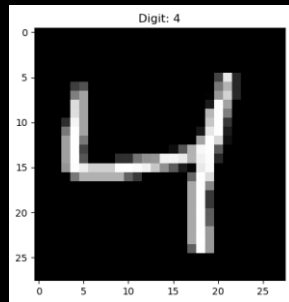
test_images = test_images.astype('float32')
train_images = train_images.astype('float32')

test_images /= 255
train_images /= 255
```

matplotlib bonus insight

```
import matplotlib.pyplot as plt

plt.imshow(train_images[2], cmap=plt.get_cmap('gray'),
            interpolation='none')
plt.title("Digit: {}".format(train_labels[2]))
```



Building a Network

John Urbanic

Parallel Computing Scientist
Pittsburgh Supercomputing Center

Distinguished Service Professor
Carnegie Mellon University

MNIST

We now know enough to attempt a problem. Only because the TensorFlow framework, and the Keras API, fills in a lot of the details that we have glossed over. That is one of its functions.

Our problem will be character recognition. We will learn to read handwritten digits by training on a large set of 28x28 greyscale samples.



First we'll do this with the simplest possible model just to show how the TensorFlow framework functions. Then we will gradually implement our way to a quite sophisticated and accurate convolutional neural network for this same problem.

Getting Into MNIST

```
import tensorflow as tf

mnist = tf.keras.datasets.mnist
(train_images, train_labels), (test_images, test_labels) = mnist.load_data()

train_images = train_images.reshape(60000, 784)
test_images = test_images.reshape(10000, 784)

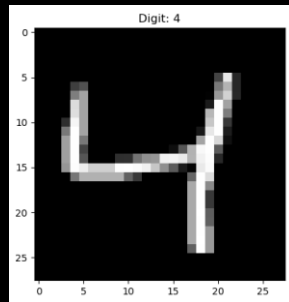
test_images = test_images.astype('float32')
train_images = train_images.astype('float32')

test_images /= 255
train_images /= 255
```

matplotlib bonus insight

```
import matplotlib.pyplot as plt

plt.imshow(train_images[2], cmap=plt.get_cmap('gray'),
            interpolation='none')
plt.title("Digit: {}".format(train_labels[2]))
```



Defining Our Network

```
import tensorflow as tf

mnist = tf.keras.datasets.mnist
(train_images, train_labels), (test_images, test_labels) = mnist.load_data()

train_images = train_images.reshape(60000, 784)
test_images = test_images.reshape(10000, 784)

test_images = test_images.astype('float32')
train_images = train_images.astype('float32')

test_images /= 255
train_images /= 255

model = tf.keras.Sequential([
    tf.keras.layers.Dense(64, activation='relu', input_shape=(784,)),
    tf.keras.layers.Dense(64, activation='relu'),
    tf.keras.layers.Dense(10, activation='softmax'),
])
```

Starting from zero?

In general, initialization values are hard to pin down analytically. Values might help optimization but hurt generalization, or vice versa.

The only certainty is you need to have different values to break the symmetry, or else units in the same layer, with the same inputs, would track each other.

Practically, we just pick some "reasonable" values.

```
model.summary()
```

Layer (type)	Output Shape	Param #
dense_6 (Dense)	(None, 64)	50240
dense_7 (Dense)	(None, 64)	4160
dense_8 (Dense)	(None, 10)	650
Total params: 55,050		
Trainable params: 55,050		
Non-trainable params: 0		

Softmax

why Softmax?

The values coming out of our matrix operations can have large, and negative values. We would like our solution vector to be conventional probabilities that sum to 1.0. An effective way to normalize our outputs is to use the popular *Softmax* function. Let's look at an example with just three possible digits:

Digit	Output	Exponential	Normalized
0	4.8	121	.87
1	-2.6	0.07	.00
2	2.9	18	.13

Solving For Weights

```
import tensorflow as tf

mnist = tf.keras.datasets.mnist
(train_images, train_labels), (test_images, test_labels) = mnist.load_data()

train_images = train_images.reshape(60000, 784)
test_images = test_images.reshape(10000, 784)

test_images = test_images.astype('float32')
train_images = train_images.astype('float32')

test_images /= 255
train_images /= 255

model = tf.keras.Sequential([
    tf.keras.layers.Dense(64, activation='relu', input_shape=(784,)),
    tf.keras.layers.Dense(64, activation='relu'),
    tf.keras.layers.Dense(10, activation='softmax'),
])

model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])
```

Cross Entropy

Given the sensible way we have constructed these outputs, the **Cross Entropy Loss** function is a good way to define the error across all possibilities. Better than squared error, which we have been using until now. It is defined as $-\sum y_i \log y_i$, or if this really is a "0", $y_i=(1,0,0)$, and

$$-1\log(0.87) - 0\log(0.0001) - 0\log(0.13) = -\log(0.87) = -0.13$$

It somewhat penalizes a slightly wrong guess, or an "unconfident" right guess, and greatly penalizes a very wrong guess.

You can also think that it "undoes" the Softmax, if you want.

Training

```
import tensorflow as tf

mnist = tf.keras.datasets.mnist
(train_images, train_labels), (test_images, test_labels) = mnist.load_data()

train_images = train_images.reshape(60000, 784)
test_images = test_images.reshape(10000, 784)

test_images = test_images.astype('float32')
train_images = train_images.astype('float32')

test_images /= 255
train_images /= 255

model = tf.keras.Sequential([
    tf.keras.layers.Dense(64, activation='relu', input_shape=(784,)),
    tf.keras.layers.Dense(64, activation='relu'),
    tf.keras.layers.Dense(10, activation='softmax'),
])

model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])

history = model.fit(train_images, train_labels, batch_size=128, epochs=40, verbose=1, validation_data=(test_images, test_labels))
```

Results

matplotlib bonus insight

```
history = model.fit(train_images, ..., ...)
```

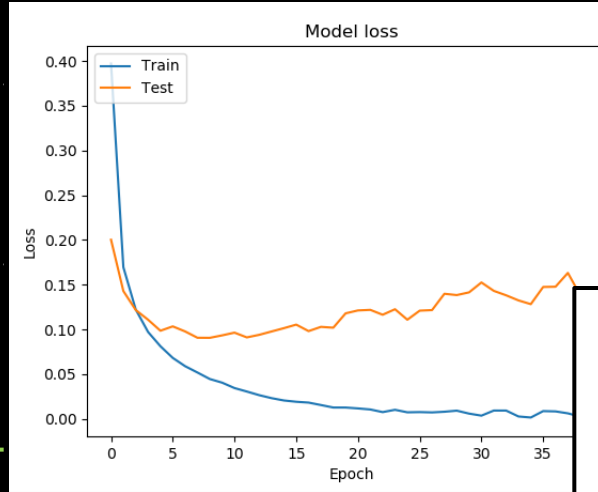
```
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title('Model accuracy')
plt.ylabel('Accuracy')
plt.xlabel('Epoch')
plt.legend(['Train', 'Test'], loc='upper
```

```
right')

plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title('Model loss')
plt.ylabel('Loss')
plt.xlabel('Epoch')
plt.legend(['Train', 'Test'], loc='upper
plt.show()
```

/sample - loss: 0.3971 - accuracy:

/sample - loss: 0.1696 - accuracy:



why would the test accuracy *ever* be better than the training (as momentarily happens here)?

The training value is the average over each batch, and the test value is only at the end of the epoch, when the model tends to be at least slightly better.

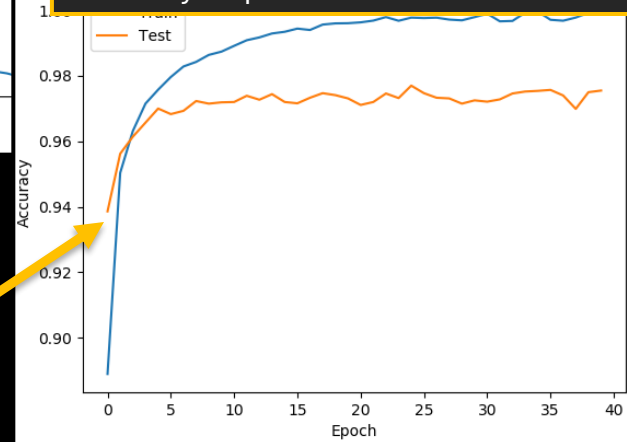
Latter on we will see that regularization techniques (which are only turned on for training) also add to this effect.

Accuracy or Loss?

Loss is the "mathematical" value we have specified in our model to use for parameter fitting.

Accuracy is simply how many we get right when we test our model as an application. It might not apply to a non-classification problem (think *Stable Diffusion*) and it doesn't capture how much right or wrong we are (we could be very confident that a dog is a cat).

The two are normally closely related and track each other. We will choose Accuracy for our graphs. Any user understands what accuracy represents.



0.9755

Let's Go Wider

```
import tensorflow as tf

mnist = tf.keras.datasets.mnist
(train_images, train_labels), (test_images, test_labels) = mnist.load_data()

train_images = train_images.reshape(60000, 784)
test_images = test_images.reshape(10000, 784)

test_images = test_images.astype('float32')
train_images = train_images.astype('float32')

test_images /= 255
train_images /= 255

model = tf.keras.Sequential([
    tf.keras.layers.Dense(512, activation='relu', input_shape=(784,)),
    tf.keras.layers.Dense(512, activation='relu'),
    tf.keras.layers.Dense(10, activation='softmax'),
])

model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])

model.fit(train_images, train_labels, batch_size=128, epochs=30, verbose=1, validation_data=(test_images, test_labels))
```

Wider Results

....

....

Epoch 30/30

60000/60000 [=====] - 2s 32us/sample - loss: 0.0083 - accuracy: 0.9977 - val_loss: 0.1027 - val_accuracy: 0.9821



wider

model.summary()

Layer (type)	Output Shape	Param #
dense_18 (Dense)	(None, 512)	401920
dense_19 (Dense)	(None, 512)	262656
dense_20 (Dense)	(None, 10)	5130

Total params: 669,706
Trainable params: 669,706
Non-trainable params: 0

55,050 for 64 wide Model

Maybe Deeper?

```
import tensorflow as tf

mnist = tf.keras.datasets.mnist
(train_images, train_labels), (test_images, test_labels) = mnist.load_data()

train_images = train_images.reshape(60000, 784)
test_images = test_images.reshape(10000, 784)

test_images = test_images.astype('float32')
train_images = train_images.astype('float32')

test_images /= 255
train_images /= 255

model = tf.keras.Sequential([
    tf.keras.layers.Dense(512, activation='relu', input_shape=(784,)),
    tf.keras.layers.Dense(512, activation='relu'),
    tf.keras.layers.Dense(512, activation='relu'),
    tf.keras.layers.Dense(10, activation='softmax'),
])

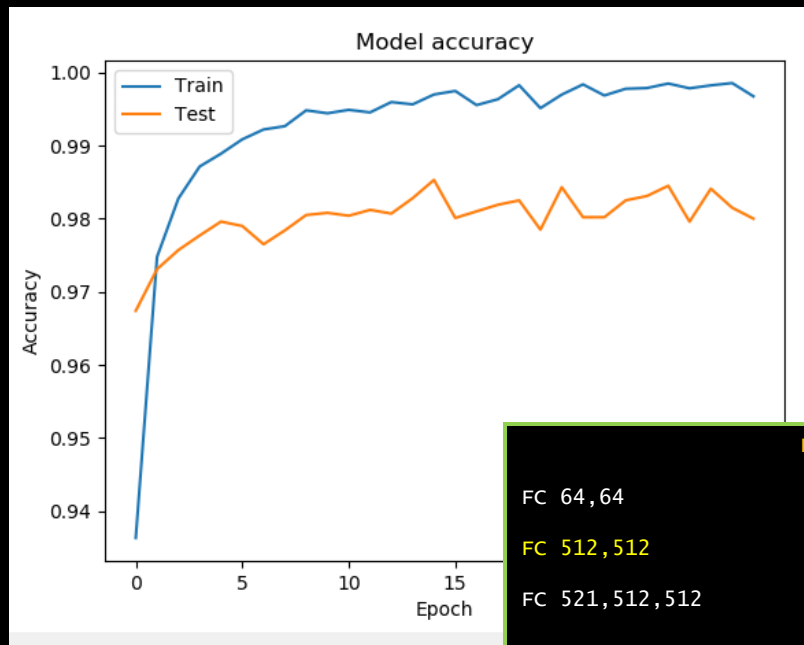
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])

model.fit(train_images, train_labels, batch_size=128, epochs=30, verbose=1, validation_data=(test_images, test_labels))
```

Wide And Deep Results

....

60000/60000 [=====] - 3s 45us/sample - loss: 0.0119 - accuracy: 0.9967 - val_loss: 0.1183 - val_accuracy: 0.9800



Deep and wide

model.summary()

Layer (type)	Output Shape	Param #
dense_24 (Dense)	(None, 512)	401920
dense_25 (Dense)	(None, 512)	262656
dense_26 (Dense)	(None, 512)	262656
dense_27 (Dense)	(None, 10)	5130
Total params: 932,362		

s: 932,362
arams: 0

Recap

FC 64,64	97.5
FC 512,512	98.2
FC 521,512,512	98.0

Brute Force Does Not Work

You usually can't just brute force your way into success. Beyond the obvious time and memory costs, you are opening yourself up to

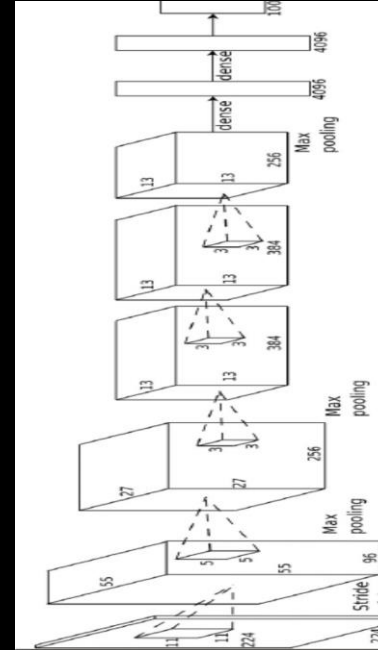
- Overfitting
- Vanishing gradients

We will have to be smarter than "bigger is better" about choosing our hyperparameters. One very smart thing to do is to choose a more appropriate architecture.

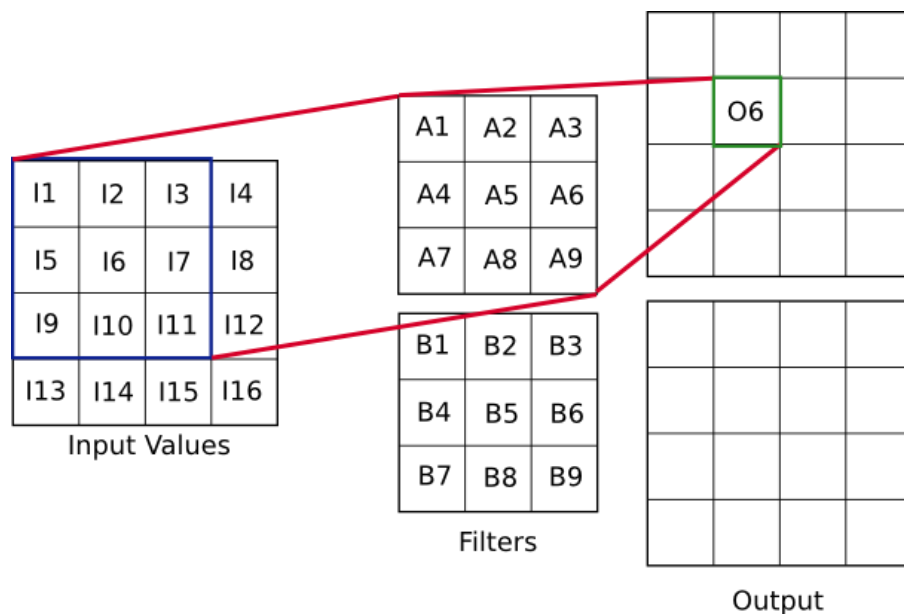
Image Recognition Done Right: CNNs

AlexNet won the 2012 ImageNet LSVRC and changed the DL world.

4M	FULL CONNECT
16M	FULL 4096/ReLU
37M	FULL 4096/ReLU
	MAX POOLING
442K	CONV 3x3/ReLU
1.3M	CONV 3x3/ReLU
884K	CONV 3x3/ReLU
	MAX POOLING 2x2sub
	LOCAL CONTRAST NORM
307K	CONV 11x11/ReLU
	MAX POOL 2x2sub
	LOCAL CONTRAST NORM
35K	CONV 11x11/ReLU



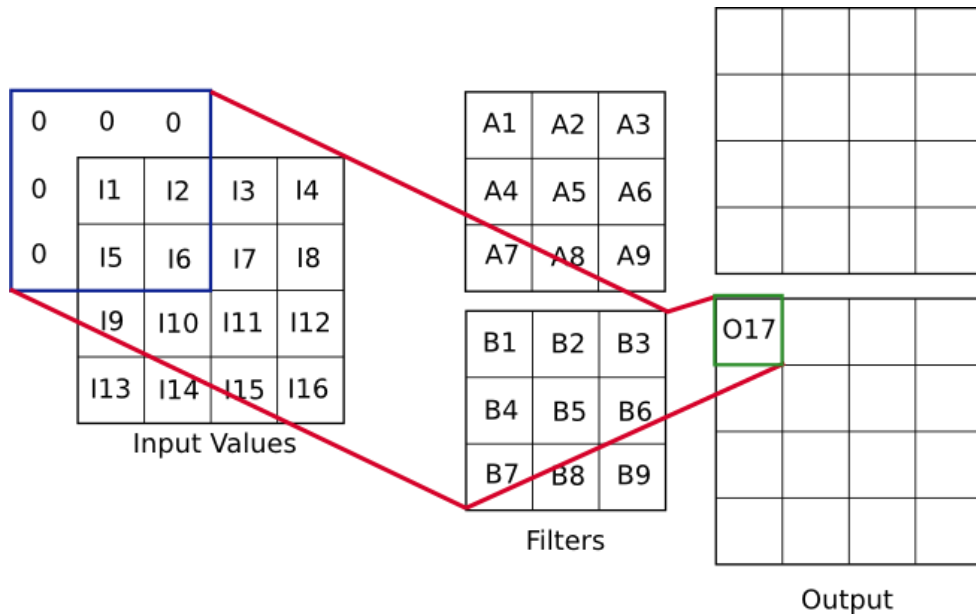
Convolution



$$\begin{aligned} O_6 = & A_1 \cdot I_1 + A_2 \cdot I_2 + A_3 \cdot I_3 \\ & + A_4 \cdot I_5 + A_5 \cdot I_6 + A_6 \cdot I_7 \\ & + A_7 \cdot I_9 + A_8 \cdot I_{10} + A_9 \cdot I_{11} \end{aligned}$$

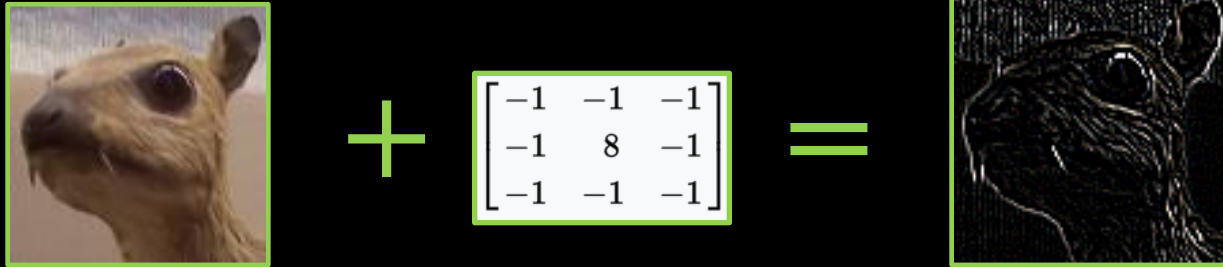
Convolution

Boundary and Index Accounting



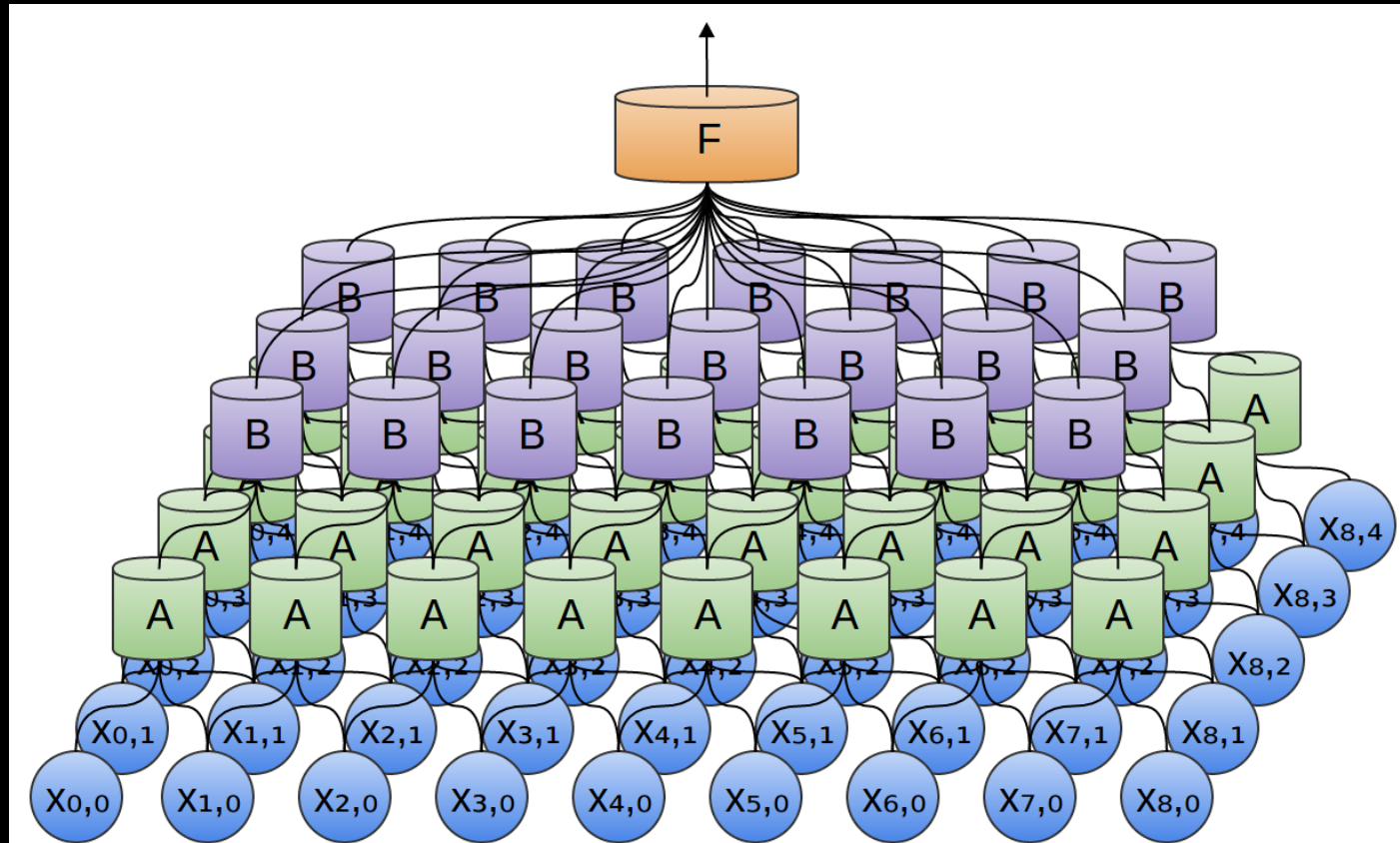
$$O_{17} = B_5 \cdot I_1 + B_6 \cdot I_2 + B_8 \cdot I_5 + B_9 \cdot I_6$$

Straight Convolution

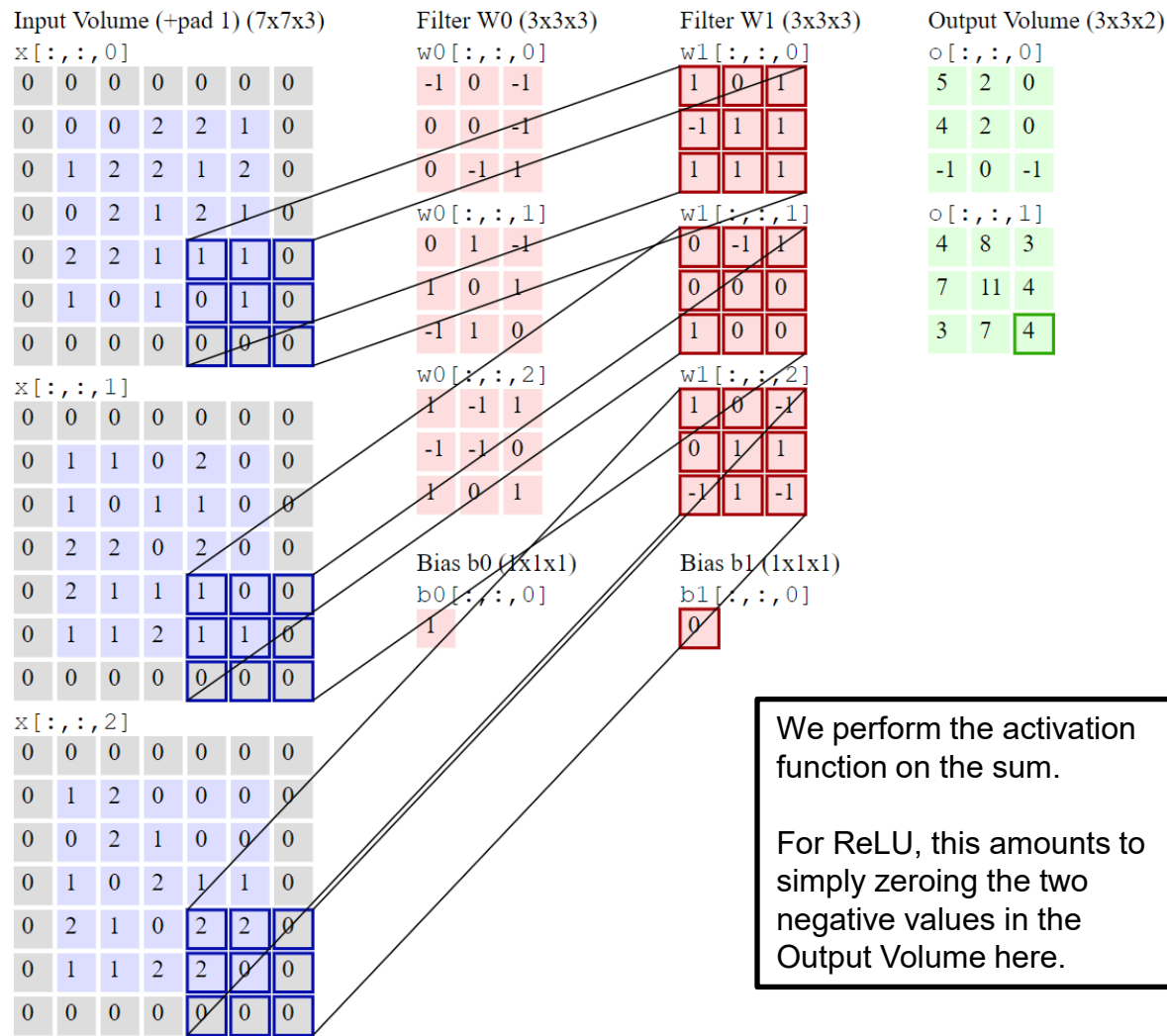

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

Edge Detector

Stacking Convolutions



C o n v o l u t i o n



From the very nice
Stanford CS231n
course at
<http://cs231n.github.io/convolutional-networks/>

Stride = 2

Convolution Math

Each Convolutional Layer:

Inputs a volume of size $W_I \times H_I \times D_I$ (D is depth)

Requires four hyperparameters:

- Number of filters K

- their spatial extent N

- the stride S

- the amount of padding P

Produces a volume of size $W_O \times H_O \times D_O$

$$W_O = (W_I - N + 2P) / S + 1$$

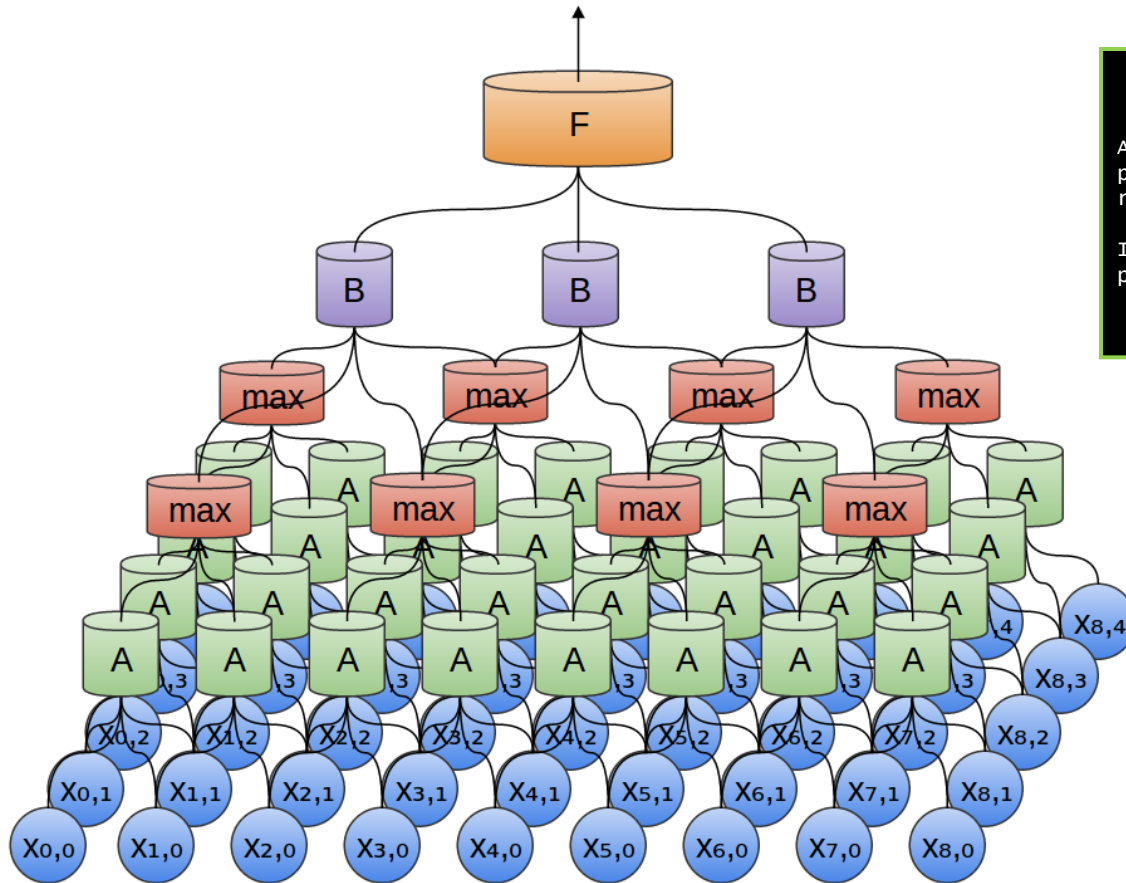
$$H_O = (H_I - F + 2P) / S + 1$$

$$D_O = K$$

This requires $N \cdot N \cdot D_I$ weights per filter, for a total of $N \cdot N \cdot D_I \cdot K$ weights and K biases

In the output volume, the d-th depth slice (of size $W_O \times H_O$) is the result of performing a convolution of the d-th filter over the input volume with a stride of S, and then offset by d-th bias.

Pooling



Another Dimensionality Reduction!

Another very valid perspective is that pooling is a simple dimensionality reduction.

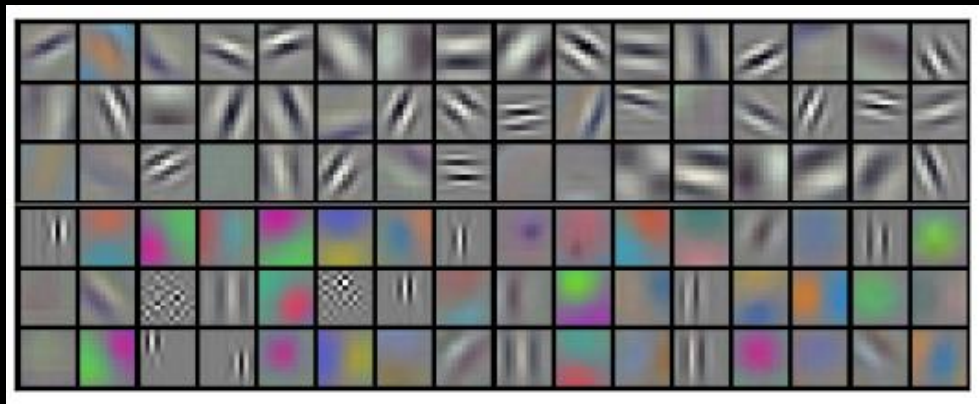
It is very helpful to possess these different perspectives.



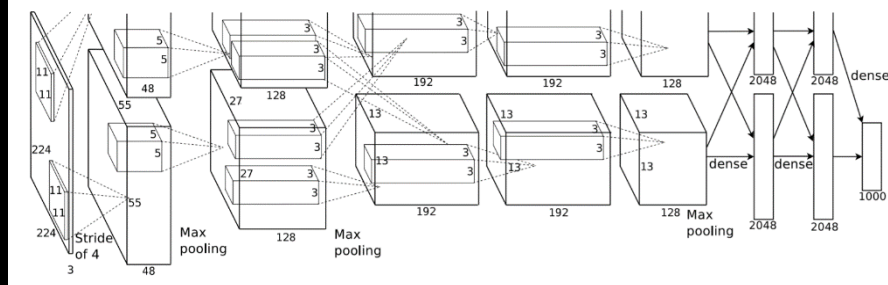
We don't use an activation function when pooling.

A Groundbreaking Example

These are the 96 first layer 11x11 (x3, RGB, stacked here) filters from AlexNet.

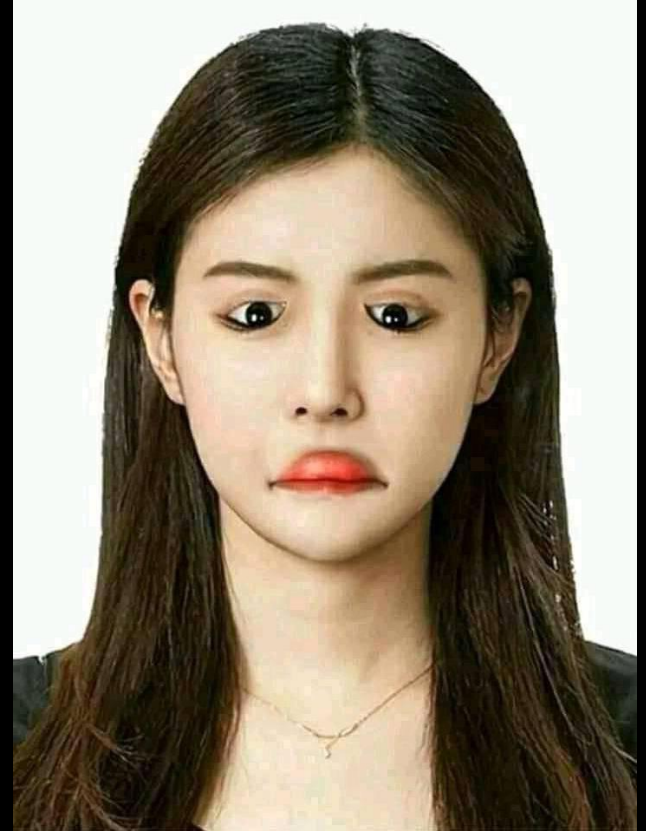


Among the several novel techniques combined in this work (such as aggressive use of ReLU), they used dual GPUs, with different flows for each, communicating only at certain layers. A result is that the bottom GPU consistently specialized on color information, and the top did not.



This is your brain on CNNs.

One of countless "illusions" I could provoke your own CNNs with.



Let's Start Small

```
import tensorflow as tf

mnist = tf.keras.datasets.mnist
(train_images, train_labels), (test_images, test_labels) = mnist.load_data()

train_images = train_images.reshape(60000, 28, 28, 1)
test_images = test_images.reshape(10000, 28, 28, 1)
train_images, test_images = train_images/255, test_images/255

model = tf.keras.Sequential([
    tf.keras.layers.Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),
    tf.keras.layers.MaxPooling2D(2,2),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(100, activation='relu'),
    tf.keras.layers.Dense(10, activation='softmax')
])

model.compile(optimizer=tf.keras.optimizers.SGD(lr=0.01, momentum=0.9), loss='sparse_categorical_crossentropy', metrics=['accuracy'])

model.fit(train_images, train_labels, batch_size=32, epochs=10, verbose=1, validation_data=(test_images, test_labels))
```

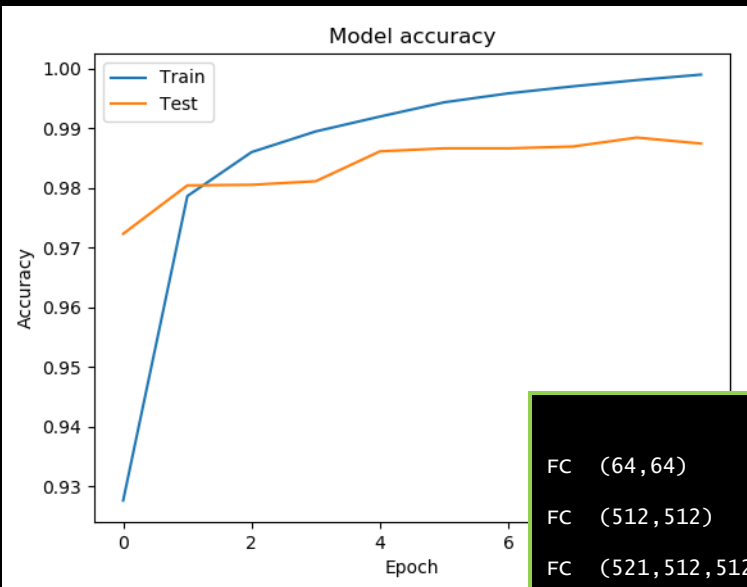
Early CNN Results

....

....

Epoch 10/10

60000/60000 [=====] - 12s 198us/sample - loss: 0.0051 - accuracy: 0.9989 - val_loss: 0.0424 - val_accuracy: 0.9874



Score Thus Far

FC (64,64)	97.5
FC (512,512)	98.2
FC (521,512,512)	98.0
CNN (1 layer)	98.7

Primitive CNN

```
model.summary()
```

Layer (type)	Output Shape	Param #
=====		
conv2d_1 (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d_1	(None, 13, 13, 32)	0
flatten_1 (Flatten)	(None, 5408)	0
dense_38 (Dense)	(None, 100)	540900
dense_39 (Dense)	(None, 10)	1010
=====		

ms: 542,230
params: 542,230
ble params: 0

Scaling Up The CNN

```
import tensorflow as tf

mnist = tf.keras.datasets.mnist
(train_images, train_labels), (test_images, test_labels) = mnist.load_data()

train_images = train_images.reshape(60000, 28, 28, 1)
test_images = test_images.reshape(10000, 28, 28, 1)
train_images, test_images = train_images/255, test_images/255

model = tf.keras.Sequential([
    tf.keras.layers.Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),
    tf.keras.layers.Conv2D(64, (3,3), activation='relu'),
    tf.keras.layers.MaxPooling2D(2,2),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dense(10, activation='softmax')
])

model.compile(optimizer=tf.keras.optimizers.SGD(lr=0.01, momentum=0.9), loss='sparse_categorical_crossentropy', metrics=['accuracy'])

model.fit(train_images, train_labels, batch_size=32, epochs=10, verbose=1, validation_data=(test_images, test_labels))
```

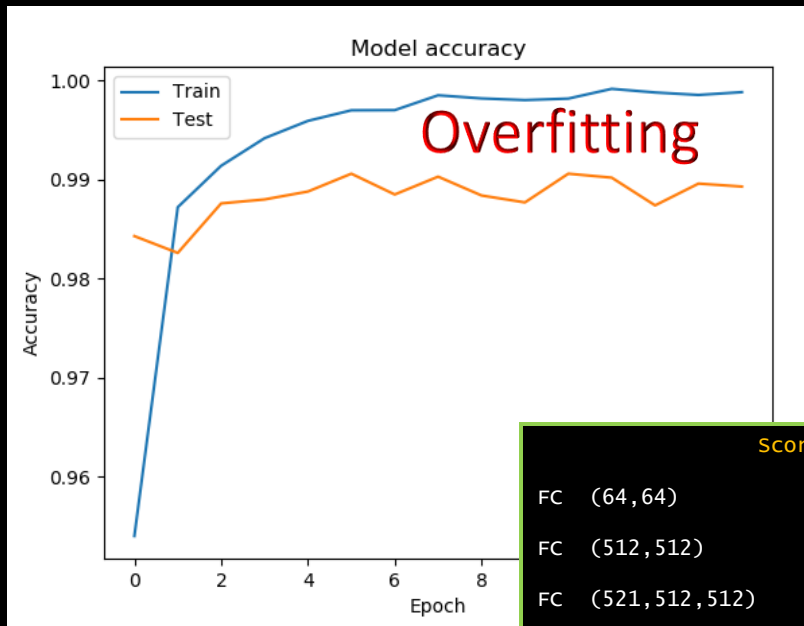
Deeper CNN Results

....

....

Epoch 15/15

60000/60000 [=====] - 34s 566us/sample - loss: 0.0052 - accuracy: 0.9985 - val_loss: 0.0342 - val_accuracy: 0.9903



Score Thus Far

FC (64,64)	97.5
FC (512,512)	98.2
FC (521,512,512)	98.0
CNN (1 layer)	98.7
CNN (2 Layer)	99.0

Deeper CNN

```
model.summary()
```

Layer (type)	Output Shape	Param #
conv2d_4 (Conv2D)	(None, 26, 26, 32)	320
conv2d_5 (Conv2D)	(None, 24, 24, 64)	18496
max_pooling2d_3	(None, 12, 12, 64)	0
flatten_3 (Flatten)	(None, 9216)	0
dense_42 (Dense)	(None, 128)	1179776
	(None, 10)	1290

=====

1,199,882

s: 1,199,882

arams: 0

Overfitting = Memorization

We now have enough parameters that the network is prone to memorizing instead of learning. This will only get worse as our larger and smarter networks grow into billions of parameters.



Cat



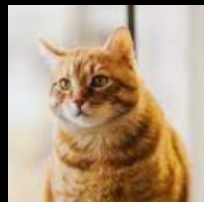
Dog



Dog



Cat



Cat



Dog



Dog



Dog



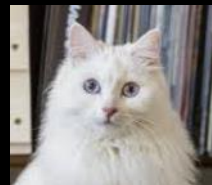
Dog



Dog

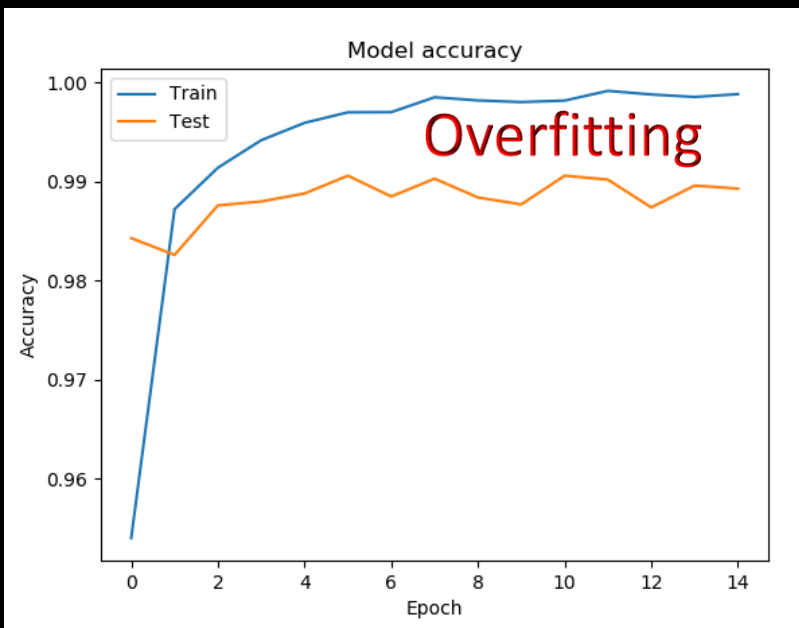


Cat



Cat

Dropout



As we know by now, we need some form of regularization to help with the overfitting. One seemingly crazy way to do this is the relatively new technique (introduced by the venerable Geoffrey Hinton in 2012) of Dropout.



Some view it as an ensemble method that trains multiple data models simultaneously. One neat perspective of this analysis-defying technique comes from Jürgen Schmidhuber, another innovator in the field; under certain circumstances, it could also be viewed as a form of training set augmentation: effectively, more and more informative complex features are removed from the training data.

CNN With Dropout

```
import tensorflow as tf

mnist = tf.keras.datasets.mnist
(train_images, train_labels), (test_images, test_labels) = mnist.load_data()

train_images = train_images.reshape(60000, 28, 28, 1)
test_images = test_images.reshape(10000, 28, 28, 1)
train_images, test_images = train_images/255, test_images/255

model = tf.keras.Sequential([
    tf.keras.layers.Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),
    tf.keras.layers.Conv2D(64, (3,3), activation='relu'),
    tf.keras.layers.MaxPooling2D(2,2),
    tf.keras.layers.Dropout(0.25),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dropout(0.5),
    tf.keras.layers.Dense(10, activation='softmax')
])

model.compile(optimizer=tf.keras.optimizers.SGD(lr=0.01, momentum=0.9), loss='sparse_categorical_crossentropy', metrics=['accuracy'])

model.fit(train_images, train_labels, batch_size=32, epochs=10, verbose=1, validation_data=(test_images, test_labels))
```

Parameter is fraction to *drop*.

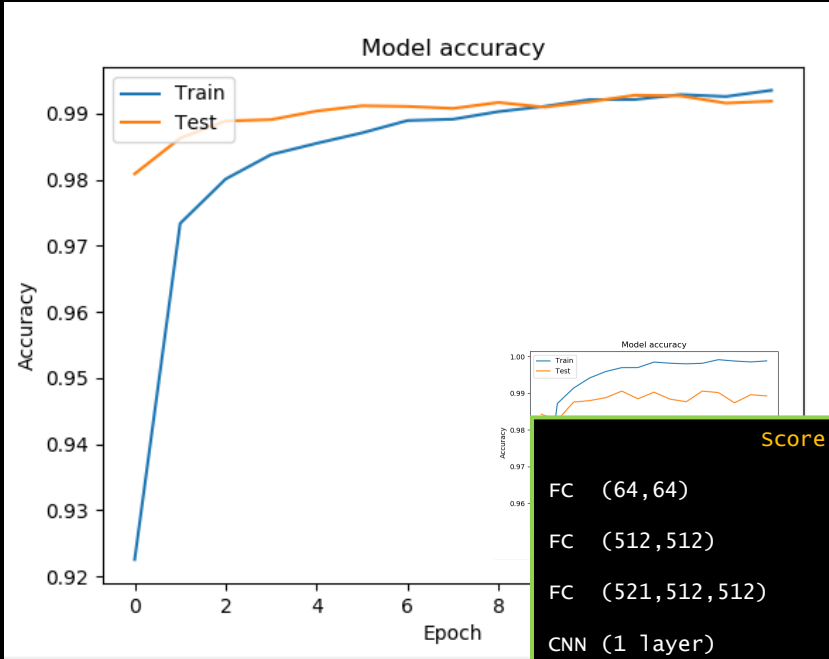
Drop out is not used in the final, trained, network. Similarly, it is automatically disabled here during testing.

Help From Dropout

....
....

Epoch 15/15

60000/60000 [=====] - 40s 667us/sample - loss: 0.0187 - accuracy: 0.9935 - val_loss: 0.0301 - val_accuracy: 0.9919



Score Thus Far

FC (64,64)	97.5
FC (512,512)	98.2
FC (521,512,512)	98.0
CNN (1 layer)	98.7
CNN (2 Layer)	99.0
CNN with Dropout	99.2

Dropout CNN

```
model.summary()
```

Layer (type)	Output Shape	Param #
conv2d_12 (Conv2D)	(None, 26, 26, 32)	320
conv2d_13 (Conv2D)	(None, 24, 24, 64)	18496
max_pooling2d_7	(None, 12, 12, 64)	0
dropout_4 (Dropout)	(None, 12, 12, 64)	0
flatten_7 (Flatten)	(None, 9216)	0
(Dense)	(None, 128)	1179776
(Dropout)	(None, 128)	0
(Dense)	(None, 10)	1290

params: 1,199,882
params: 1,199,882
able params: 0

Batch Normalization

Another "between layers" layer that is quite popular is Batch Normalization. This technique really helps with vanishing or exploding gradients. So it is better with deeper networks.

- Maybe not so compatible with Dropout, but the subject of research (and debate).
- Maybe Apply Dropout after all BN layers: <https://arxiv.org/pdf/1801.05134.pdf>
- Before or after non-linear activation function? Oddly, also open to debate. But, it may be more appropriate after the activation function if for s-shaped functions like the hyperbolic tangent and logistic function, and before the activation function for activations that result in non-Gaussian distributions like ReLU.

How could we apply it before or after our activation function if we wanted to? We haven't been peeling our layers apart, but we can micro-manage more if we want to:

```
model.add(tf.keras.layers.Conv2D(64, (3, 3), use_bias=False))
model.add(tf.keras.layers.BatchNormalization())
model.add(tf.keras.layers.Activation("relu"))

model.add(tf.keras.layers.Conv2D(64, kernel_size=3, strides=2, padding="same"))
model.add(tf.keras.layers.LeakyReLU(alpha=0.2))
model.add(tf.keras.layers.BatchNormalization(momentum=0.8))
```

There are also normalizations that work on single samples instead of batches, so better for recurrent networks. In TensorFlow we have Group Normalization, Instance Normalization and Layer Normalization.

Fashion MNIST

John Urbanic

Parallel Computing Scientist
Pittsburgh Supercomputing Center

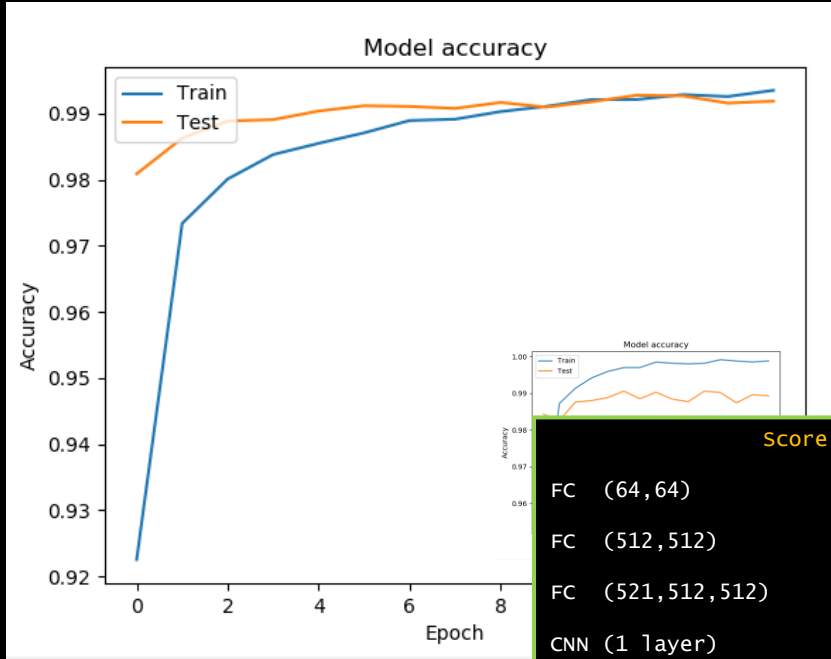
Distinguished Service Professor
Carnegie Mellon University

Help From Dropout

....
....

Epoch 15/15

60000/60000 [=====] - 40s 667us/sample - loss: 0.0187 - accuracy: 0.9935 - val_loss: 0.0301 - val_accuracy: 0.9919



Score Thus Far

FC (64,64)	97.5
FC (512,512)	98.2
FC (521,512,512)	98.0
CNN (1 layer)	98.7
CNN (2 Layer)	99.0
CNN with Dropout	99.2

Dropout CNN

model.summary()

Layer (type)	Output Shape	Param #
conv2d_12 (Conv2D)	(None, 26, 26, 32)	320
conv2d_13 (Conv2D)	(None, 24, 24, 64)	18496
max_pooling2d_7	(None, 12, 12, 64)	0
dropout_4 (Dropout)	(None, 12, 12, 64)	0
flatten_7 (Flatten)	(None, 9216)	0
(Dense)	(None, 128)	1179776
(Dropout)	(None, 128)	0
(Dense)	(None, 10)	1290

params: 1,199,882
trainable params: 0

Batch Normalization

Another "between layers" layer that is quite popular is Batch Normalization. This technique really helps with vanishing or exploding gradients. So it is better with deeper networks.

- Maybe not so compatible with Dropout, but the subject of research (and debate).
- Maybe Apply Dropout after all BN layers: <https://arxiv.org/pdf/1801.05134.pdf>
- Before or after non-linear activation function? Oddly, also open to debate. But, it may be more appropriate after the activation function if for s-shaped functions like the hyperbolic tangent and logistic function, and before the activation function for activations that result in non-Gaussian distributions like ReLU.

How could we apply it before or after our activation function if we wanted to? We haven't been peeling our layers apart, but we can micro-manage more if we want to:

```
model.add(tf.keras.layers.Conv2D(64, (3, 3), use_bias=False))
model.add(tf.keras.layers.BatchNormalization())
model.add(tf.keras.layers.Activation("relu"))

model.add(tf.keras.layers.Conv2D(64, kernel_size=3, strides=2, padding="same"))
model.add(tf.keras.layers.LeakyReLU(alpha=0.2))
model.add(tf.keras.layers.BatchNormalization(momentum=0.8))
```

There are also normalizations that work on single samples instead of batches, so better for recurrent networks. In TensorFlow we have Group Normalization, Instance Normalization and Layer Normalization.

Trying Batch Normalization

```
import tensorflow as tf

mnist = tf.keras.datasets.mnist
(train_images, train_labels), (test_images, test_labels) = mnist.load_data()

train_images = train_images.reshape(60000, 28, 28, 1)
test_images = test_images.reshape(10000, 28, 28, 1)
train_images, test_images = train_images/255, test_images/255

model = tf.keras.Sequential([
    tf.keras.layers.Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.Conv2D(64, (3,3), activation='relu'),
    tf.keras.layers.MaxPooling2D(2,2),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.Dense(10, activation='softmax')
])

model.compile(optimizer=tf.keras.optimizers.SGD(lr=0.01, momentum=0.9), loss='sparse_categorical_crossentropy', metrics=['accuracy'])

model.fit(train_images, train_labels, batch_size=32, epochs=10, verbose=1, validation_data=(test_images, test_labels))
```

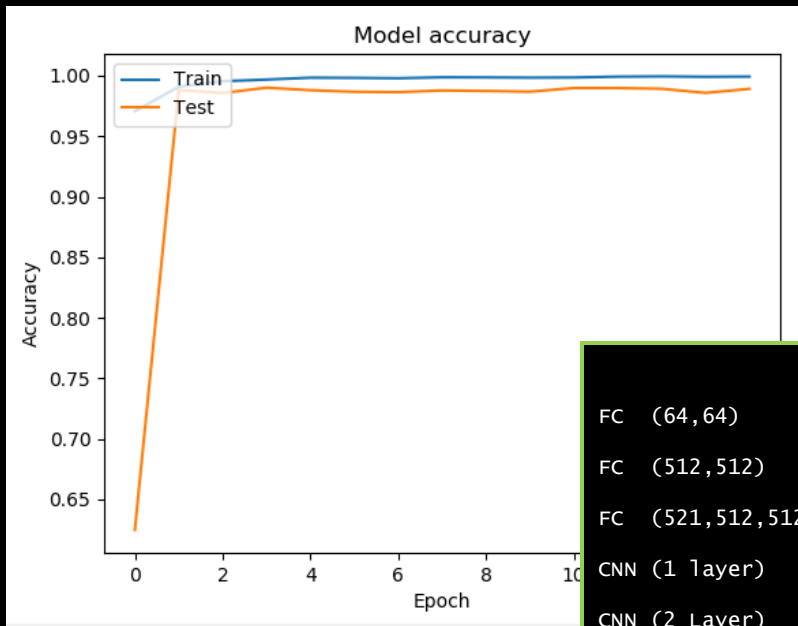
Not So Helpful

....

....

Epoch 15/15

60000/60000 [=====] - 50s 834us/sample - loss: 0.0027 - accuracy: 0.9993 - val_loss: 0.0385 - val_accuracy: 0.9891



Score Thus Far

FC (64,64)	97.5
FC (512,512)	98.2
FC (521,512,512)	98.0
CNN (1 layer)	98.7
CNN (2 Layer)	99.0
CNN with Dropout	99.2
Batch Normalization	98.9

Batch Normalization CNN

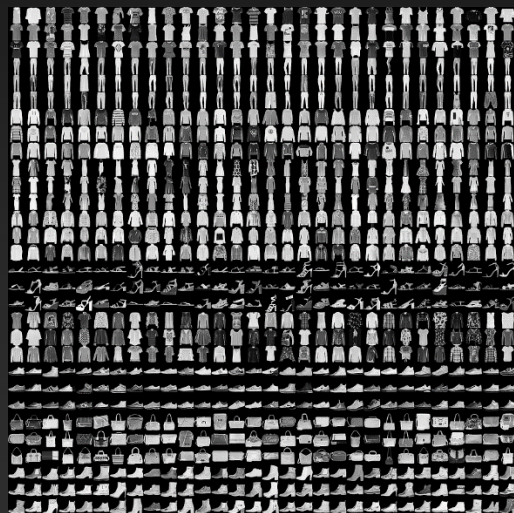
model.summary()

Layer (type)	Output Shape	Param #
conv2d_2 (Conv2D)	(None, 26, 26, 32)	320
batch_normalization	(None, 26, 26, 32)	128
conv2d_3 (Conv2D)	(None, 24, 24, 64)	18496
max_pooling2d_1	(None, 12, 12, 64)	0
batch_normalization_1	(None, 12, 12, 64)	256
flatten	(None, 9216)	0
conv2d_4 (Conv2D)	(None, 128)	1179776
batch_normalization_2	(Batch (None, 128))	512
conv2d_5 (Conv2D)	(None, 10)	1290
=====		
1,200,778		
params: 1,200,330		
params: 448		

Assignment #5: Fashion MNIST

We have done very well with MNIST, and trying to improve it much more will have very diminishing returns. Let us find a more challenging problem.

We have a very obvious and incremental next step with Fashion MNIST. These are 60,000 training images, and 10,000 test images of 10 types of clothing, in 28x28 greyscale. Sound familiar?



Assignment #5: Fashion MNIST



It is very easy to just change one line of code and we are using this new and more challenging dataset:

```
mnist = tf.keras.datasets.fashion_mnist
```

And, if you run using our current networks, you even get reasonable results. **You should consider this your "baseline"**. However, we now have a lot more room for improvement.

Your job is to find a better network, and get those improvements. You have plenty of hyperparameters to play with here, so you won't be short of options to explore.

You can change some of the obvious ones we have just investigated in obvious ways (brute force, bigger, bigger!) or you can peruse the documentation for inspiration.

This is an open-ended research project. There is no right answer, and you may even surpass my own solutions. However, if you put in a reasonable effort you should have no problem getting a measurable improvement over the baseline. And if you want to determine what a reasonable ceiling might be, I suggest a little Googling. ***A literature search is the essential starting point for any deep learning project!***

Assignment #5: Logistics

Deadline: November 25th. Before we start the lecture.

Points: 5 points.

Format: Email me your most accurate TensorFlow script as an attachment and the final results in the body of the email. This includes at least the test accuracy.

You will also need to include some kind of table or log of the hyperparameter tests that led you there. This might include some dead ends, or it might just be a serious investigation of one hyperparameter that worked out to improve over baseline. By "serious investigation" you should at least have some data points to indicate that this value is optimal in its own slice of the parameter space. This is typically an accuracy graph that shows "this kept giving better results, and then I went too far". This will be most of the grade!

Feel free to include comments in the body of the mail (and the code!). If you have problems, let me know what you were thinking and I will try to give you as much credit as possible.

I will be happy to answer any questions at any time. This often includes the night before things are due, but you don't want to count on that.

Another Fantastic Demo

This *amazing, stunning, beautiful* demo from Adam Harley is very similar to what we just did, but different enough to be interesting.

https://aharley.github.io/nn_vis/cnn/2d.html

It is worth experiment with. Note that this is an excellent demonstration of how efficient the forward network is. You are getting very real-time analysis from a lightweight web program. Training it took some time.

Draw your number here

2



Downsampled drawing: 2

First guess: 2

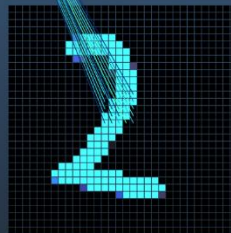
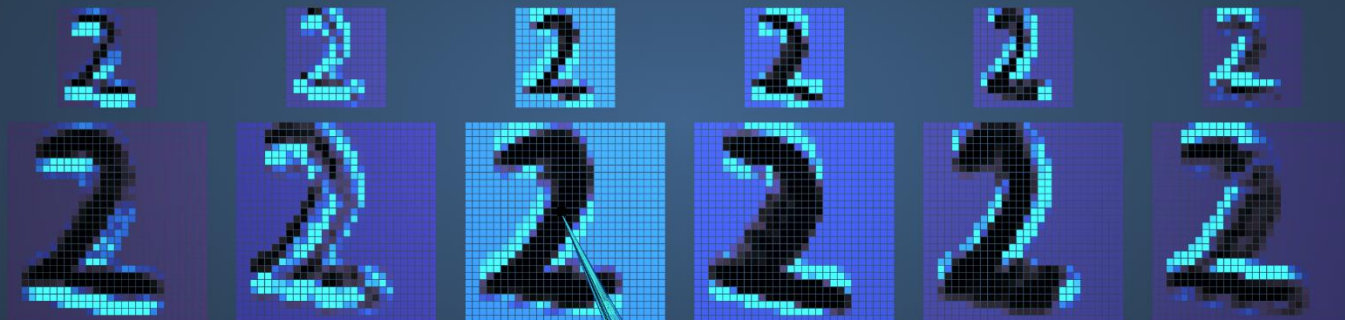
Second guess: 0

Layer visibility

Input layer	Show
Convolution layer 1	Show
Downsampling layer 1	Show
Convolution layer 2	Show
Downsampling layer 2	Show
Fully-connected layer 1	Show
Fully-connected layer 2	Show
Output layer	Show

0123456789

0123456789



Data Augmentation

As I've mentioned, labeled data is valuable. This type of *supervised learning* often requires human-labeled data. Getting more out of our expensive data is very desirable. More datapoints generally equals better accuracy. The process of generating more training data from our existing pool is called *Data Augmentation*, and is an extremely common technique, especially for classification

Our MNIST network has learned to recognize ve

What if we wanted to teach it:

How many samples do we need?

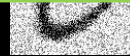
This is another hyperparameter (yes), where we can only offer a vague rule of thumb. And that suggestion is about 5000 per category for competence, 10 million for a real task with human performance.



Scale Invariance



Rotation Invariance



Noise Tolerance



Translation Invariance

You can see how straightforward and mechanical this is. And yet very effective. You will often see detailed explanations of the data augmentation techniques employed in any given project.

Note that `tf.image` makes many of these processes very convenient.

Stupid Neural Nets

Why can't they learn like we do? Can't I just tell you a fact, or an algorithm, and you can just "get it" it without countless iteration?



English for Infants

On the other hand, maybe I could teach adult particular mad bomber with your airport fac

There is such a thing as "one-shot" (or N-shot) learning. But it is harder, requires more specialized techniques, and is straying into the area of unsupervised learning. We will come back to this, but it is no magic bullet for sparse data.

How did you learn the English "rules"?



Matthew Anderson

@MattAndersonBBC

Things native English speakers know, but don't know we know:

adjectives in English absolutely have to be in this order: opinion-size-age-shape-colour-origin-material-purpose Noun. So you can have a lovely little old rectangular green French silver whittling knife. But if you mess with that word order in the slightest you'll sound like a maniac. It's an odd thing that every English speaker uses that list, but almost none of us could write it out. And as size comes before colour, green great dragons can't exist.

He wrote a fantastic follow-up, with some other surprisingly complex rules that we all somehow "know", but were never taught.

<https://www.bbc.com/culture/article/20160908-the-language-rules-we-know-but-dont-know-we-know>

Adding TensorBoard To Your Code

TensorBoard is a very versatile tool that allows us multiple types of insight into our TensorFlow codes. We need only add a callback into the model to activate the necessary logging.

```
...  
...  
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])  
tensorboard_callback = tf.keras.callbacks.TensorBoard(log_dir='TB_logDir', histogram_freq=1)  
history = model.fit(train_images, train_labels, batch_size=128, epochs=15, verbose=1,  
                    validation_data=(test_images, test_labels), callbacks=[tensorboard_callback])  
...  
...
```

TensorBoard runs as a server, because it has useful run-time capabilities, and requires you to start it separately, and to access it via a browser.

Somewhere else:

```
tensorboard --logdir=TB_logD
```

Somewhere else:

Start your Browser and point it at port 6006: <http://localhost:6006/>

If you are running on Bridges login nodes, from your computer something like:

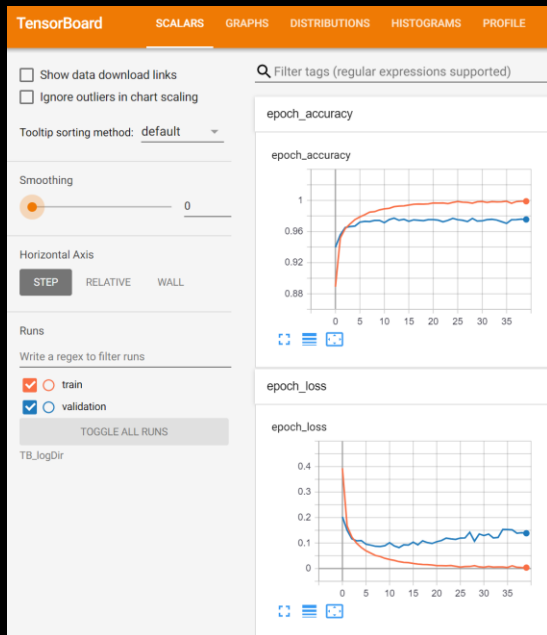
```
ssh -2 -Nf -L 6006:127.0.0.1:6006 br014.bridges2.psc.edu
```

If you are running on a Bridges compute nodes, you need to use the compute's IB address/hostname, for example:

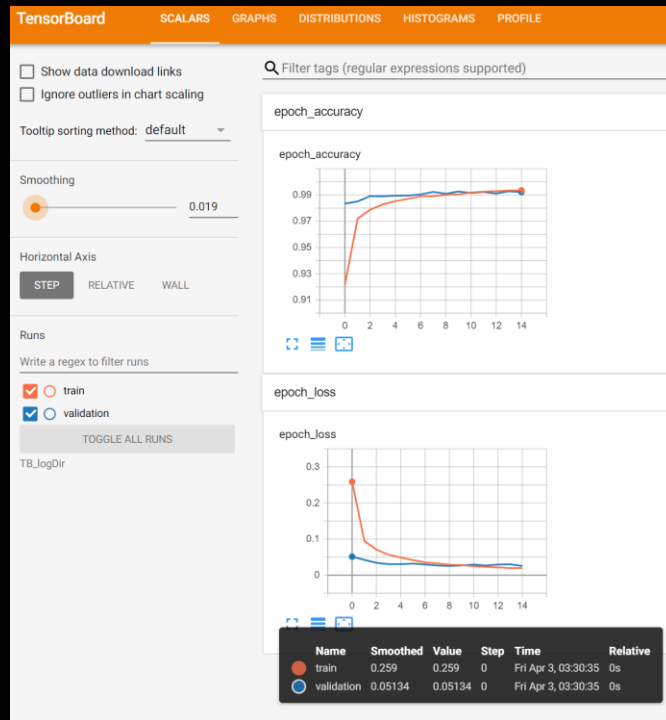
```
ssh -2 -Nf -L 6006:r001.ib.bridges2.psc.edu:6006 br014.bridges2.psc.edu
```

TensorBoard Analysis

The most obvious thing we can do is to look at our training loss. Note that TB is happy to do this in real-time as the model runs. This can be very useful for you to monitor overfitting.



Our First Model
64 Wide FC



Our CNN

Early Stopping

Of course, manual monitoring is not the only way to prevent needless training after the loss has plateaued. There are numerous ways in any framework to enable early stopping.

The most common is to add the *EarlyStopping* callback to your training loop.

```
tf.keras.callbacks.EarlyStopping(  
    monitor='val_loss',  
    min_delta=0,  
    patience=0,  
    verbose=0,  
    mode='auto',  
    baseline=None,  
    restore_best_weights=False,  
    start_from_epoch=0  
)
```

monitor	Quantity to be monitored. Defaults to "val_loss".
min_delta	Minimum change in the monitored quantity to qualify as an improvement, i.e. an absolute change of less than min_delta, will count as no improvement. Defaults to 0.
patience	Number of epochs with no improvement after which training will be stopped. Defaults to 0.
verbose	Verbosity mode, 0 or 1. Mode 0 is silent, and mode 1 displays messages when the callback takes an action. Defaults to 0.
mode	One of {"auto", "min", "max"}. In min mode, training will stop when the quantity monitored has stopped decreasing; in "max" mode it will stop when the quantity monitored has stopped increasing; in "auto" mode, the direction is automatically inferred from the name of the monitored quantity. Defaults to "auto".
baseline	Baseline value for the monitored quantity. If not None, training will stop if the model doesn't show improvement over the baseline. Defaults to None.
restore_best_weights	Whether to restore model weights from the best epoch. If False, the model weights are not restored regardless of the performance. If True, training will run for patience epochs after the last improvement before restoring the best weights. Defaults to False.
start_from_epoch	Number of epochs to wait before starting to monitor improvement. This allows for a warm-up period in which no improvement is expected and thus training will not be stopped. Defaults to 0.

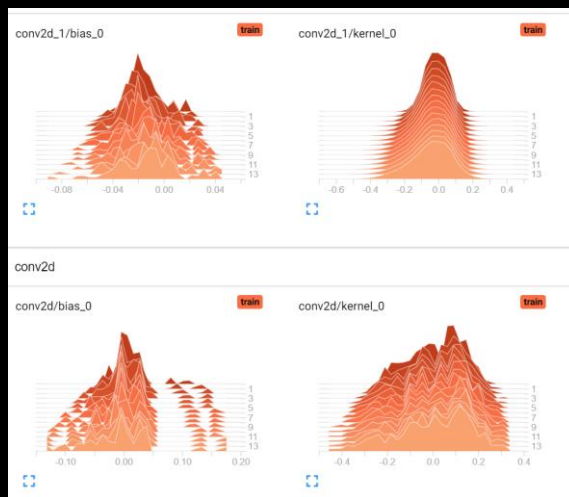
Note that early stopping can also be considered a form of regularization. We are preventing overfitting by stopping the training before it descends into pure memorization.

TensorBoard Parameter Visualization

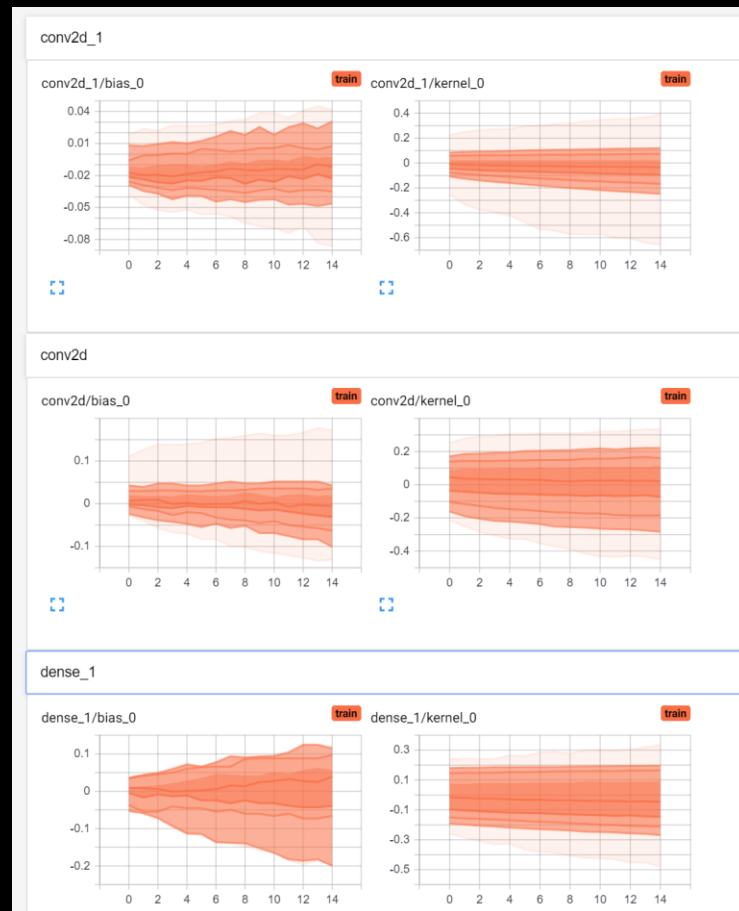
Distribution View

And we can observe the time evolution of our weights and biases, or at least their distributions.

This can be very telling, but requires some deeper application and architecture dependent understanding.



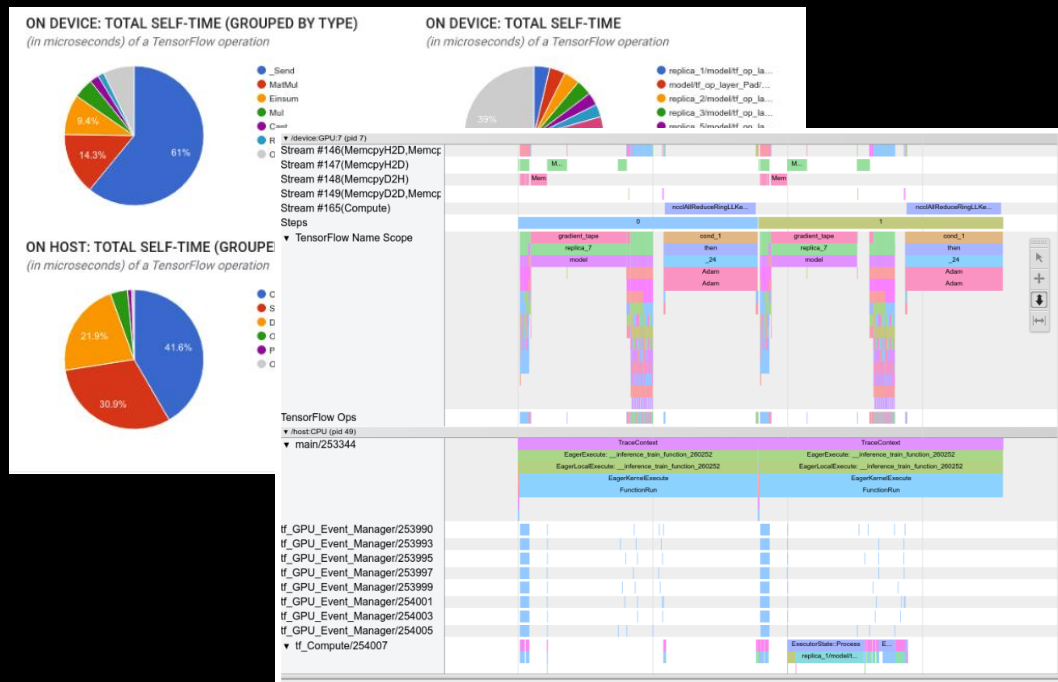
Histogram View



TensorBoard Add Ons

TensorBoard has lots of extended capabilities. Two particularly useful and powerful ones are Hyperparameter Search and Performance Profiling.

Performance Profiling



Hyperparameter Search

Requires some scripting on your part. Look at https://www.tensorflow.org/tensorboard/hyperparameter_tuning_with_hparams for a good introduction.

Going beyond basics, like **IO time**, requires integration of hardware specific tools. This is well covered if you are using NVIDIA, otherwise you may have a little experimentation to do. The end result is a user friendly interface and valuable guidance.

Scaling

If one GPU is good, more must be better! This is largely true, and you will notice our GPU nodes are stuffed full with 4 or more GPUs each.

You might also notice that most of the machines in the "Top 10" have a lot of GPUs in them. They deliver most of the FLOPS for scientific codes, but are also an enviable Deep Learning resource.

You might have noticed that most of the interesting leading-edge research seems to involve a lot of GPUs these days.

And the very public battles in the Large Language Model space seem to be about who can get their hands on the largest GPU clusters.

How might you reach these levels of capability?

And what about those scaling limitations I mentioned earlier?



Actually a Crypto miner. We hate these guys for hoarding our GPUs!!!

Data Parallelism

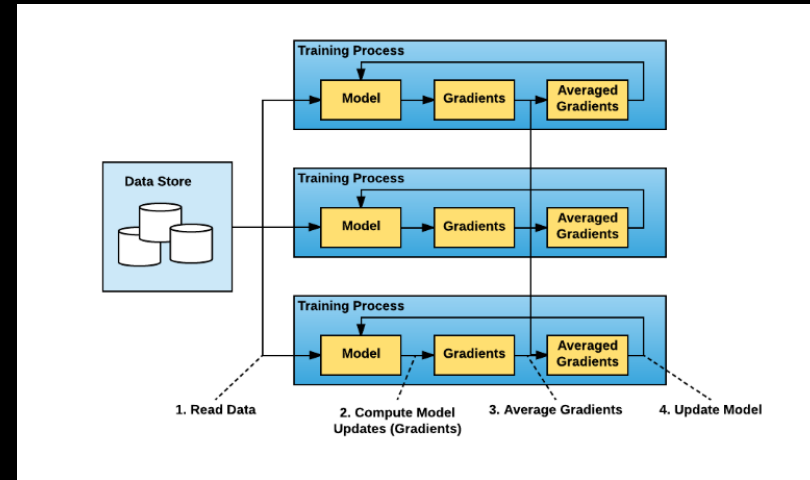
One early technique to utilize multiple GPUs was to independently train an ensemble of GPUs on the same task, and then have them vote on the answer. This method does work, but then the end user needs to have an ensemble of their own GPUs. This is not ideal for an application that you wish to run on your phone, or in a self-driving car.

It would be better if we could use a lot of GPUs for the training step, but end up with one great set of parameters that will fit on a single GPU when we are done. Then our users don't need to own supercomputers.

One technique to achieve this is to use *Data Parallelism* so that each GPU trains on a separate batch of data, and at the end of that batch we average the collective wisdom of all of these GPUs to arrive at our new and improved parameters.

Now when we finish we have one super set of parameters that fits on a single GPU.

This gradient averaging requires an *all-reduce*, which can be quite expensive given the number of weights involved.



TensorFlow Scalability

This is very straightforward to implement in TensorFlow using the **MirroredStrategy** on a single node with multiple GPUs, or **MultiWorkerMirroredStrategy** across multiple nodes.

```
strategy = tf.distribute.MirroredStrategy()
with strategy.scope():
    model = tf.keras.Sequential([
        tf.keras.layers.Dropout(rate=0.2, input_shape=X.shape[1:]),
        tf.keras.layers.Dense(units=64, activation='relu'),
        ...
    ])
    model.compile(...)
    model.fit(...)
```

The screenshot shows the TensorFlow API documentation for `tf.distribute.cluster_resolver.SlurmClusterResolver`. The left sidebar lists various TensorFlow components, with `cluster_resolver` expanded to show `SlurmClusterResolver` selected. The main content area displays the class name, a link to view the source on GitHub, a description: "ClusterResolver for system with Slurm workload manager.", and the text "Inherits From: ClusterResolver". Below this is a "View aliases" link and the class signature: `tf.distribute.cluster_resolver.SlurmClusterResolver(jobs=None, port_base=8888, gpus_per_node=None, gpus_per_task=None, tasks_per_node=None, auto_set_gpu=True, rpc_layer='grpc')`. A description follows: "This is an implementation of ClusterResolver for Slurm clusters. This counts, number of tasks per node, number of GPUs on each node and retrieves system attributes by Slurm environment variables, resolves a cluster and returns a ClusterResolver object which can be used to create a ClusterSession." At the bottom, there is an "Args" section with a table showing the `jobs` argument as a "Dictionary with job names as key and number of GPUs as value".

An alternative that has proven itself at extreme scale is *Horovod*.

MNIST with Horovod!

```
# Horovod: initialize Horovod.
hvd.init()

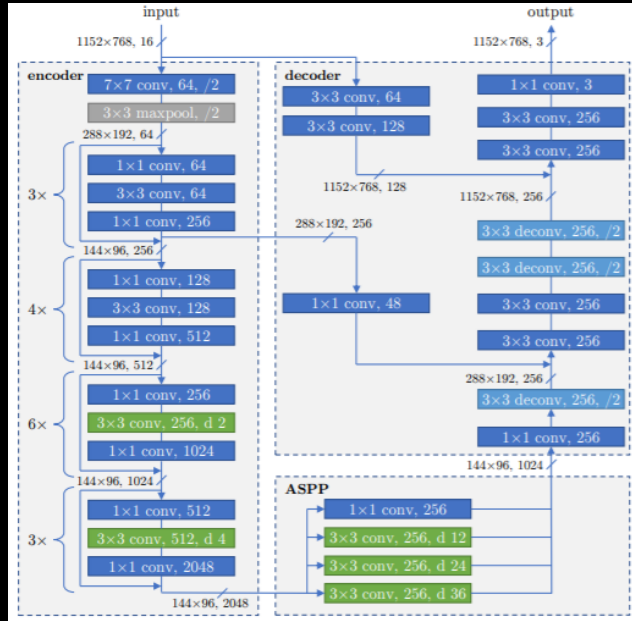
# Horovod: pin GPU to be used to process local rank (one GPU per process)
config = tf.ConfigProto()
config.gpu_options.allow_growth = True
config.gpu_options.visible_device_list = str(hvd.local_rank())
K.set_session(tf.Session(config=config))
...
# Horovod: adjust number of epochs based on number of GPUs.
epochs = int(math.ceil(12.0 / hvd.size()))
...
# Horovod: adjust learning rate based on number of GPUs.
opt = keras.optimizers.Adadelta(1.0 * hvd.size())
...
# Horovod: add Horovod Distributed Optimizer.
opt = hvd.DistributedOptimizer(opt)
...
model.compile(loss=keras.losses.categorical_crossentropy, optimizer=opt, metrics=['accuracy'])

callbacks = [hvd.callbacks.BroadcastGlobalVariablesCallback(0)]
if hvd.rank() == 0: callbacks.append(keras.callbacks.ModelCheckpoint('./checkpoint-{epoch}.h5'))
```

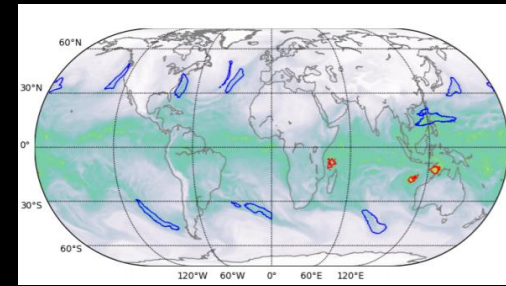
You can find a full example of using Horovod with a Keras MNIST code at: <https://horovod.readthedocs.io/en/latest/keras.html>

Scaling Up Massively

Horovod demonstrates its excellent scalability with a Climate Analytics code that won the Gordon Bell prize in 2018. It predicts Tropical Cyclones and Atmospheric River events based upon climate models. It shows not only the reach of deep learning in the sciences, but the scale at which networks can be trained.



- *1.13 ExaFlops (mixed precision) peak training performance*
- *On 4560 6 GPU nodes (27,360 GPUs total)*
- *High-accuracy (harder when predicting "no hurricane today" is 98% accurate), solved with weighted loss function.*
- *Layers each have different learning rate*



Model Parallelism

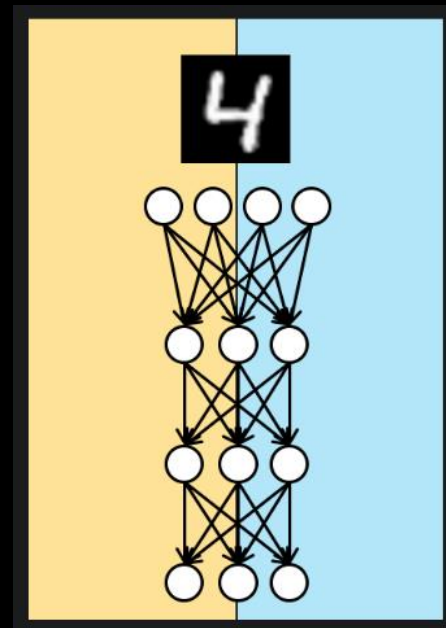
What about all these LLMs you have been hearing about that use *trillions* of parameters? Now, we don't have enough memory to fit the whole model on one GPU.



Instead we spread the parts of the model around (mostly their parameters, but could also be different sections of a more complex model) by using *Model Parallelism*.

The most popular way to do this in TensorFlow is via the *Mesh TensorFlow* API.

And, we can mix the way we distribute these parameters, layers, pipelines and model branches in various hybrid methods as well.



From the Chainer docs on their parallelism API. Yet another DL framework.

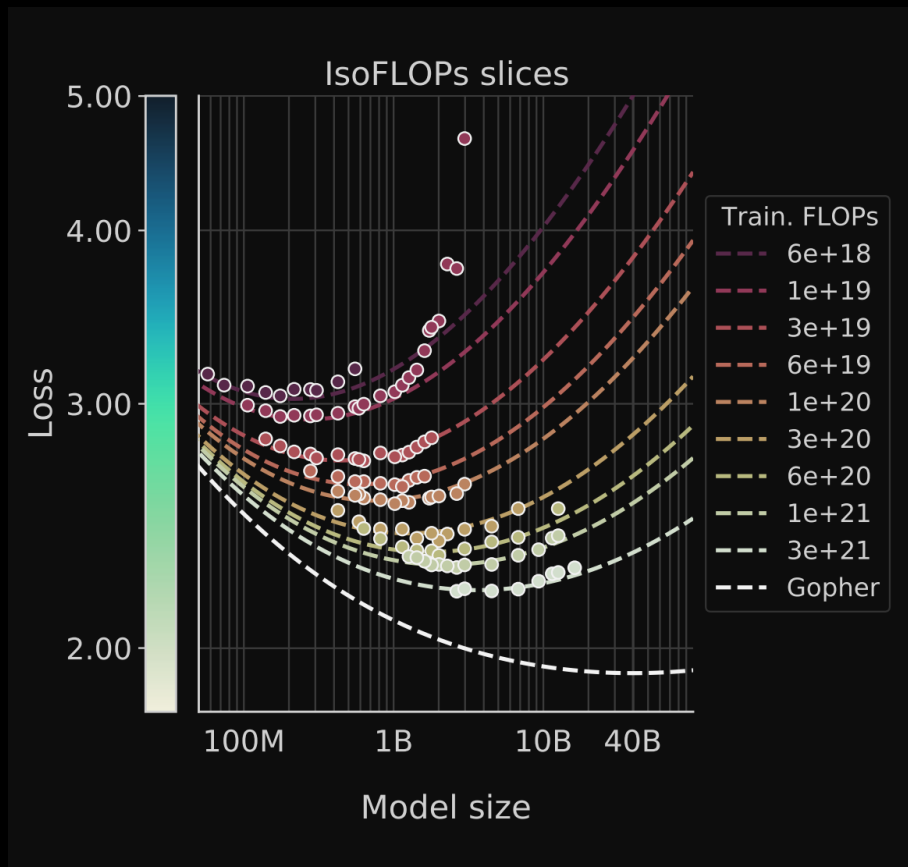
Scaling of LLMs

For LLMs that have been designed to scale well (avoiding overfitting and vanishing gradients, for example), we find that performance is a predictable function of:

- The dataset size
- The number of parameters

And these curves show no signs of ending yet.

So, in these applications we do expect better performance through brute force scaling.



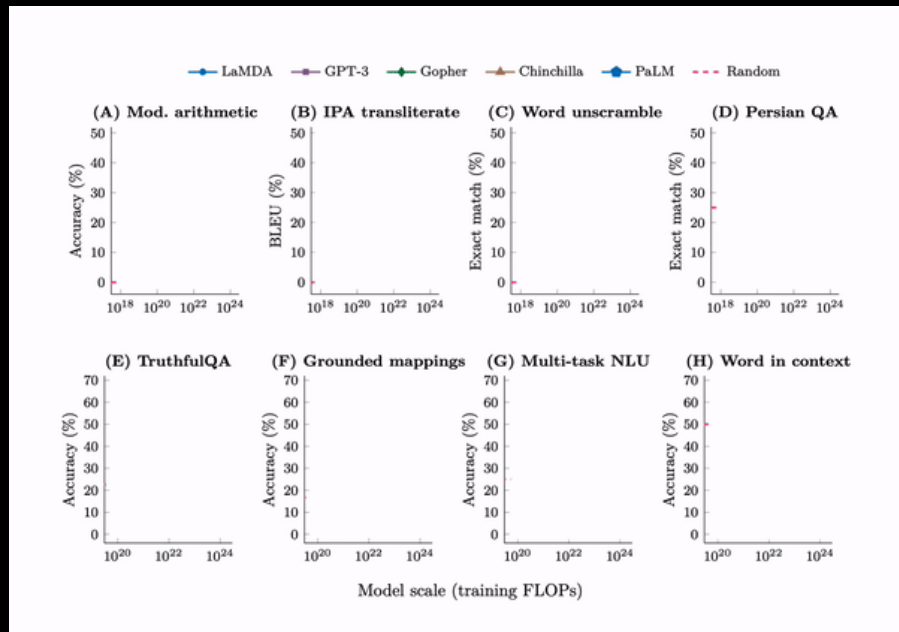
Emergent Capability

An emergent ability as an ability that is not present in small models but develops as the model is scaled. These might be unanticipated.

Some areas of science are familiar with this idea (physics and biology, for sure). In computer science this concept has largely been a novelty (Conway's Game of Life is a notable example). In Deep Learning, it has become a very significant phenomena.

Once again, bear in mind that many of the principles we have mentioned (overfitting, vanishing gradients, etc.) mean that brute force scaling is not going to be a default route to better performance.

Instead, understanding of those principles will allow you the option to scale.



<https://www.jasonwei.net/blog/emergence>

Also a good associated paper.

PyTorch?

```
import os
os.environ["KERAS_BACKEND"] = "torch"
```

```
import keras
```

```
mnist = keras.datasets.mnist
(train_images, train_labels), (test_images, test_labels) = mnist.load_data()
```

```
train_images = train_images.reshape(60000, 28, 28, 1)
test_images = test_images.reshape(10000, 28, 28, 1)
train_images, test_images = train_images/255, test_images/255
```

```
model = keras.Sequential([
    keras.layers.Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),
    keras.layers.Conv2D(64, (3,3), activation='relu'),
    keras.layers.MaxPooling2D(2,2),
    keras.layers.Dropout(0.25),
    keras.layers.Flatten(),
    keras.layers.Dense(128, activation='relu'),
    keras.layers.Dropout(0.5),
    keras.layers.Dense(10, activation='softmax')
])
```

```
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])
```

```
model.fit(train_images, train_labels, batch_size=128, epochs=15, verbose=1, validation_data=(test_images, test_labels))
```



This could be done in the shell.

Use the Latest Environment

```
interact -p GPU-shared --gres=gpu:h100-80:1
```

```
singularity shell --nv /ocean/containers/ngc/pytorch/pytorch_latest.sif
```

```

from __future__ import print_function
import argparse
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
from torchvision import datasets, transforms
from torch.optim.lr_scheduler import StepLR

```

```

class Net(nn.Module):
    def __init__(self):
        super(Net, self).__init__()
        self.conv1 = nn.Conv2d(1, 32, 3, 1)
        self.conv2 = nn.Conv2d(32, 64, 3, 1)
        self.dropout1 = nn.Dropout2d(0.25)
        self.dropout2 = nn.Dropout2d(0.5)
        self.fc1 = nn.Linear(9216, 128)
        self.fc2 = nn.Linear(128, 10)

```

```

    def forward(self, x):
        x = self.conv1(x)
        x = F.relu(x)
        x = self.conv2(x)
        x = F.relu(x)
        x = F.max_pool2d(x, 2)
        x = self.dropout1(x)
        x = torch.flatten(x, 1)
        x = self.fc1(x)
        x = F.relu(x)
        x = self.dropout2(x)
        x = self.fc2(x)
        output = F.log_softmax(x, dim=1)
        return output

```

```

def train(args, model, device, train_loader, optimizer, epoch):
    model.train()
    for batch_idx, (data, target) in enumerate(train_loader):
        data, target = data.to(device), target.to(device)
        optimizer.zero_grad()
        output = model(data)
        loss = F.nll_loss(output, target)
        loss.backward()
        optimizer.step()
        if batch_idx % args.log_interval == 0:
            print('Train Epoch: {} [{}/{}] ({:.0f}%) \tLoss: {:.6f}'.format(
                epoch, batch_idx * len(data), len(train_loader.dataset),
                100. * batch_idx / len(train_loader), loss.item()))

```

```

def test(args, model, device, test_loader):
    model.eval()
    test_loss = 0
    correct = 0
    with torch.no_grad():
        for data, target in test_loader:
            data, target = data.to(device), target.to(device)
            output = model(data)
            test_loss += F.nll_loss(output, target, reduction='sum').item() # sum up batch loss
            pred = output.argmax(dim=1, keepdim=True) # get the index of the max log-probability
            correct += pred.eq(target.view_as(pred)).sum().item()

    test_loss /= len(test_loader.dataset)

```

Old School PyTorch CNN MNIST

Not a fair comparison of terseness as this version has a
lot of extra flexibility.

From:
<https://github.com/pytorch/examples/blob/master/mnist/main.py>

```

model = Net().to(device)
optimizer = optim.Adadelta(model.parameters(), lr=args.lr)

```

```

scheduler = StepLR(optimizer, step_size=1, gamma=args.gamma)
for epoch in range(1, args.epochs + 1):
    train(args, model, device, train_loader, optimizer, epoch)
    test(args, model, device, test_loader)
    scheduler.step()

```

```

if args.save_model:
    torch.save(model.state_dict(), "mnist_cnn.pt")

```

```

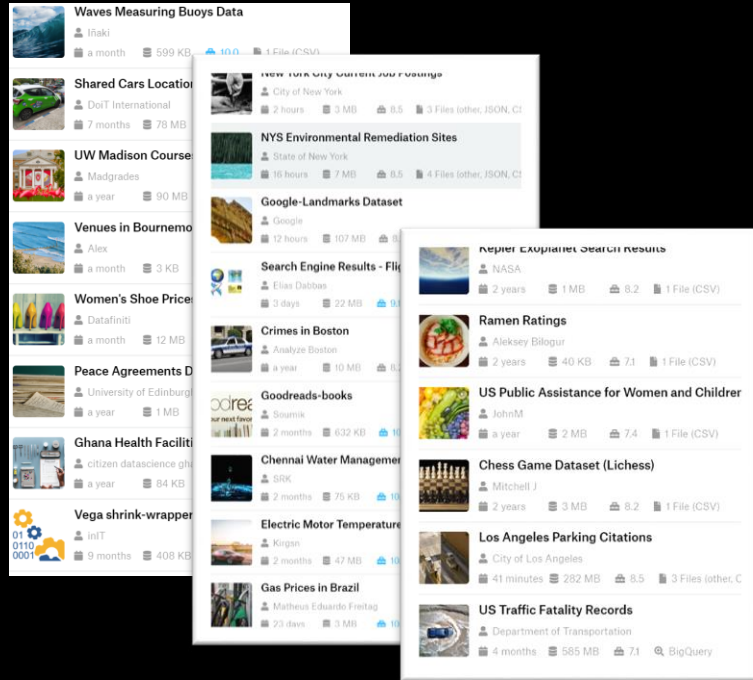
if __name__ == '__main__':
    main()

```

Endless Datasets and Challenges

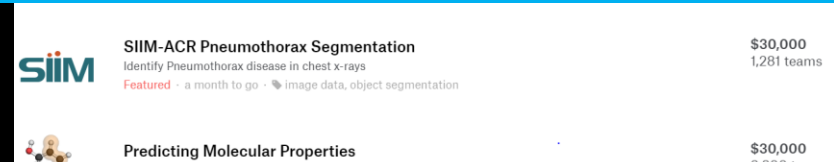
Kaggle Challenge

The benchmark driven nature of deep learning research, and its competitive consequences, have found a nexus at Kaggle.com. There you can find over 500,000 datasets:

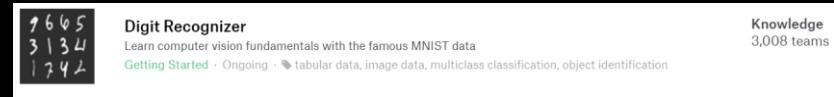


Hugging Face

- Also a lot of datasets (450,000)
- and models (1,900,000)
- No competitions
- A little more profit driven
- and a lot more advanced/confusing.



Including this one:



Deeper Learning

John Urbanic

Parallel Computing Scientist
Pittsburgh Supercomputing Center

Distinguished Service Professor
Carnegie Mellon University

Building Blocks

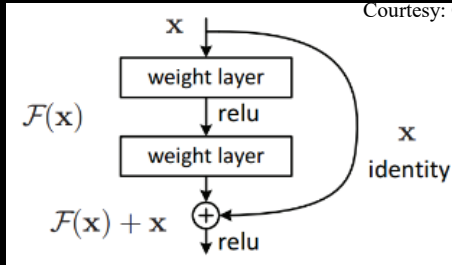
So far, we have used Fully Connected and Convolutional layers. These are ubiquitous, but there are many others:

- Fully Connected (FC)
- Convolutional (CNN)
- Residual (ResNet) [Feed forward]
- Recurrent (RNN), [Feedback, but has vanishing gradients so...]
- Long Short Term Memory (LSTM)
- Bidirectional LSTM
- Transformer (Attention based)
- Restricted Boltzmann Machine
-
-

Several of these are particularly common...

Residual Neural Nets

We've mentioned that disappearing gradients can be an issue, and we know that deeper networks are more powerful. How do we reconcile these two phenomena? One, very successful, method is to use some feedforward.



- Helps preserve reasonable gradients for very deep networks
- Very effective at imagery
- Used by AlphaGo Zero (40 residual CNN layers) in place of previous complex dual network
- 100s of layers common, Pushing 1000

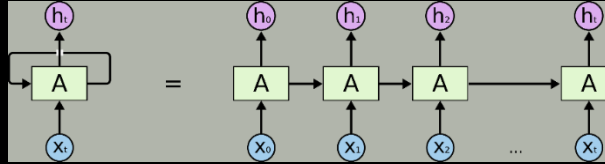
```
#Example: input 3-channel 256x256 image
x = Input(shape=(256, 256, 3))
y = Conv2D(3, (3, 3))(x)
z = keras.layers.add([x, y])
```

Haven't all of our Keras networks been built as strict layers in a *sequential* method? Indeed, but Keras supports a *functional* API that provides the ability to define network that branch in other ways (multiple inputs or multiple outputs, or layers with multiple inputs or multiple outputs, or any non-linear topology such as here). It is easy and here (<https://www.tensorflow.org/guide/keras/functional>) is an MNIST example with a 3 dense layers.

More to our current point, here (<https://www.kaggle.com/yadavsarthak/residual-networks-and-mnist>) is a neat experiment that uses 15(!) residual layers to do MNIST. Not the most effective approach, but it works and illustrates the concept beautifully.

Recurrent Networks (RNNs)

If feedforward is useful, is there a place for feedback? Indeed, it is currently at the center of the many of the most effective techniques in deep learning.



Many problems occur in some context. Our MNIST characters are just pulled from a hat. However most character recognition has some context that can greatly aid the interpretation, as suggested by the following - not quite true - text.

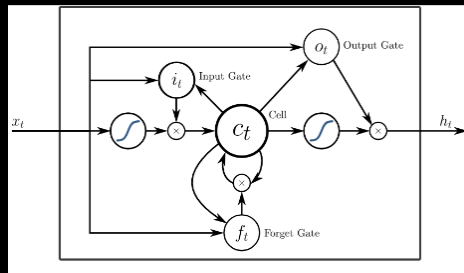
"Aoccdrnig to a rscheearch at Cmabrigde Uinervtisy, it deosn't mttar in waht oredr the ltteers in a wrod are, the olny iprmoatnt tihng is taht the frist and lsat ltteers be at the rghit pclae. The rset can be a toatl mses and you can sitll raed it wouthit porbelm. Tihs is bcuseae the huamn mnid deos not raed ervey lteter by istlef, but the wrod as a wlohe."

To pick a less confounding example. The following smudged character is pretty obvious by its context. If our network can "look back" to the previous words, it has a good chance at guessing the, otherwise unreadable, "a".

The dog chased the  at up the tree.

LSTMs

This RNN idea seems an awful lot like "memory", and suggests that we might actually incorporate a memory into networks. While the Long Short Term Memory (LSTM) idea was first formally proposed in 1997 by Hochreiter and Schmidhuber, it has taken on many variants since. This is often not explained and can be confusing if you aren't aware. I recommend "LSTM: A Search Space Odyssey" (Greff, et. al.) to help.



The basic design involves a memory cell, and some method of triggering a forget. `tf.keras.layers.LSTM` takes care of the details for us (but has a *lot* of options).

The Keras folks even provide us with an MNIST version (https://keras.io/examples/mnist_hierarchical_rnn/), although I think it is confusing as we are now killing a fly with a bazooka.

I recommend https://keras.io/examples/conv_lstm/, which uses network is used to predict the next frame of an artificially generated movie which contains moving squares. A much more natural fit.

Bi-directional LSTMs

Often, and especially in language processing, it is helpful to see both forward and backward. Take this example:

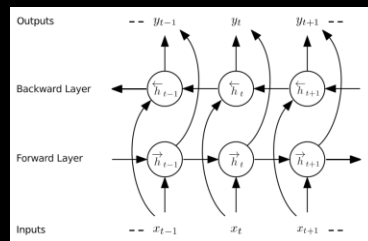
The dog chased the cat

Is the dog chasing a cat, or a car? If we read the rest of the sentence, it is obvious:

The dog chased the cat up the tree.

Adding even this very sophisticated type of network is easy in TF. Here is the network definition from the Keras *IMDB movie review sentiment analysis* example (https://www.tensorflow.org/tutorials/text/text_classification_rnn).

```
model = tf.keras.Sequential([  
    tf.keras.layers.Embedding(encoder.vocab_size, 64),  
    tf.keras.layers.Bidirectional(tf.keras.layers.LSTM(64, return_sequences=True)),  
    tf.keras.layers.Dense(64, activation='relu'),  
    tf.keras.layers.Dropout(0.5),  
    tf.keras.layers.Dense(1)  
])
```



The first, embedding, layer introduces the concept of **word embeddings** - of central importance to any of you interested in natural language processing, and related to our running theme of **dimensionality reduction**. To oversimplify, here we are asking TF to reduce our vocabulary of `vocab_size`, so that every word's meaning is represented by a 64 dimensional vector.



Transformers

We have strayed solidly into the realm of Natural Language Processing (NLP). The current state of the art here, which has largely subsumed these earlier techniques, are Transformer, or self-attention based networks. These form the basis of ChatGPT and similar applications.

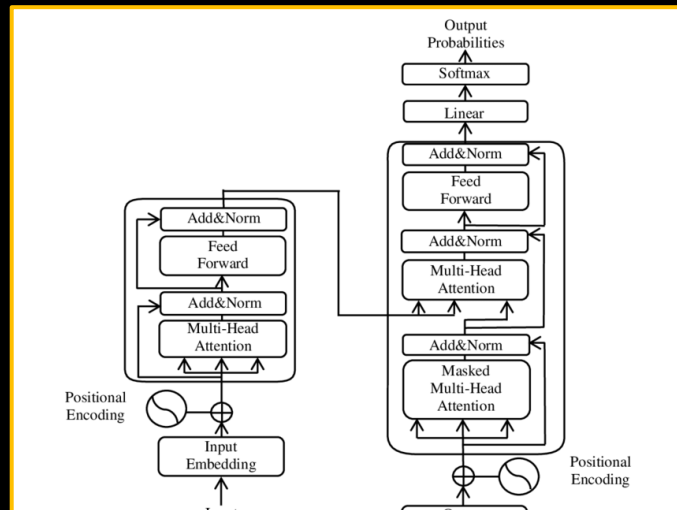
The seminal implementation goes all the way back to 2017 with “*Attention Is All You Need*,” Vaswani et al.

While we use the basic building blocks you have learned, the overall architectures have a lot of parts.

The idea is to process the kind of sequential data we have been discussing, but with the ability to learn the relative importance of different, perhaps distant, tokens. In other words, pay more attention to some relationships than others.

While these designs have proven surprisingly powerful in NLP (*emergent!* *emergent!*), they have yet to find a central use in scientific problems.

Given the incredible effort and funding involved, it is likely that the scientific community will find some



Once again, 3Blue1Brown

<https://www.youtube.com/watch?v=wjZofJX0v4M>

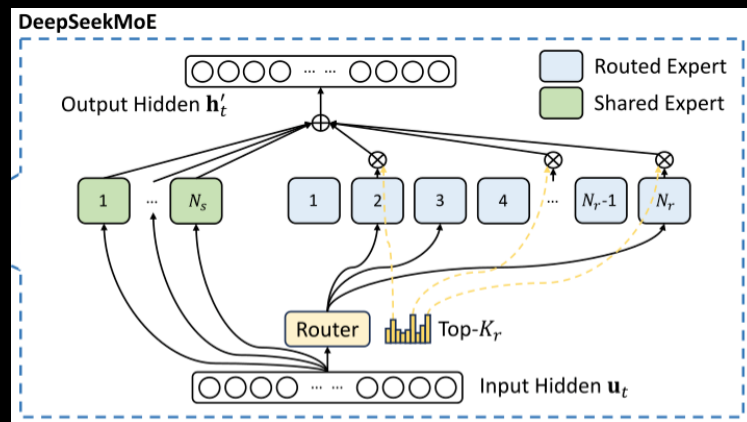
Mixture of Experts

And now we have reached the point where our 100+ billion parameter models are both expensive to train and to operate. As transformers in particular grow larger, their feedforward portions come to dominate, with 90+% of the parameters in the feedforward layers.

A technique that dates back to 1991 ("Adaptive Mixture of Local Experts") has become a very effective way to mitigate this issue.

MoE's work by designating a number of experts, each its own sub-network within a larger neural network, and training a gating network to activate only the specific experts best suited to a given input.

This overall parameter count is commonly referenced as the *sparse parameter count* and can generally be understood as a measure of model capacity as well as how much memory your model will actually require. The number of parameters that will actually be used to process an individual token (as it transits through some expert blocks and bypasses others) is called the *active parameter count*, and can be understood as a measure of the model's inference cost.



Mixture of Experts

As we have already mentioned numerous times, GPU training hours have become the limiting resource. Training a fully dense model has become prohibitively expensive in terms of both time and cost. The Llama 2 set of models trained by Meta reportedly spent 3.3 million NVIDIA A100 GPU hours in training. This is roughly 3.3 million GPU hours on 1,024 GPUs at full capacity. With no downtime this would take over *3 months*! This does not account for any hyperparameter investigation at all. So even though they introduce complexity, the ability for MoEs to greatly reduce training time has made them essential in this space.

As an example of this complexity: using the most obvious top-expert routing strategy introduces the potential for the gating network to converge to activating just a few experts. This is a self-reinforcing problem: if a handful of experts are disproportionately selected early on, those experts will be trained more quickly, and then continue to be selected more as they now output more reliable predictions than the other, less-trained experts.

MoE's are also very effective at inference time, which is becoming an increasing resource sink. Not only do they decrease the overall computation required, they also reduce latency. Latency decreases are particularly important in use cases like retrieval-augmented generation (RAG) and autonomous agents, which may require many calls to the model, compounding single-call latency.

In summary, the primary attraction of the MoE approach is that by enforcing sparsity, rather than activating the entire neural network for each input token, model capacity can be increased while essentially keeping computational costs constant.

Foundation Models

The most pragmatic solution to these great training costs is to build on top of existing models. Foundation Models are trained on broad data (and are often multi-model) and only require fine-tuning for a specialized task. Most of the well-known LLMs are foundation models.

Fine tuning consists of additional training to target a specific application. Keeping some of the parameters fixed or frozen is an effective way to decrease the fine-tuning training cost of these very large models. This can also help to preserve the generalization of the original model.

Two related techniques are often employed in helping to make these very large foundation models less resource intensive. Quantization is where the parameter precision of the original model is reduced, and distillation is where a smaller model is trained to mimic the larger model.

Foundation models aren't just the giant LLMs. There are many more modest ones. For example, the YOLO object detection network is very popular as a starting point for many image recognition applications. There are many different versions which can be selected as a foundation for fine tuning to a specific application.

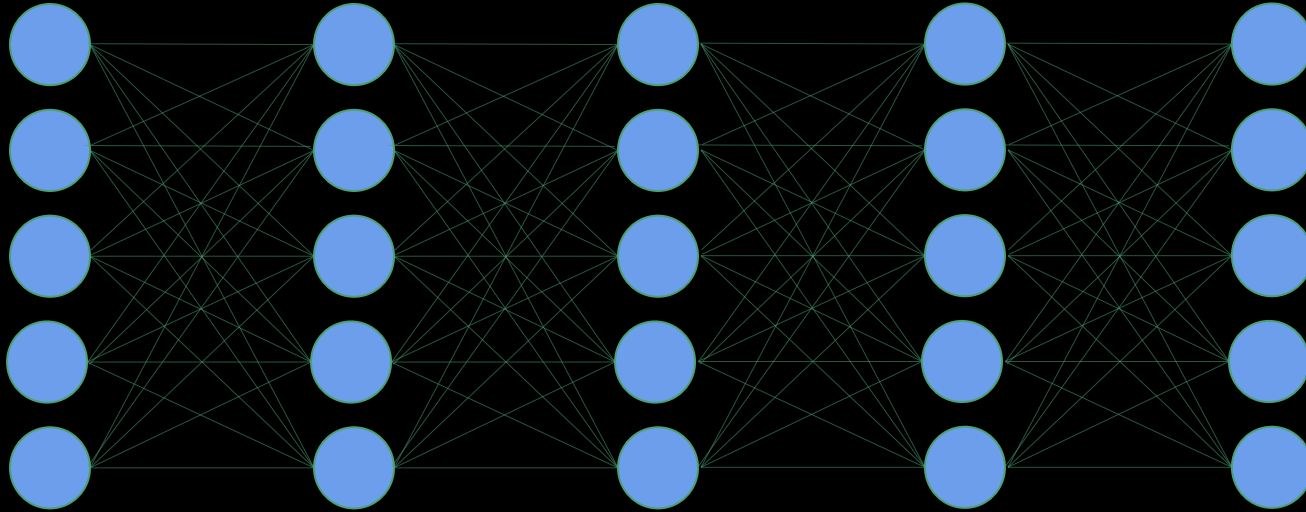
Autoencoder



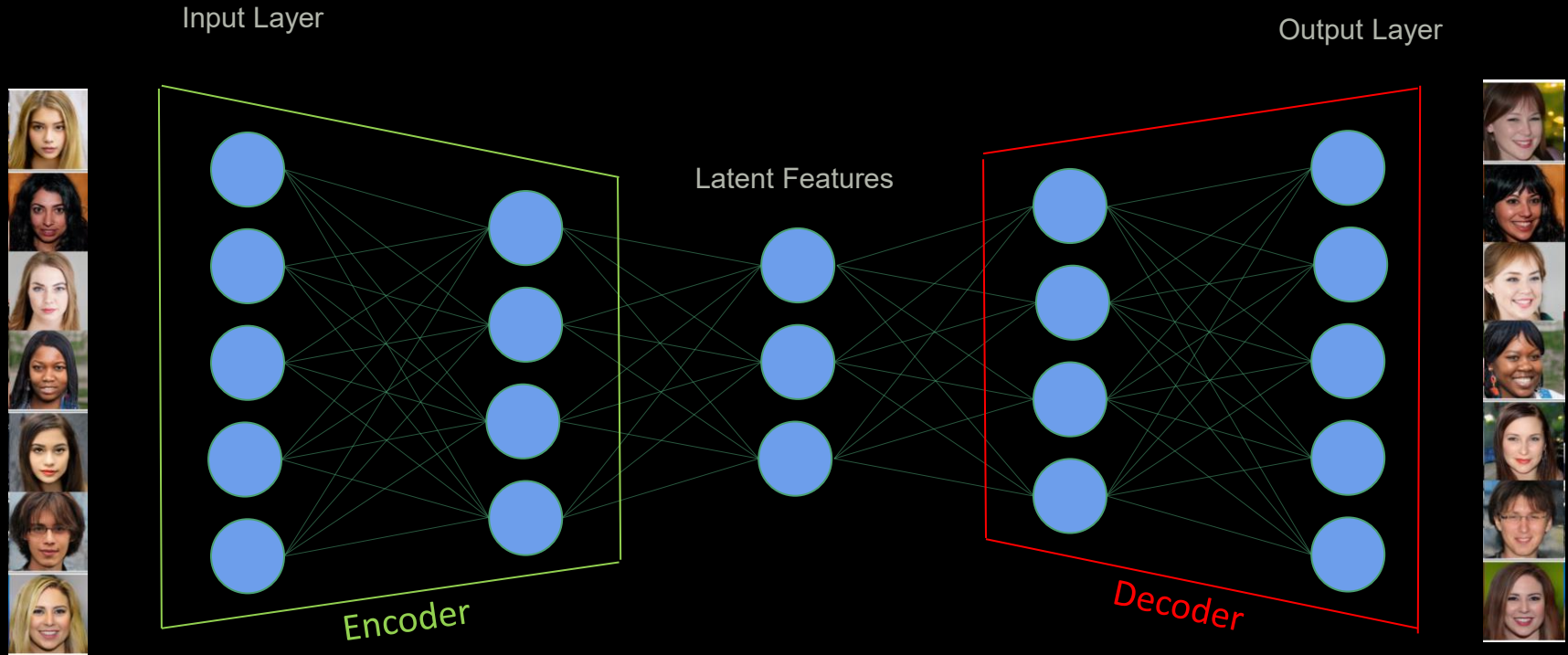
Input Layer

Hidden Layers

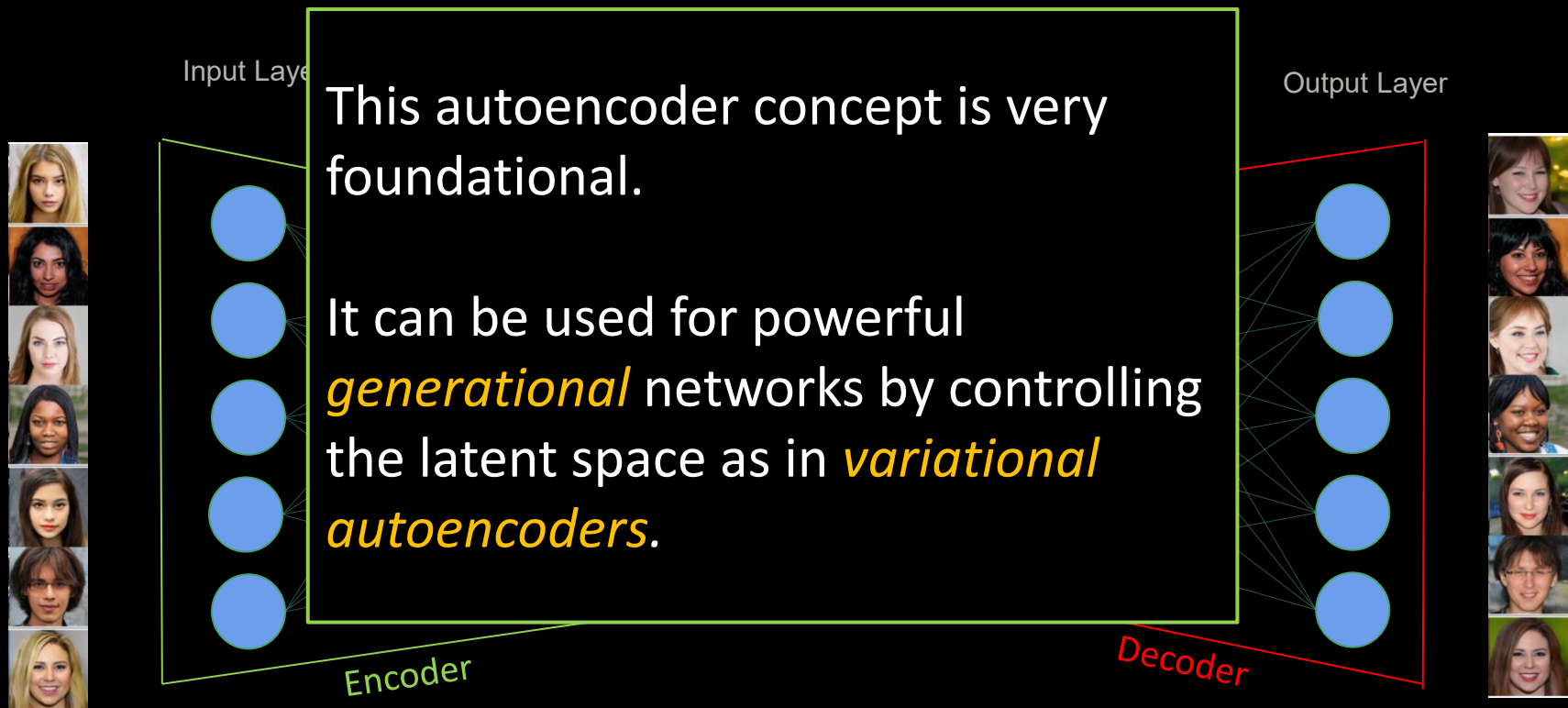
Output Layer



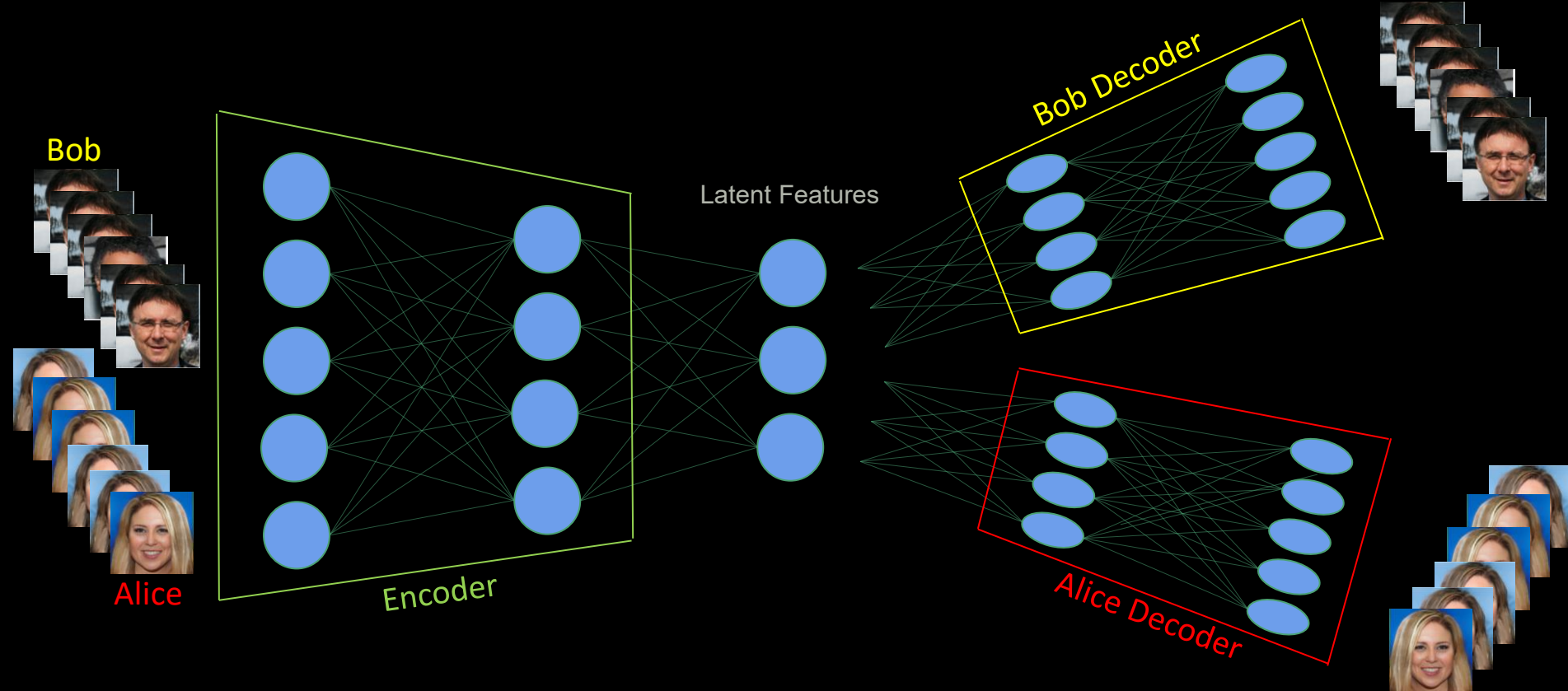
Autoencoder



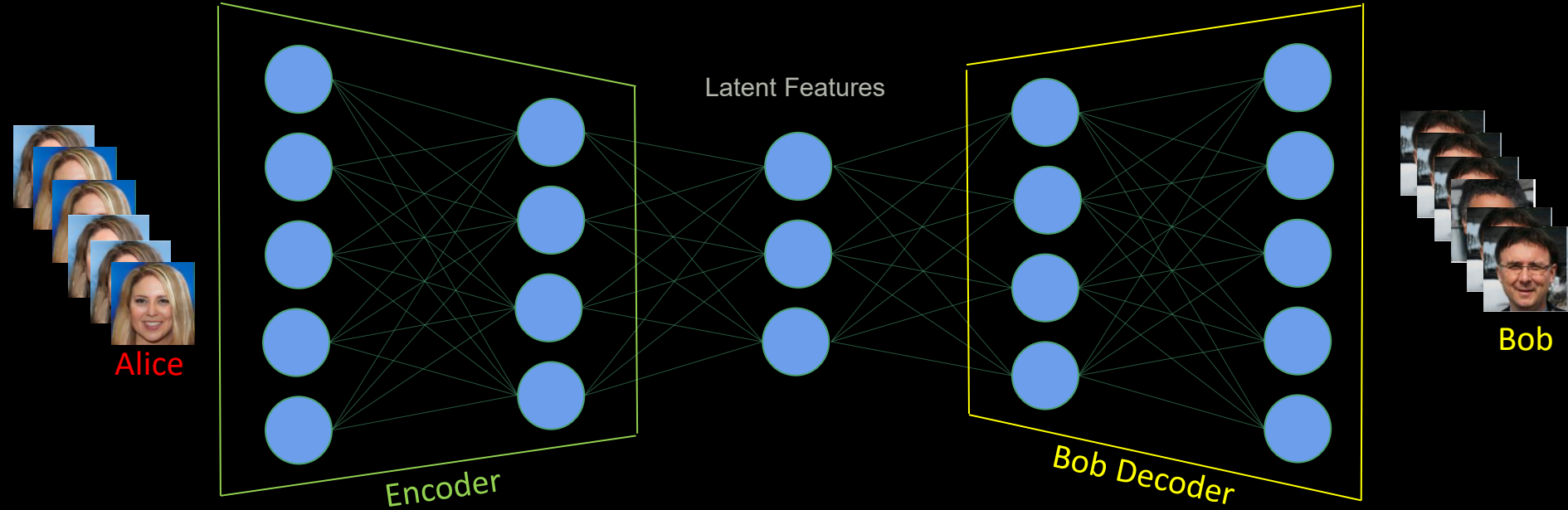
Autoencoder



Deepfake Training



Deepfake At Work



Rando on Reddit

<https://github.com/hacksider/Deep-Live-Cam>

As he says, "me with a laptop in my bedroom."

Deep Fake I'm working on. Running on python 3.10 in real time. I struggled to find the correct drivers but ended up getting a good result.



Super Resolution

VideoGigaGAN: Towards Detail-rich Video Super-Resolution

Yiran Xu^{1,2}, Taesung Park¹, Richard Zhang¹, Yang Zhou¹, Eli Shechtman¹,
Feng Liu¹, Jia-Bin Huang², and Difan Liu¹

¹ Adobe Research

² University of Maryland, College Park
<http://videogigagan.github.io>



Fig. 1: We present **VideoGigaGAN**, a generative video super-resolution model that can upsample videos with high-frequency details while maintaining temporal consistency. *Top:* we show the comparison of our approach with TTVSR [33] BasicVSR++ [7]. Our method produces temporally consistent videos with more fine-grained details than previous methods. *Bottom:* our model can produce high-quality videos with 8× super-resolution. Please see the video results on our [project page](http://videogigagan.github.io).

3.6 Loss functions

We use standard, non-saturating GAN loss [14], R1 regularization [34], LPIPS [62] and Charbonnier loss [9] during the training.

$$\mathcal{L}(\mathbf{X}_t, \mathbf{x}_t) = \mu_{GAN} \mathcal{L}_{GAN}(\mathcal{G}(\mathbf{x}_t), \mathcal{D}(\mathcal{G}(\mathbf{x}_t))) + \mu_{R1} \mathcal{L}_{R1}(\mathcal{D}(\mathbf{X}_t)) + \mu_{LPIPS} \mathcal{L}_{LPIPS}(\mathbf{X}_t, \mathbf{x}_t) + \mu_{Char} \mathcal{L}_{Char}(\mathbf{X}_t, \mathbf{x}_t), \quad (2)$$

where Charbonnier loss is a smoothed version of pixelwise ℓ_1 loss, $\mu_{GAN}, \mu_{R1}, \mu_{LPIPS}, \mu_{Char}$ are the scales of different loss functions. $\mathbf{x}_t$ is one of the LR input frames, $\mathbf{X}_t$ is the corresponding ground-truth HR frame. We average the loss over all the frames in a video clip during the training.

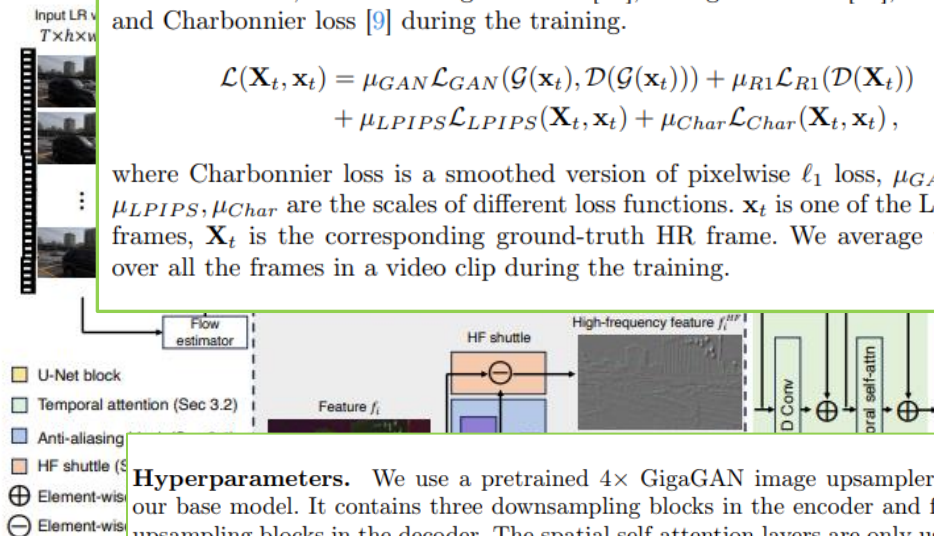


Fig. 3: Overview

(VSR) model is a GigaGAN upsampler into decoder blocks. flow-guided pro Anti-aliasing bl

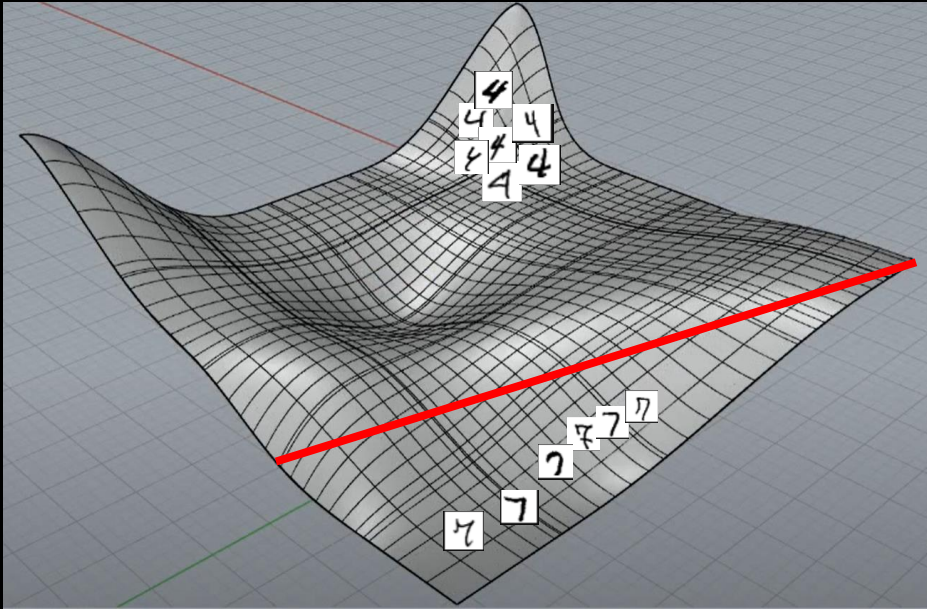
Hyperparameters. We use a pretrained 4× GigaGAN image upsampler as our base model. It contains three downsampling blocks in the encoder and five upsampling blocks in the decoder. The spatial self-attention layers are only used in the first block of the decoder for memory efficiency. For the flow network,

we use a lightweight SpyNet [38]. For the low-pass filters, we use a kernel of $\frac{1}{16} [1, 4, 6, 4, 1]$ before the downsampling. We set $\mu_{GAN} = 0.05$, $\mu_{R1} = 0.2048$, $\mu_{LPIPS} = 5$, $\mu_{Char} = 10$ in Eqn. 2. During training, we randomly crop a 64×64 patch from each LR input frame at the same location. We use 10 frames of each video and a batch size of 32 for training. The batch is distributed into 32 NVIDIA A100 GPUs. We use a fixed learning rate of 5×10^{-5} for both generator and discriminator. The total number of training iterations is 100,000.

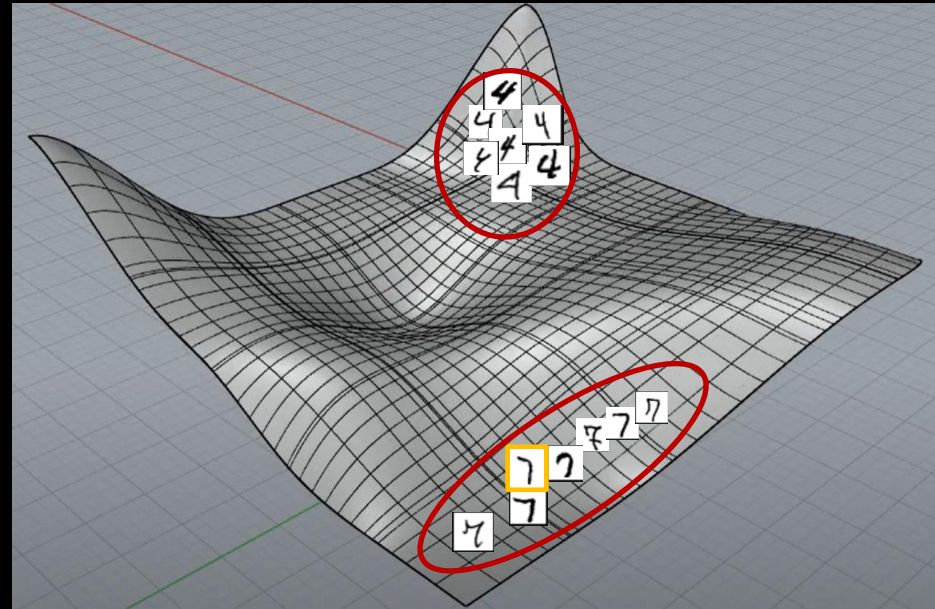
shuttle the high frequency features via skip connection to the decoder layers to compensate for the loss of details in the BlurPool process.

Discriminative vs. Generative

Discriminative models classify things, and need only know which side of the hyper-plane the instance lies on. Generative models need to understand the distribution to generate new instances.



Discriminative



Generative

Discriminative models need only capture the conditional probability of digit Y , given image X : $P(Y|X)$. Generative models must understand the joint probability $P(X,Y)$.

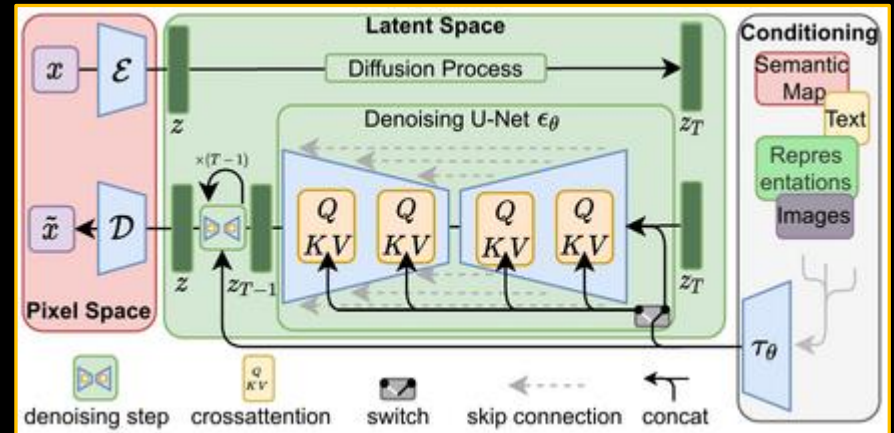
Generative in Action

Stable Diffusion, DALL-E, Midjourney and other such applications are built upon this idea.

For example, Stable Diffusion was trained on pairs of images and captions from Common Crawl data scraped from the web, where 5 billion image-text pairs were classified.

In a clever mashup of ideas we have discussed, this network attempts to de-noise images in conjunction with text prompts, resulting in some amazing "comprehension".

Stable Diffusion's code and model weights have been released, and it runs on consumer GPUs with 8 GB of VRAM!



Meet DALL-E, the AI That Draws Anything at

New technology that graphic artists — and

Give this article



Image Generated by AI

DALL-E generated this image from a prompt: "a kitten playing chess." OpenAI

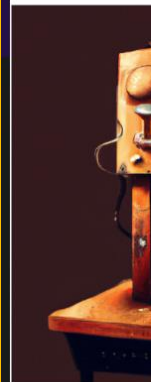


Image Generated by AI

DALL-E generated these images by following a command, or a prompt, in the simplest of sentences. OpenAI



vibrant portrait painting



an espresso machine that makes coffee from human souls, artstation

Hierarchical Text-Conditional Image Generation with CLIP Latents

Aditya Ramesh*
OpenAI
aramesh@openai.com

Prafulla Dhariwal*
OpenAI
prafulla@openai.com

Alex Nichol*
OpenAI
alex@openai.com

Casey Chu*
OpenAI
casey@openai.com

Mark Chen
OpenAI
mark@openai.com

Abstract

Contrastive models like CLIP have been shown to learn robust representations of images that capture both semantics and style. To leverage these representations for image generation, we propose a two-stage model: a prior that generates a CLIP image embedding given a text caption, and a decoder that generates an image conditioned on the image embedding. We show that explicitly generating image representations improves image diversity with minimal loss in photorealism and caption similarity. Our decoders conditioned on image representations can also produce variations of an image that preserve both its semantics and style, while varying the non-essential details absent from the image representation. Moreover, the joint embedding space of CLIP enables language-guided image manipulations in a zero-shot fashion. We use diffusion models for the decoder and experiment with both autoregressive and diffusion models for the prior, finding that the latter are computationally more efficient and produce higher-quality samples.

1 Introduction

Recent progress in computer vision has been driven by scaling models on large datasets of captioned images collected from the internet [10, 44, 60, 39, 31, 16]. Within this framework, CLIP [39] has emerged as a successful representation learner for images. CLIP embeddings have a number of desirable properties: they are robust to image distribution shift, have impressive zero-shot capabilities, and have been fine-tuned to achieve state-of-the-art results on a wide variety of vision and language tasks [45]. Concurrently, diffusion models [46, 48, 25] have emerged as a promising generative modeling framework, pushing the state-of-the-art on image and video generation tasks [11, 26, 24]. To achieve best results, diffusion models leverage a guidance technique [11, 24] which improves sample fidelity (for images, photorealism) at the cost of sample diversity.

In this work, we combine these two approaches for the problem of text-conditional image generation. We first train a diffusion *decoder* to invert the CLIP image *encoder*. Our inverter is non-deterministic, and can produce multiple images corresponding to a given image embedding. The presence of an encoder and its approximate inverse (the decoder) allows for capabilities beyond text-to-image translation. As in GAN inversion [62, 55], encoding and decoding an input image produces semantically similar output images (Figure 3). We can also interpolate between input images by inverting interpolations of their image embeddings (Figure 4). However, one notable advantage of using the CLIP latent space is the ability to semantically modify images by moving in the direction of any encoded text vector (Figure 5), whereas discovering these directions in GAN latent space involves

*Equal contribution



a close up of a handpalm with leaves growing from it



a corgi's head depicted as an explosion of a nebula

panda mad scientist mixing sparkling chemicals, artstation

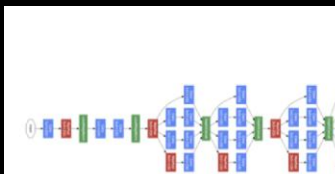
Architectures

With the layers we have discussed, we can build countless different networks (and use TensorFlow to define them). Indeed, you may get the feel that the current "building block" is actually a functional network.

Generative Adversarial Network

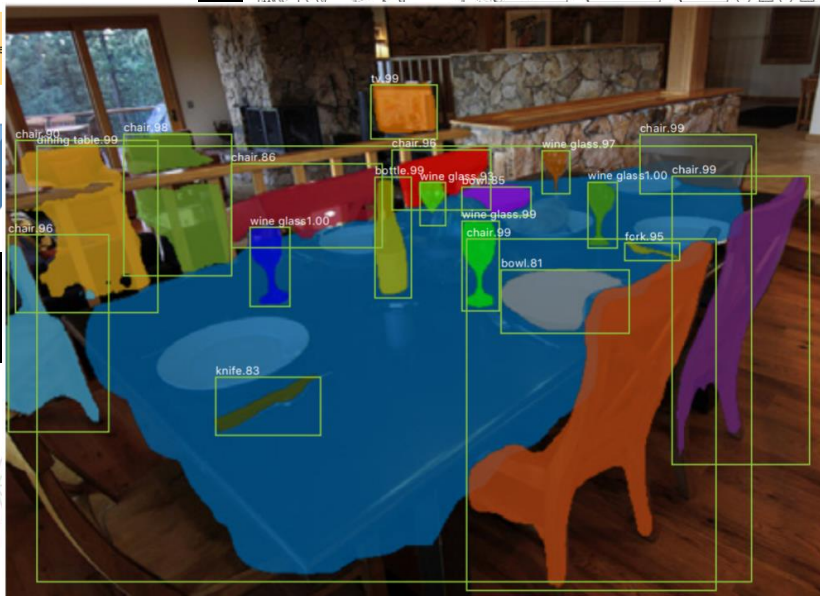
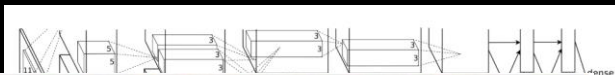


Credit: O'Reilly

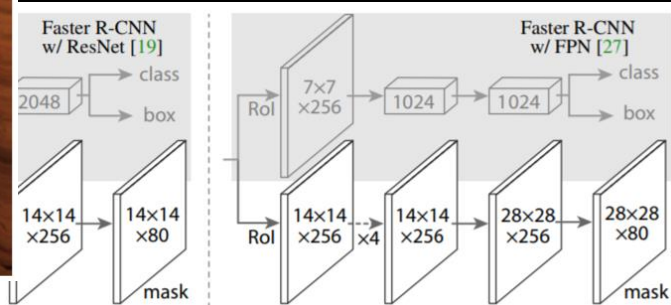
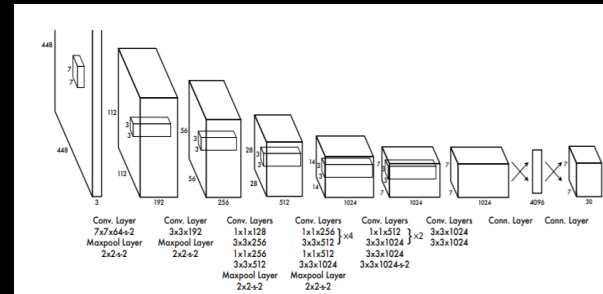


GoogLeNet / Inception

Convolution
Pooling
Softmax
Other



YOLO (You Only Look Once)



Mask R-CNN

Some Taxonomies

So far we have focused on images, and their classification. You know that deep learning has had success across a wide, and rapidly expanding, number of domains. Even our digit recognition task could be more sophisticated:

- Classification (What we did)
- Localization (Where is the digit?)
- Detection (Are there digits? How many?)
- Segmentation (Which pixels are the digits?)

These tasks would call for different network designs. This is where our Day 3 would begin, and we would use some other building blocks.

As you learn more about machine learning, you will see various ways to categorize the algorithms or tasks or general approaches to doing something useful. Don't believe any of them are either comprehensive or canonical. They are just useful ways to keep track of the explosion of options in this space.

Tasks

Classification	What we've been doing.
Regression	Return a value. <i>Stock price.</i>
Transcription	Convert between representations. <i>OCR, speech recognition.</i>
Synthesis	Create new input examples. <i>Speech synthesizer. Lots of science these days!</i>
Translation	Like the word says. <i>Google Translate.</i>
Segmentation	Return a relabeled input vector. <i>Tumor detection.</i>
Denoising	Return uncorrupted example. <i>Video game ray tracing.</i>

Again, neither comprehensive nor definitive. The definitions vary from one author to the next, and the list grows all the time.

Learning Approaches

Supervised Learning

How you learned colors.

What we have been doing just now.

Used for: image recognition, tumor identification, segmentation.

Requires labeled data. Lots of it. Augmenting helps.

Essence: Learning to map one vector to another, given enough examples of the mapping.

Unsupervised Learning

(Maybe) how you learned to see.

What we did earlier with clustering and our recommender, and Deepfake.

Find patterns in data, compress data into model, find reducible representation of data.

Used for: Learning from unlabeled data.

Might be a great way to bootstrap Supervised Learning (train an autoencoder and build from those weights).

Reinforcement Learning

How you learned to walk.

Requires goals (maybe long term, i.e. arbitrary delays between action and reward).

Used for: Go (AlphaGo Zero), robot motion, video games.

Don't just read data, but *interact* with it!

Fuzzy Line

All of these have been done with and without deep learning. DL has moved to the forefront of all of these.

AI Based Simulation?

A wise man once (not that long ago) told me "John, I don't need a neural net to rediscover conservation of energy."

Model-Free Prediction of Large Spatiotemporally Chaotic Systems from Data: A Reservoir Computing Approach

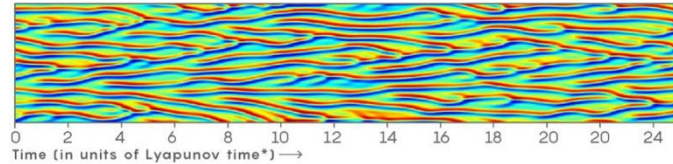
Jaideep Pathak, Brian Hunt, Michelle Girvan, Zhixin Lu, and Edward Ott
Phys. Rev. Lett. 120, 024102 – Published 12 January 2018

Training Computers to Tame Chaos

A machine-learning algorithm has been shown to accurately predict a chaotic system far further into the future than previously possible.

A Chaos Model

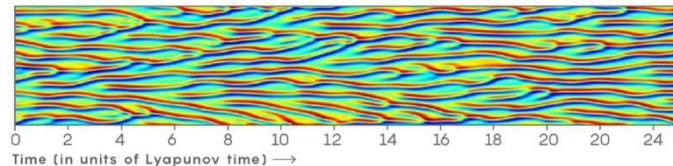
Researchers started with the evolving solution to the Kuramoto-Sivashinsky equation, which models propagating flames:



* Lyapunov time = Length of time before a small difference in the system's initial state begins to diverge exponentially. It typically sets the horizon of predictability, which varies from system to system.

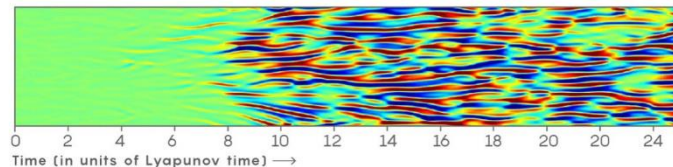
B Machine Learning

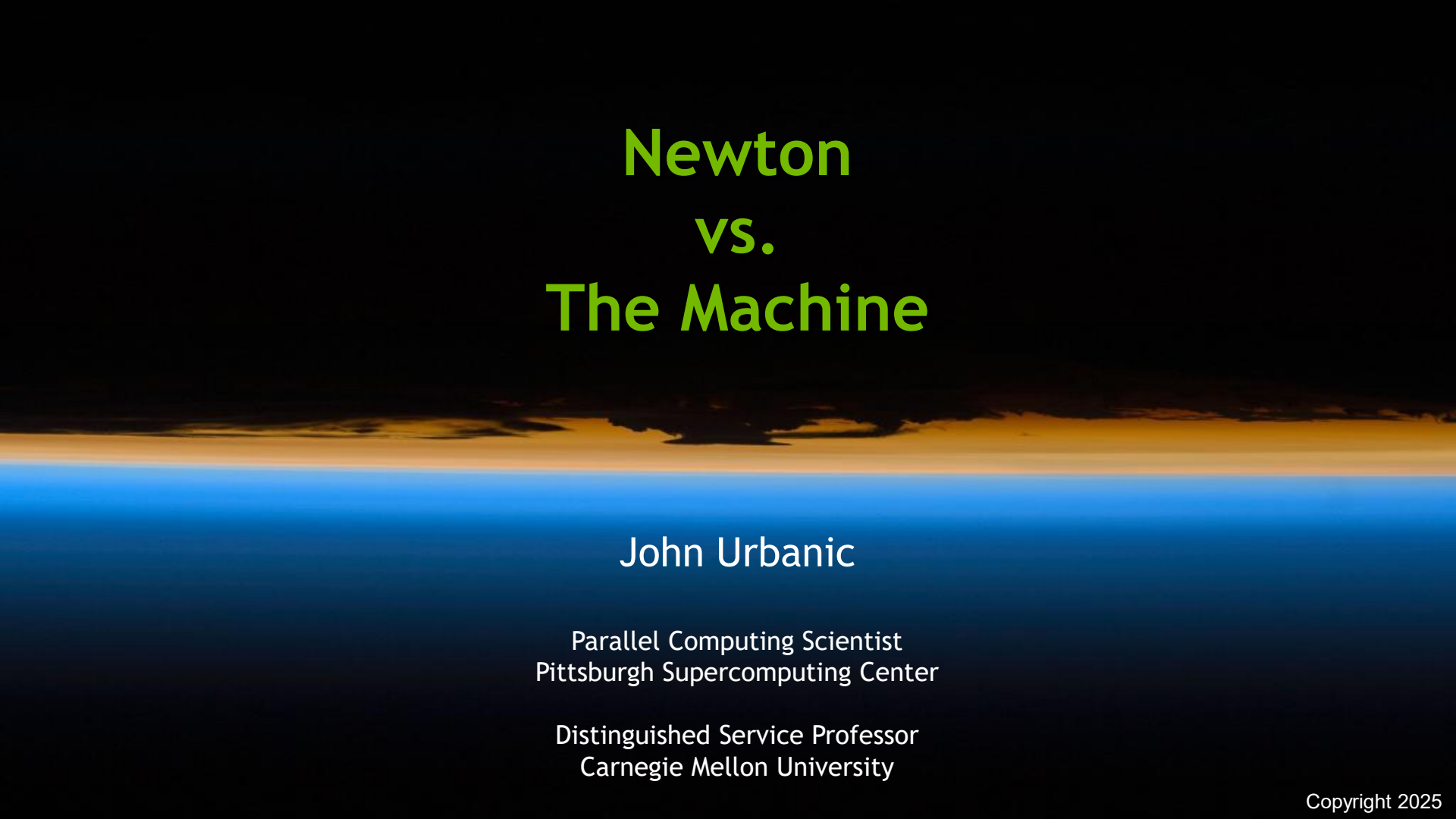
After training itself on data from the past evolution of the Kuramoto-Sivashinsky system, the "reservoir computing" algorithm predicts its future evolution:



A - B Do They Match?

Subtracting B from A shows that the algorithm accurately predicts the model out to an impressive 8 Lyapunov times, before chaos ultimately prevails:





Newton vs. The Machine

John Urbanic

Parallel Computing Scientist
Pittsburgh Supercomputing Center

Distinguished Service Professor
Carnegie Mellon University

But First:

Some Assignment #5 Solutions

Where to start?

Step 1: What do I mean by "better network"? This sounds like an actual research assignment! Well, if I run this

```
import tensorflow as tf

mnist = tf.keras.datasets.fashion_mnist
(train_images, train_labels), (test_images, test_labels) = mnist.load_data()

train_images = train_images.reshape(60000, 28, 28, 1)
test_images = test_images.reshape(10000, 28, 28, 1)
train_images, test_images = train_images/255, test_images/255

model = tf.keras.Sequential([
    tf.keras.layers.Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),
    tf.keras.layers.Conv2D(64, (3,3), activation='relu'),
    tf.keras.layers.MaxPooling2D(2,2),
    tf.keras.layers.Dropout(0.25),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dropout(0.5),
    tf.keras.layers.Dense(10, activation='softmax')
])

model.compile(optimizer=tf.keras.optimizers.SGD(lr=0.01, momentum=0.9), loss='sparse_categorical_crossentropy', metrics=['accuracy'])

model.fit(train_images, train_labels, batch_size=32, epochs=10, verbose=1, validation_data=(test_images, test_labels))
```

we get this for our baseline.

Epoch 15/15
469/469 [=====] - 1s 3ms/step - loss: 0.1213 - accuracy: 0.9555 - val_loss: 0.2384 - val_accuracy: 0.9284

How far to go?

Step 2: As per my hint (and I hope your developing instincts), a good place to gain some perspective is always a little research.

It looks like some networks are getting 96%.

It also looks like those are some fancy networks.

Maybe I'll click a few more of those links...

A screenshot of a Google search results page for the query "fashion mnist best accuracy". The search bar at the top shows the query and the Google logo. Below the search bar, there are tabs for "All", "Images", "Videos", "Shopping", "News", and "More". The search results show "About 175,000 results (0.49 seconds)". The main content area displays a table titled "Image Classification on Fashion-MNIST". The table has three columns: "Rank", "Model", and "Accuracy". The top four results are:

Rank	Model	Accuracy
1	Fine-Tuning DARTS	96.91
2	Shake-Shake (SAM)	96.41
3	PreAct-ResNet18 + FMix	
4	Random Erasing	

Below the table, there is a link to "7 more rows". The search results also include a snippet from "https://paperswithcode.com › sota › image-classification-o..." titled "Fashion-MNIST Benchmark (Image Classification) - Papers ...". At the bottom, there is a section titled "People also ask" with two questions: "What is a good Fashion-MNIST accuracy?" and "What is the highest accuracy on MNIST?".

Still researching.

This top link happens to provide a nice summary of the state-of-the-art in MNIST. That "kmnist" is a Japanese character version.

We know Alexnet is a big complex and powerful network, and yet it isn't doing as well as our baseline. This is a clue that bigger is not better.

Those other networks are also pretty big and fancy. So it looks like getting even 94% here is a big ask.

And we know I'm not an unreasonable fellow. So let's set our sights appropriately.

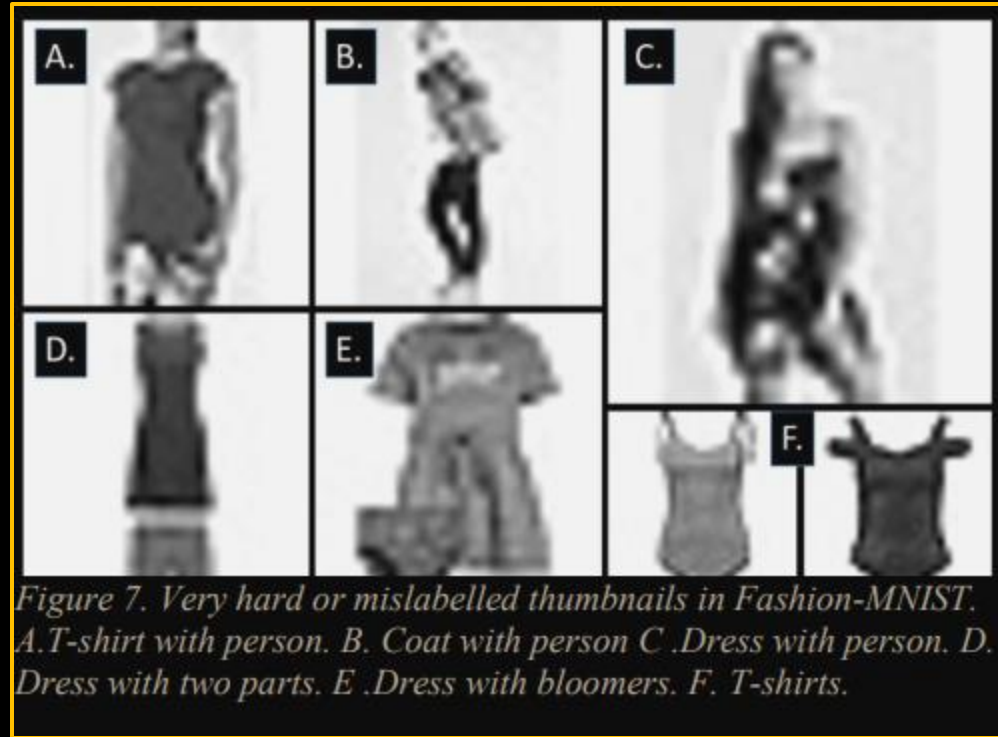
Dataset	Model	Best Epoch	Train Loss	Train Acc.	Val. Loss	Val. Acc.
kmnist	mobilenet_v2	89	0.036	99.988	0.095	98.468
kmnist	resnet101	90	0.020	99.993	0.097	98.147
kmnist	resnet14	73	0.021	99.975	0.070	98.748
kmnist	resnet152	85	0.020	99.988	0.090	98.197
kmnist	resnet18	73	0.020	99.997	0.077	98.628
kmnist	resnet34	83	0.018	99.998	0.081	98.538
kmnist	resnet50	80	0.020	99.982	0.097	98.067
kmnist	resnet9	54	0.008	99.997	0.069	98.528
kmnist	vgg11_bn	62	0.016	99.998	0.078	98.427
kmnist	vgg13_bn	54	0.016	99.980	0.069	98.698
kmnist	vgg16_bn	79	0.015	99.998	0.070	98.698
kmnist	vgg19_bn	99	0.015	99.998	0.078	98.518
fashionmnist	alexnet	97	0.296	89.248	0.308	89.042
fashionmnist	densenet121	92	0.037	99.947	0.266	93.950
fashionmnist	densenet161	93	0.038	99.937	0.264	94.171
fashionmnist	densenet169	97	0.038	99.933	0.258	94.291
fashionmnist	googlenet	96	0.044	99.973	0.240	93.439
fashionmnist	inception_v3	97	0.050	99.861	0.244	94.441
fashionmnist	mobilenet_v2	98	0.040	99.913	0.252	93.860
fashionmnist	resnet101	94	0.019	99.985	0.281	93.740
fashionmnist	resnet14	89	0.021	99.997	0.228	94.040
fashionmnist	resnet152	91	0.020	99.970	0.286	93.770
fashionmnist	resnet18	86	0.020	99.998	0.228	93.970
fashionmnist	resnet34	96	0.018	99.993	0.261	93.910
fashionmnist	resnet50	89	0.019	99.985	0.261	93.810
fashionmnist	resnet9	63	0.009	99.998	0.203	94.071
fashionmnist	vgg11_bn	91	0.017	100.000	0.229	93.600
fashionmnist	vgg13_bn	80	0.017	99.998	0.211	94.111
fashionmnist	vgg16_bn	86	0.018	99.995	0.226	94.030
fashionmnist	vgg19_bn	95	0.018	99.987	0.244	93.960

Why so hard?

As per this analysis of the dataset, there are a lot debatable labels, and some that are apparently wrong.

This is just a trickier problem. Especially at this resolution.

I could not find the number for "superhuman" on this dataset, but it is clearly lower than for the digits.



Get to work!

Step 3: There is no substitute for trial and error with the hyperparameters. Maybe we can hope to spot a trend.

This is a great example of the kind of approach to take.

This made it to over 94%.

Nesterov is just a fancier version of gradient descent (it "looks ahead") that you can select.

Change from Base Code (Acc = 0.8832)	Accuracy
Wider (512)	0.8927
Deeper (512,512,512)	0.8896
CNN (1 layer) - 100 Epochs	0.8790
CNN (2 layers) (32, 64)	0.9223
CNN (2 layers) (64, 64)	0.9217
Drop (0.25,0.5) - 100 Epochs	0.9322
Drop (0.5,0.5) - 80 Epochs	0.9338
Batch Normalization	0.9279
Changing Activation Function ("sigmoid")	0.8678
Conv2D strides = (5,5), lr = 0.02, momentum = 0.9	0.9252
Conv2D strides = (3,3), lr = 0.02, momentum = 0.8	0.9256
lr = 0.02, momentum = 0.9	0.9301
Changing (padding="same") - 40 epochs	0.9364
Changing (padding="same") - 80 epochs	0.9372
nesterov=True - 40 epochs	0.9362
maxPooling(padding="same")	0.9359
nesterov=False	0.9334
nesterov=False - 120 epochs	0.9381
Dropout (0.5,0.6), Epochs = 40	0.9314 (Max: 0.9388)
Dropout (0.6,0.6), Epochs = 40	0.9356
Dropout (0.75,0.75), Epochs = 40	0.9266
Dropout (0.2,0.6), Epochs = 40	0.9336
Dropout (0.2,0.6), Epochs = 100	0.9355
Dropout (0.6,0.6), Epochs = 100	0.9384 (Max: 0.9422)
nesterov=True, Epochs = 100	0.9418

Another fine pursuit of good hyperparameters.

93.7% and a systematic two-steps-forward-one-step-back attack.

#	Model	Train	Test
1	Baseline w/ Dropout (trained for 20 epochs)	0.9427	0.918
2	Changed optimizer to ADAM	0.9619	0.9267
3	Increase first conv layer to 64 filters	0.9612	0.9315
4	Increase second conv layer to 128 filters	0.968	0.9303
5	Increase dense layer to 192	0.9746	0.9335
6	Increase dense layer to 256	0.9814	0.9264
7	Increase dropout after conv layer to 0.4	0.971	0.932
8	Increase dropout after conv layer to 0.5	0.9627	0.9317
9	Add conv layer with 96 filters between existing conv layers	0.9634	0.9318
10	Removed new layer, added third conv layer with 192 filters	0.9667	0.9343
11	Add dense layer with 256 neurons, dropout 0.5	0.9467	0.931
12	Reduce dropout of new layer to 0.25	0.9653	0.9308
13	Remove dropout of new layer	0.9817	0.9256
14	Change new layer to have 320 neurons, reintroduce 0.25 dropout	0.9702	0.9303
15	Revert to model 10, changed third conv layer to have 224 filters	0.9648	0.9348
16	Revert to model 10, changed second conv layer to have 192 filters	0.9626	0.9356
17	Changed first conv layer to have 128 filters	0.9655	0.932
18	Revert to model 16, increase dense layer to have 320 neurons	0.9677	0.9293
19	Revert to model 16, train for 30 epochs	0.9688	0.9338
20	Changed third conv layer to have 256 filters, train for 25 epochs	0.9691	0.9303
21	Changed dense layer to have 320 neurons	0.9733	0.9331
22	Changed dense layer to have 512 neurons	0.9749	0.9357
23	Add BatchNorm after all conv layers before pooling	0.9691	0.9316
24	Add first dense layer with 1024 neurons, dropout 0.5	0.952	0.9342
25	Add conv layer with 64 filters and BatchNorm after first layer	0.948	0.9335
26	Revert to model 16, add BatchNorm after all conv layers before pooling	0.9654	0.9368

And finally:

Newton vs. The Machine

Newton vs the machine: solving the chaotic three-body problem using deep neural networks

Philip G. Breen^{1*}†, Christopher N. Foley² *‡, Tjarda Boekholt³
and Simon Portegies Zwart⁴

¹*School of Mathematics and Maxwell Institute for Mathematical Sciences, University of Edinburgh, Kings Buildings, Edinburgh, EH9 3JZ*

²*MRC Biostatistics Unit, University of Cambridge, Cambridge, CB2 0SR, UK.*

³*Instituto de Telecomunicações, Campus Universitário de Santiago, 3810-193, Aveiro, Portugal*

⁴*Leiden Observatory, Leiden University, PO Box 9513, 2300 RA, Leiden, The Netherlands.*

Accepted XXX. Received YYY; in original form ZZZ

ABSTRACT

Since its formulation by Sir Isaac Newton, the problem of solving the equations of motion for three bodies under their own gravitational force has remained practically unsolved. Currently, the solution for a given initialization can only be found by performing laborious iterative calculations that have unpredictable and potentially infinite computational cost, due to the system’s chaotic nature. We show that an ensemble of solutions obtained using an arbitrarily precise numerical integrator can be used to train a deep artificial neural network (ANN) that, over a bounded time interval, provides accurate solutions at fixed computational cost and up to 100 million times faster than a state-of-the-art solver. Our results provide evidence that, for computationally challenging regions of phase-space, a trained ANN can replace existing numerical solvers, enabling fast and scalable simulations of many-body systems to shed light on outstanding phenomena such as the formation of black-hole binary systems or the origin of the core collapse in dense star clusters.

Key words: stars: kinematics and dynamics, methods: numerical, statistical

1 INTRODUCTION

Newton’s equations of motion describe the evolution of many bodies in space under the influence of their own gravitational force (Newton 1687). The equations have a central role in many classical problems in Physics. For example, the equations explain the dynamical evolution of globular star clusters and galactic nuclei, which are thought to be the production sites of tight black-hole binaries that ultimately merge to produce gravitational waves (Portegies Zwart & McMillan 2000). The fate of these systems depends crucially on the three-body interactions between black-hole binaries and single black-holes (e.g. see Breen & Heggie 2013A,B; Samsing & D’Orazio 2018), often referred to as close encounters. These events typically occur over a fixed time interval and, owing to the tight interactions between the three nearby bodies, the background influence of the other bodies can be ignored, i.e. the trajectories of three bodies can be generally computed in isolation (Portegies Zwart & McMillan 2018).

The focus of the present study is therefore the timely computation of accurate solutions to the three-body problem.

Despite its age and interest from numerous distinguished scientists (de Lagrange 1772; Heggie 1975; Hut & Bahcall 1983; Montgomery 1998; Stone & Leigh 2019), the problem of solving the equations of motion for three-bodies remains impenetrable due to the system’s chaotic nature (Valtonen et al 2016) which typically renders identification of solutions feasible only through laborious numerical integration. Analytic solutions exist for several special cases (de Lagrange 1772) and a solution to the problem for all time has been proposed (Valtonen et al 2016), but this is based on an infinite series expansion and has limited use in practice. Computation of a numerical solution, however, can require holding an exponentially growing number of decimal places in memory and using a time-step that approaches zero (Boekholt et al 2019). Integrators which do not allow for this often fail spectacularly, meaning that a single numerical solution is unreliable whereas the average of an ensemble of numerical solutions appear valid in a statistical sense, a concept referred to as nagh Hoch (Portegies Zwart & Boekholt 2018). To overcome these issues, the Brutus integrator was developed (Boekholt & Portegies Zwart 2015),

* Authors contributed equally

† Contact e-mail: phil.breen@ed.ac.uk

‡ Contact e-mail: christopher.foley@mrc-bsu.cam.ac.uk

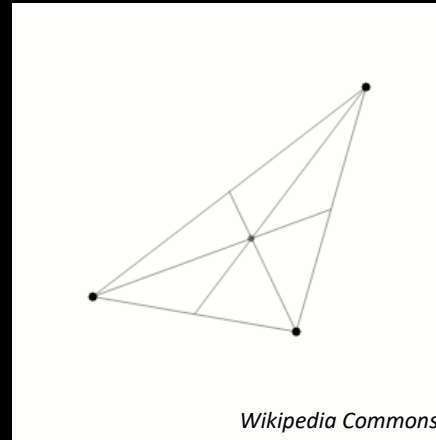
The Three-body Problem

You may have studied the two-body problem. One of its characteristics is that it is wonderfully analyzable for a variety of potentials.

The next step up the ladder of n-body problems is the three-body model. This has been an active area of research since Newton as it not only applies to such important cases as the Sun-Earth-Moon, but it also defies simple analysis.

We know that the general solution can not be represented by any closed solution (although there has been relatively recent work regarding infinite series).

Many of the greatest mathematicians have devoted efforts to the problem and some particular solutions were found by Newton, Jacob Bernoulli, Daniel Bernoulli, Leonhard Euler, Joseph-Louis Lagrange, Pierre-Simon Laplace, and others. Henri Poincaré's efforts were a large part of the foundation of modern chaos theory.



Wikipedia Commons

Numerically Challenging

Generating the training data here exemplifies how difficult it is to do this accurately. The "classical" method that they used (actually quite sophisticated, and a very modern code) uses an advanced numerical method and includes variable precision.

Despite its age and interest from numerous distinguished scientists (de Lagrange 1772; Heggie 1975; Hut & Bahcall 1983; Montgomery 1998; Stone & Leigh 2019), the problem of solving the equations of motion for three-bodies remains impenetrable due to the system's chaotic nature (Valtonen et al 2016) which typically renders identification of solutions feasible only through laborious numerical integration. Analytic solutions exist for several special cases (de Lagrange 1772) and a solution to the problem for all time has been proposed (Valtonen et al 2016), but this is based on an infinite series expansion and has limited use in practice. Computation of a numerical solution, however, can require holding an exponentially growing number of decimal places in memory and using a time-step that approaches zero (Boekholt et al 2019). Integrators which do not allow for this often fail spectacularly, meaning that a single numerical solution is unreliable whereas the average of an ensemble of numerical solutions appear valid in a statistical sense, a concept referred to as nagh Hoch (Portegies Zwart & Boekholt 2018). To overcome these issues, the Brutus integrator was developed (Boekholt & Portegies Zwart 2015), allowing for close-to-zero time-steps and arbitrary precision. Brutus is capable of computing converged solutions to any gravitational N-body problem, however the process is laborious and can be extremely prohibitive in terms of computer time. In general, there does not exist a theoretical framework capable of determining a priori the precision required to deduce that a numerical solution has converged for an arbitrary initialization (Stone & Leigh 2019). This makes the expense of acquiring a converged solution through brute-force integration unpredictable and regularly impractical.

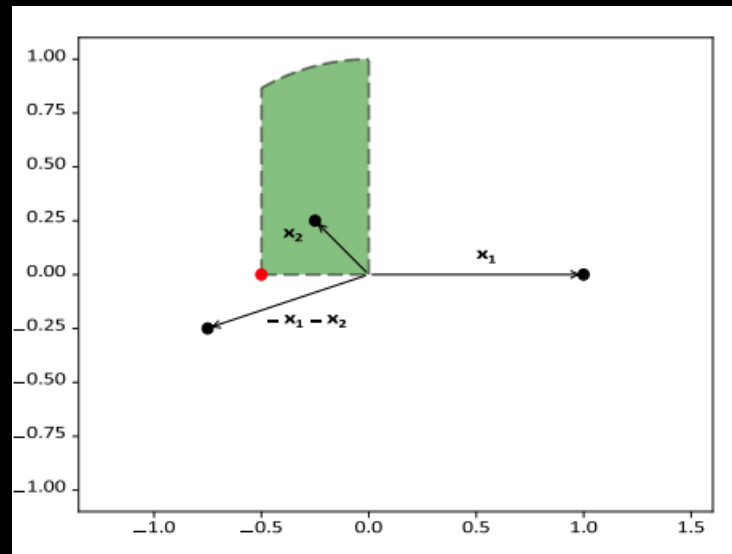
A converged solution was acquired by iteratively reducing two parameters during integration: (i) the tolerance parameter (ϵ), controlling accuracy, that accepts convergence of the Bulirsch-Stoer multi-step integration scheme (Bulirsch & Stoer 1964) and; (ii) the word length (L_w) measured in bits, which controls numerical precision (Boekholt & Portegies Zwart 2015). Our ensemble of initial realizations all converged for values of $\epsilon = 10^{-11}$ and $L_w = 128$ (see Appendix A). Generating these data required over 10 days of computer time. Some initialisations gave rise to very close

Reducing the Parameters

For this restricted version of the problem (3 equal masses with zero initial velocities) we can use a little physics and reduce the number of parameters required to represent the initial configurations.

- Reference frame is center of mass
- X_1 is always at (1,0)
- Specify X_2 's x,y coordinates
- X_3 is given by symmetry ($-X_1 - X_2$)

We only need two parameters (X_2 x,y coordinates) to specify any desired initial condition within the unit circle!



If you recall our "sophisticated" view of neural nets as mappings from input vectors to output vectors, you can imagine we have simplified the task for our network a good deal.

Training Data

Our training data is going to have an input file with each sample consisting of an initial configuration and a timestamp for the desired final configuration.

The corresponding output is going to be the numerically generated final configuration in a reduced form: X_1 and X_2 positions (X_3 follows from symmetry).

train_X.csv

```
0.652344,-0.1648881236291216,0.9141665763358605
0.519531,-0.1648881236291216,0.9141665763358605
0.21875,-0.1648881236291216,0.9141665763358605
...
```

train_Y.csv

```
0.8724009514007633,0.03988576497011263,-0.1041928066877228,0.7964733750758993
0.9205503757602744,0.02465380676629187,-0.127317806825031,0.8407822474813996
0.9862529823431486,0.004226939675911646,-0.1584345882281039,0.9014472881328324
...
```

Our dataset will only contain the <3.9s class of data - or about 39% of the longest, 10s, trajectories mentioned in the paper.

Your Model

The paper describes the network used to generate the results. In particular, **all of the hyperparameters**.

This is everything you need to know to reproduce their results.

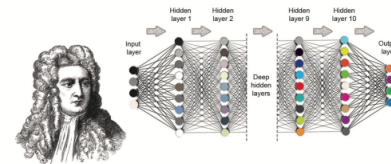


Figure 2. Newton and the machine. Image of sir Isaac Newton alongside a schematic of a 10-layer deep neural network. In each layer (apart from the input layer), a node takes the weighted input from the previous layer's nodes (plus a bias) and then applies an activation function before passing data to the next node. The weights (and bias) are free parameters which are updated during training.

counters, and computation of converged solutions in these situations is costly¹ (Boekholt et al 2019).

We used a feed-forward ANN consisting of 10 hidden layers of 128 interconnected nodes (Fig. 2 and Appendix B). Training was performed using the adaptive moment estimation optimization algorithm ADAM (20) with 10000 passes over the data, in which each epoch was separated into batches of 5000, and setting the rectified linear unit (ReLU) activation function to $\max(0, x)$ (Glorot, Bordes & Bengio, 2011). By entering a time t and the initial location of particle x_2 into the input layer, the ANN returns the locations of the particles x_1 and x_2 at time t , thereby approximating the latent analytical solution to the general three-body problem.

To assess performance of the trained ANN across a range of time intervals, we partitioned the training and validation datasets into three segments: $t \lesssim 3.9$, $t \lesssim 7.8$ and $t \lesssim 10$ (which includes all data). For each scenario, we assessed the loss-function (taken as the mean absolute error MAE) against epoch. Examples are given in Fig. 3. In all scenarios the loss in the validation set closely follows the loss in the training set. We also assessed sensitivity to the choice of activation function, however no appreciable improvement was obtained when using either the exponential rectified (Clevert, Unterthiner & Hochreiter 2011) or leaky rectified (Maas, Hannun & Ng 2013) linear unit functions. In addition, we assessed the performance of other optimization schemes for training the ANN, namely an adaptive gradient algorithm (Duchi, Hazan & Singer 2011) and a stochastic gradient descent method using Nesterov momentum, but these regularly failed to match the performance of the ADAM optimizer.

Results

You should be able to reproduce these same results for the <3.9s data.

You should include an error graph that looks very similar to theirs.

Reproducing some sample trajectories is optional, but highly recommended to make sure you really understand what you are doing. *This is worth a lot of partial credit if you need it!*

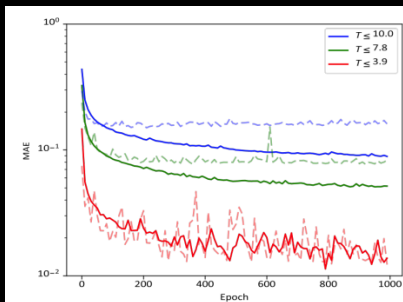


Figure 3. Mean Absolute Error (MAE) vs epoch. The ANN has the same training structure in each time interval. Solids lines are the loss on the training set and dashed are the loss on the validation set. $T \leq 3.9$ corresponds to 1000 labels per simulation, similarly $T \leq 7.8$ to 2000 labels and $T \leq 10.0$ to 2561 labels/time-points (the entire dataset). The results illustrate a typical occurrence in ANN training, there is an initial phase of rapid learning, e.g. ~ 100 epochs, followed by a stage of much slower learning in which relative prediction gains are smaller with each epoch.

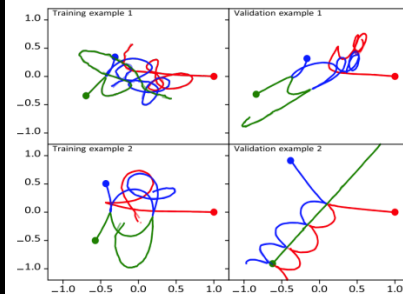


Figure 4. Validation of the trained ANN. Presented are two examples from the training set (left) and two from the validation set (right). All examples were randomly chosen from their datasets. The bullets indicate the initial conditions. The curves represent the orbits of the three bodies (red, blue and green, the latter obtained from symmetry). The solution from the trained network (solid curves) is hardly distinguishable from the converged solutions (dashes, acquired using Brutus (Boekholt & Portegies Zwart 2015)). The two scenarios presented to the right were not included in the training dataset.

Chaotic Motion

Chaotic motions like these are not only interesting and challenging, but they represent a great deal of everyday mechanics, not just double-pendulums. Most fluid dynamics (meteorology, etc.) is firmly in this category.

10 years ago, we might have spent considerable time on this topic. Alas, it has been displaced by all of our very modern techniques. The good news is that these very modern techniques have given us exciting new tools for these classic problems.

Some flavor of the central characteristic of chaotic motion - a sensitivity to initial conditions - is explored here. Some very basic insight into how the trajectories diverge with consistency is seen here.

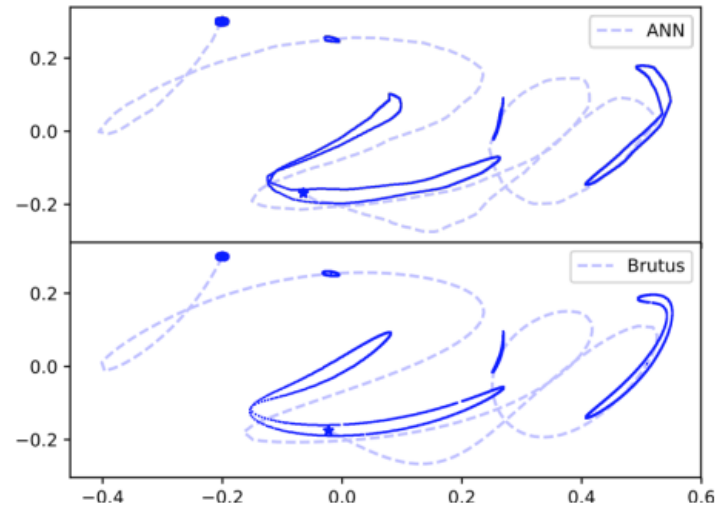


Figure 5. Visualization of the sensitive dependence on initial position. Presented are trajectories from 1000 random initializations in which particle x_2 is initially situated on the circumference of a ring of radius 0.01 centred at $(-0.2, 0.3)$. For clarity, these locations were benchmarked against the trajectory of x_2 initially located at the centre of the ring (hatched line), the star denotes the end of this trajectory after 3.8 time-units. None of these trajectories were part of the training or validation datasets. The locations of the particles at each of five timepoints, $t \in \{0.0, 0.95, 1.9, 2.95, 3.8\}$, are computed using either the trained ANN (top) or Brutus (bottom) and these solutions are denoted by the bold line. The results from both methods closely match one another and illustrate a complex temporal relationship which underpins the growth in deviations between particle trajectories, owing to a change in the initial position of x_2 on the ring.

Conserved Quantities

Even complex systems may have conserved quantities. Ours certainly does, and it makes a great check of correctness.

Our network doesn't actually output velocities directly. The most obvious thing to do would be to derive these using the trajectories.

The paper discusses a network that actually generates velocities. It also discusses an improved algorithm that incorporates this into an improved error term.

There remain interesting areas of improvement to explore. That you could do!

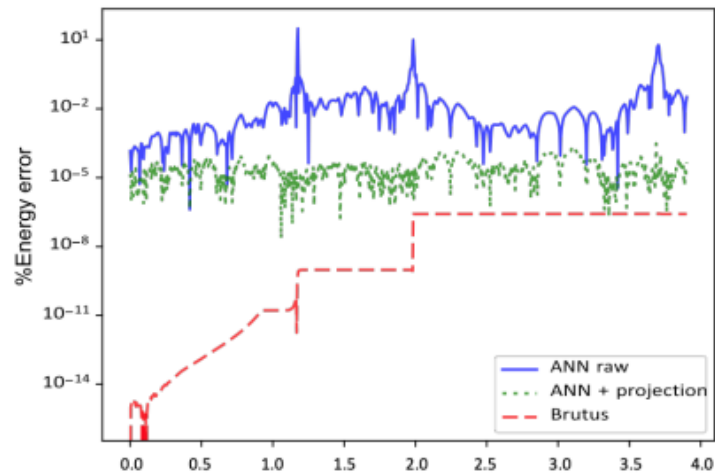


Figure 6. Relative energy error. An example of the relative energy error for a typical simulation. The raw output of the two ANN's typically have errors of around 10^{-2} , after projecting onto a near by energy surface the error reduces down to order 10^{-5} .

Assignment #6: Newton vs. The Machine

We are only concerned with the most accurate trajectories, the case of $t < 3.9s$. Otherwise the dataset is as described in the paper*.

The files are found at `~urbanic/Large_Scale_Computing/Newton_vs_Machine`, along with a copy of the paper.

In this format, the data can be easily read. One way to get it into a TensorFlow compatible numpy array is with

```
import pandas as pd
inputf = pd.read_csv("train_X.csv").values
target = pd.read_csv("train_Y.csv").values
```

but feel free to do something else.

I used the AI module for my own tests (as opposed to the singularity container I recommended for the MNIST work). Either should work, but you can use the AI module with a simple

```
module load AI
```

and then just start your python shell or run your python script and TensorFlow will be there.

* We did eliminate two corrupted trajectories. You won't notice unless you count all the samples.

Assignment #6: Newton vs. The Machine

To reproduce the results shown in Figure 3 of the paper (for $t < 3.9s$) you will need to run 1000 epochs. We found this to take about 100 minutes.

The good news is that you shouldn't be doing much of a hyperparameter search as the paper is handing you all of the effective values. If you nail it, you will only need one run.

I don't expect you to nail it the first time. So, pay attention and if the error isn't looking something like Figure 3, kill your run and try again.

The paper has the (very small) validation split of only 1% of the data. You can split this out of the given data yourself (use Spark!). Or, this might be a good time to learn about the `validation_split` option in `model.fit()`.

Assignment #9: Logistics

Deadline: December 4th, last day of class. Before we start the lecture.

Points: 10 points.

Format: Email me your TensorFlow script as an attachment and the detailed results in the body of the email. This includes at least the final test accuracy and the error graph. You may also want to include comments as your incremental efforts along the way will count for credit.

Machine Learning Wrap Up

John Urbanic

Parallel Computing Scientist
Pittsburgh Supercomputing Center

Distinguished Service Professor
Carnegie Mellon University

AI Based Simulation?

A wise man once (not that long ago) told me "John, I don't need a neural net to rediscover conservation of energy."

Model-Free Prediction of Large Spatiotemporally Chaotic Systems from Data: A Reservoir Computing Approach

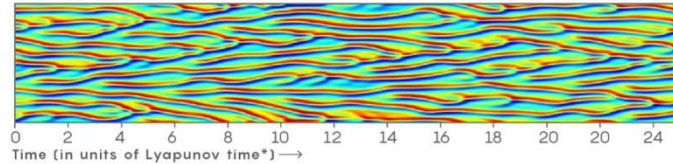
Jaideep Pathak, Brian Hunt, Michelle Girvan, Zhixin Lu, and Edward Ott
Phys. Rev. Lett. 120, 024102 – Published 12 January 2018

Training Computers to Tame Chaos

A machine-learning algorithm has been shown to accurately predict a chaotic system far further into the future than previously possible.

A Chaos Model

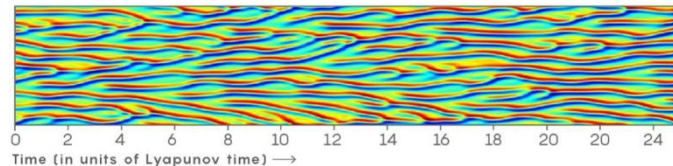
Researchers started with the evolving solution to the Kuramoto-Sivashinsky equation, which models propagating flames:



* Lyapunov time = Length of time before a small difference in the system's initial state begins to diverge exponentially. It typically sets the horizon of predictability, which varies from system to system.

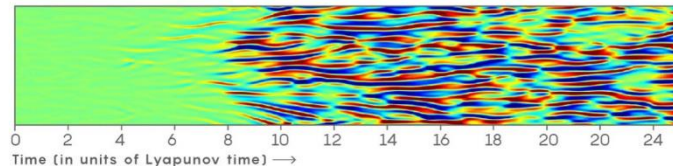
B Machine Learning

After training itself on data from the past evolution of the Kuramoto-Sivashinsky system, the "reservoir computing" algorithm predicts its future evolution:



A - B Do They Match?

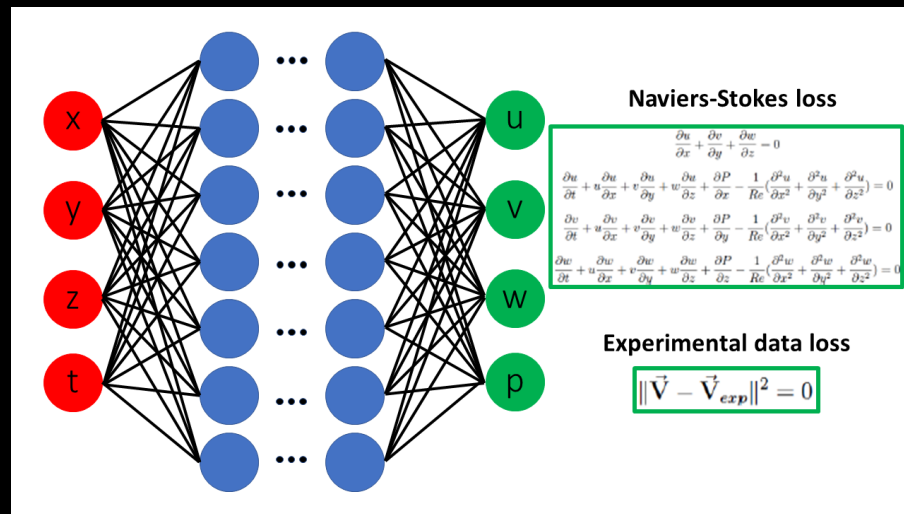
Subtracting B from A shows that the algorithm accurately predicts the model out to an impressive 8 Lyapunov times, before chaos ultimately prevails:



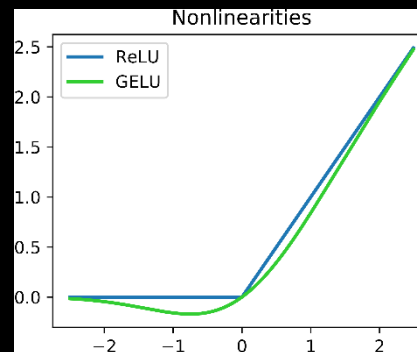
Physics Informed Neural Networks

But maybe we can include our *a priori* knowledge. These types of networks (PINNs) are rapidly gaining interest in the world of physical modeling.

They are also no magic bullet. We now have two competing loss functions, for the data and the physics. And if the gradients in our network now have physical significance, we have to be more rigorous in our treatment of them. No ReLU activation functions, but instead something like the Gaussian Error Linear Unit (GELU).

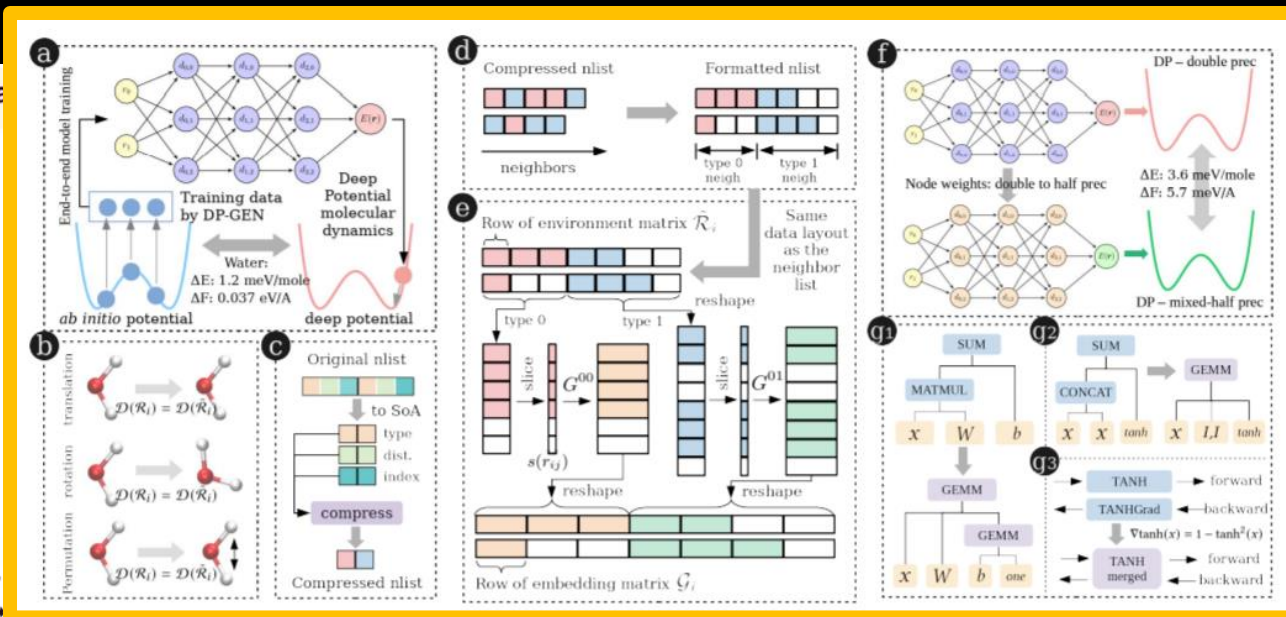
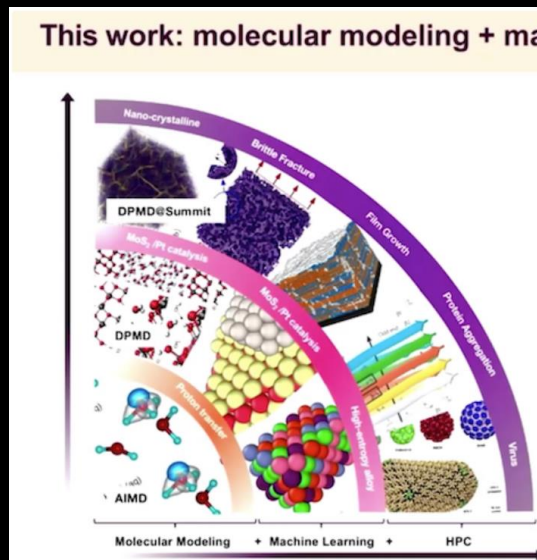


Wikipedia



AI Based Simulation Is Here To Stay

“We report that a machine learning-based simulation protocol (Deep Potential Molecular Dynamics), while retaining *ab initio* accuracy, can simulate more than 1 nanosecond-long trajectory of over 100 million atoms per day, using a highly optimized code (GPU DeePMD-kit) on the Summit supercomputer. Our code can efficiently scale up to the entire Summit supercomputer, attaining 91 PFLOPS in double precision (45.5% of the peak) and 162/275 PFLOPS in mixed-single/half precision.



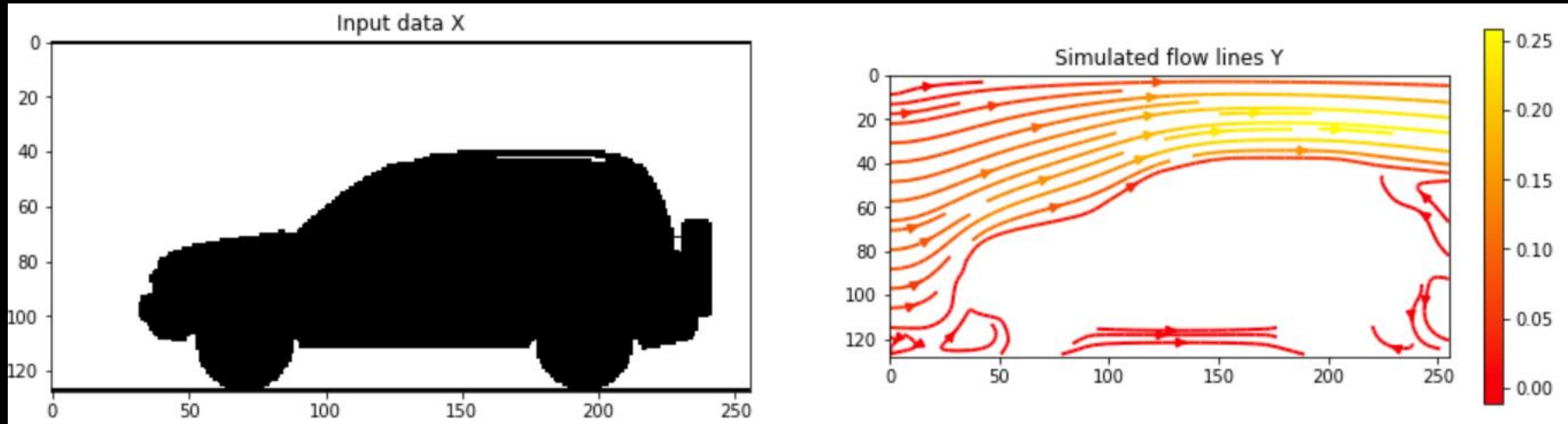
Pushing the Limit of Molecular Dynamics with Ab Initio Accuracy to 100 Million Atoms with Machine Learning

Weile Jia, Han Wang, Mohan Chen, Denghui Lu, Lin Lin, Roberto Car, Weinan E, Linfeng Zhang

2020 ACM Gordon Bell Prize Winner

Try It Yourself

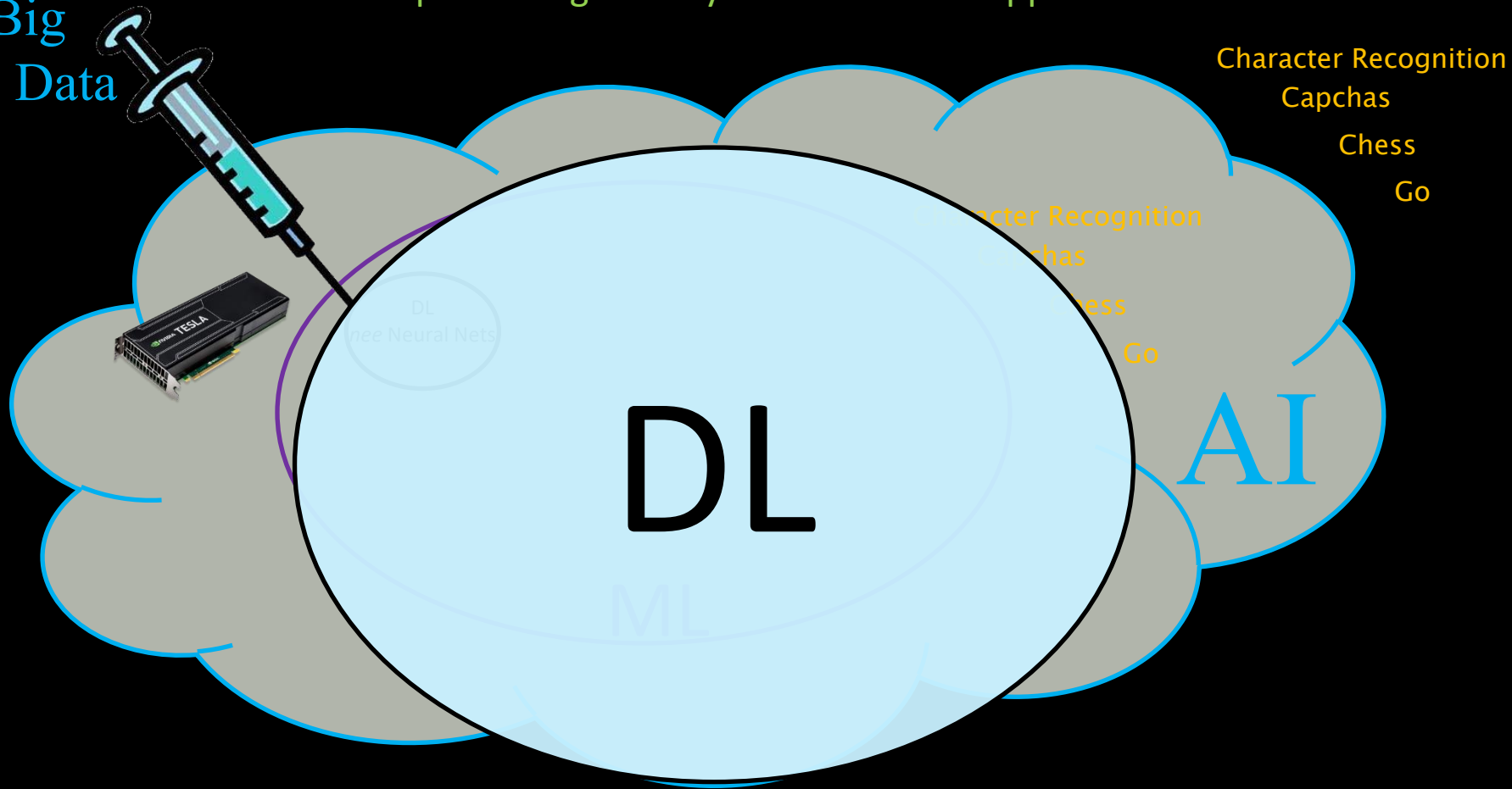
NVIDIA's GPU Bootcamp materials contain a great example of this type of work. The premise is to learn a mapping from boundary conditions to steady state fluid flow. The tutorial works through several different models, starting with a Fully Connected Network, then using a CNN and finally introducing a more advance Residual Network approach. You should be able to jump right in with what we have learned here.



https://github.com/gpuhackathons-org/gpubootcamp/blob/78e9fee3432b60348489682a978fa63f29f7e839/hpc_ai/ai_science_cfd/English/python/jupyter_notebook/CFD/Start_Here.ipynb

Big
Data

Has Deep Learning left any room for other approaches?



“Theoretician’s Nightmare” and Other Perspectives

The above is paraphrasing Yann LeCun, the godfather of Deep Learning.

If it feels like this is an oddly empirical branch of computer science, you are spot on.

Many of these techniques were developed through experimentation, and many of them are not amenable to classical analysis. A theoretician would suggest that non-convex loss functions are at the heart of the matter, and that situation isn’t getting better, as many of the latest techniques have made this much worse.

You may also have noticed that many of the techniques we have used today have very recent provenance. This is true throughout the field. Rarely is the undergraduate researcher so reliant upon groundbreaking papers of a few years ago.

The previously mentioned Christopher Olah has this rather useful summation: *"People sometimes complain: 'Neural networks are so hard to understand! Why can't we use understandable models, like SVMs?' Well, you understand SVMs, and you don't understand visual pattern recognition. If SVMs could solve visual pattern recognition, you would understand it. Therefore, SVMs are not capable of this, nor is any other model you can really understand."*

My own humble observation: Deep Learning looks a lot like late 19th century chemistry. There is a weak theoretical basis, but significant experimental breakthroughs of great utility. The lesson from that era was "expect a lot more perspiration than inspiration."

Most meta.
With some advice for you!

User

Can you explain why this is funny. Think about it step-by-step.



GPT-4

The comic is satirizing the difference in approaches to improving model performance between statistical learning and neural networks.

In statistical learning, the character is shown to be concerned with overfitting and suggests a series of complex and technical solutions, such as minimizing structural risk, reworking the loss function, and using a soft margin.

In contrast, the neural networks character simply suggests adding more layers to the model. This is often seen as a common solution to improving performance in neural networks, but it's also considered a simplistic and brute-force approach.

The humor comes from the contrast between the complexity and specificity of the statistical learning approach and the simplicity and generality of the neural network approach. The "But unironically" comment adds to the humor by implying that, despite being simplistic, the "stack more layers" approach is often effective in practice.

Lazy Scientist's Survey of the Field

Kaggle Challenge

The benchmark driven nature of deep learning research, and its competitive consequences, have found a nexus at Kaggle.com. There you can find over 20,000 datasets:

and competitions:

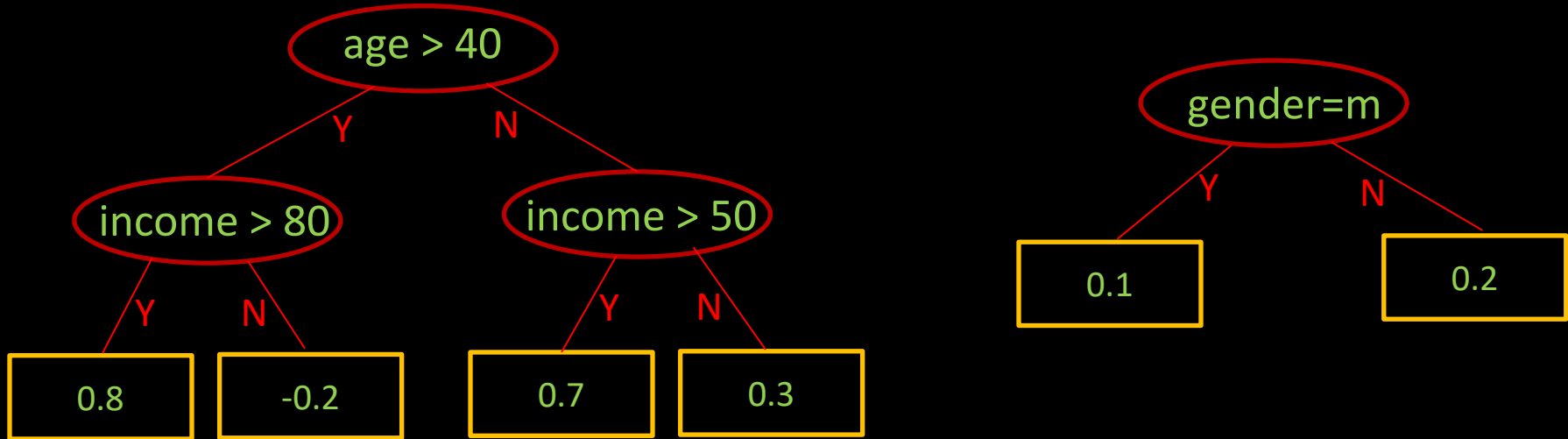
XGBoost?

	Severstal: Steel Defect Detection Can you detect and classify defects in steel? Featured · Kernels Competition · 3 months to go · manufacturing, image data	\$120,000 299 teams
	Digit Recognizer Learn computer vision fundamentals with the famous MNIST data Getting Started · Ongoing · tabular data, image data, multiclass classification, object identification	\$30,000 2,300 teams
	Chess Game Dataset (Lichess) Learn computer vision fundamentals with the famous MNIST data Getting Started · Ongoing · tabular data, image data, multiclass classification, object identification	\$100,000 2,927 teams
	Los Angeles Parking Citations City of Los Angeles 41 minutes · 282 MB · 8.5 · 3 Files (other, C)	\$50,000 2,106 teams
	US Traffic Fatality Records Department of Transportation 4 months · 585 MB · 7.1 · BigQuery	\$30,000 1,281 teams
	Gas Prices in Brazil Matheus Eduardo Freitag 23 days · 3 MB · 15	\$25,000 384 teams
	Chennai Water Management SRK 2 months · 75 KB · 15	Knowledge 3,008 teams
	Electric Motor Temperature Kingan 2 months · 47 MB · 15	
	Vega shrink-wrapper 01 0110 0001 9 months · 408 KB	
	Ghana Health Facilities citizen datascience gha a year · 84 KB	
	Peace Agreements Dataset University of Edinburgh a year · 1 MB	
	Women's Shoe Price Datafiniti a month · 12 MB	
	Venues in Bournemouth Alex a month · 3 KB	
	UW Madison Course Madgrades a year · 90 MB	
	Shared Cars Location DoIT International 7 months · 78 MB	
	Waves Measuring Buoys Data Ifaki a month · 599 KB · 10.0 · 1 File (CSV)	

Trees

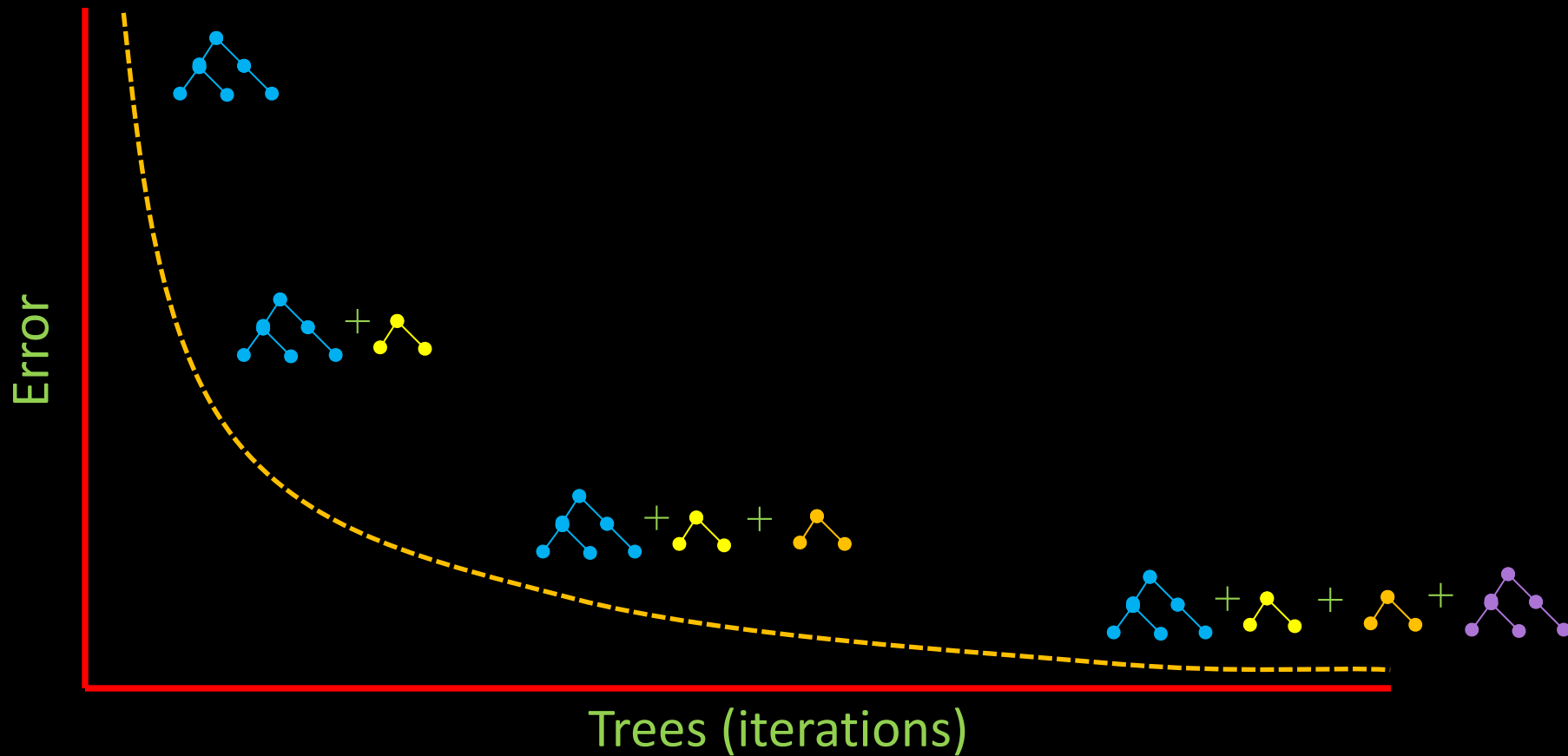
(How much of our earlier learning can we apply here?)

XGBoost is the latest, and most popular, evolution of the Decision Tree approach. Let's say we want to predict is some given person is likely to be a buyer of a certain car model:

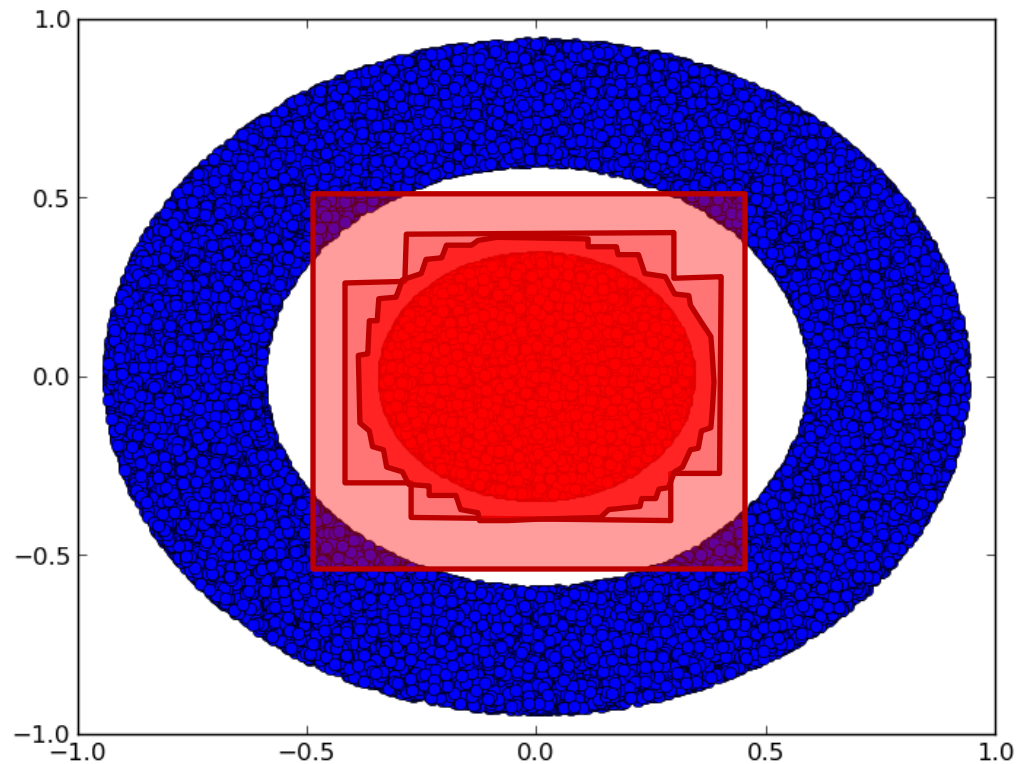


Trees are desirable in that they are non-linear, but still analytically tractable, and can do both regression and classification.

Gradient Boosted Trees

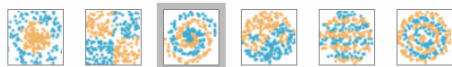


Remember This?

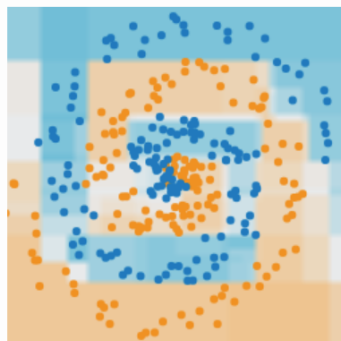


XGBoost

Dataset to classify:



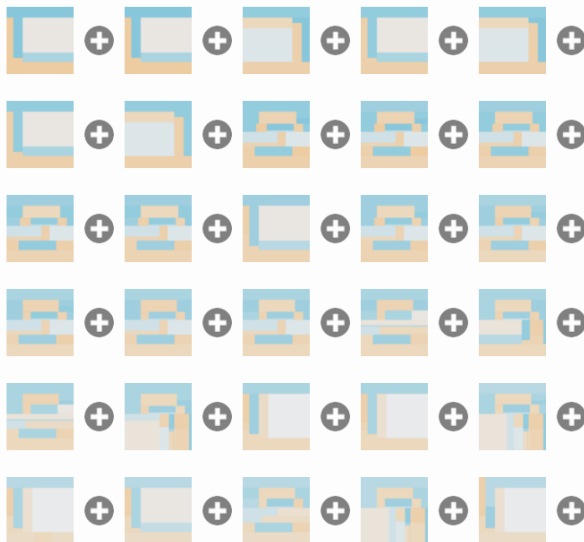
Prediction:



↑
predictions of GB (all 50 trees)

train loss: 0.381 test loss: 0.430

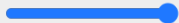
Decision functions of first 30 trees



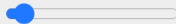
tree depth: 5



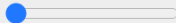
subsample: 100%



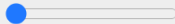
learning rate: 0.1



trees: 50



rotate dataset:



- ☐ rotate trees
- ☒ show gradients on hover
- ☐ use Newton-Raphson update

A very cool interactive application to explore these concepts, and try various **hyperparameters**, was done by Alex Rogozhnikov and can be found at:

http://arogozhnikov.github.io/2016/07/05/gradient_boosting_playground.html

If you want to understand XGBoost in detail, you can find the original paper at:

<https://arxiv.org/pdf/1603.02754.pdf>

An in-depth, but still beginner-friendly, video from StatsQuest can be found at:

<https://www.youtube.com/watch?v=GrJP9FLV3FE>

Great Way To Compare



Epoch
002,086

Learning rate
0.03

Activation
ReLU

Regularization
None

Regularization rate
0

Problem type
Classification

DATA

Which dataset do you want to use?



Ratio of training to test data: 50%

Noise: 0

Batch size: 18

REGENERATE

FEATURES

Which properties do you want to feed in?

X_1
 X_2
 X_1^2
 X_2^2
 X_1X_2
 $\sin(X_1)$
 $\sin(X_2)$

+ - 3 HIDDEN LAYERS

+ -

5 neurons

+ -

5 neurons

+ -

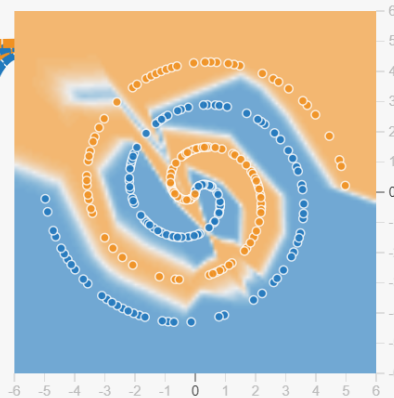
5 neurons

This is the output from one **neuron**. Hover to see it larger.

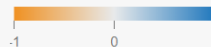
The outputs are mixed with varying **weights**, shown by the thickness of the lines.

OUTPUT

Test loss 0.059
Training loss 0.012



Colors shows data, neuron and weight values.



☐ Show test data

☐ Discretize output

Great Way To Compare



Epoch
000,382

Learning rate

0.03

Activation

Tanh

Regularization

None

Regularization rate

0

Problem type

Classification

DATA

Which dataset do you want to use?



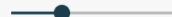
Ratio of training to test data: 50%



Noise: 0



Batch size: 10



REGENERATE

FEATURES

Which properties do you want to feed in?

X_1

X_2

X_1^2

X_2^2

$X_1 X_2$

$\sin(X_1)$

$\sin(X_2)$

+ - 2 HIDDEN LAYERS

+ -

4 neurons

+ -

2 neurons

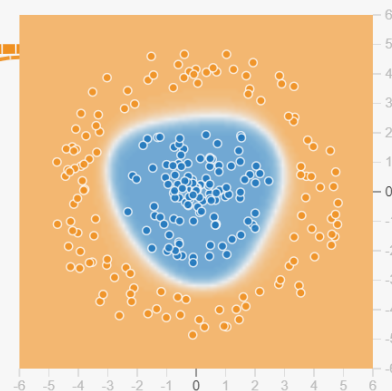
The outputs are mixed with varying **weights**, shown by the thickness of the lines.

This is the output from one **neuron**. Hover to see it larger.

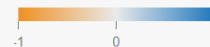
OUTPUT

Test loss 0.001

Training loss 0.001



Colors shows data, neuron and weight values.



☐ Show test data

☐ Discretize output

The Classic Conundrum



Epoch
000,233

Learning rate

0.03

Activation

Tanh

Regularization

None

Regularization rate

0

Problem type

Classification

DATA

Which dataset do you want to use?



Ratio of training to test data: 50%



Noise: 0



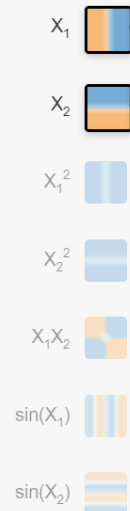
Batch size: 10



REGENERATE

FEATURES

Which properties do you want to feed in?



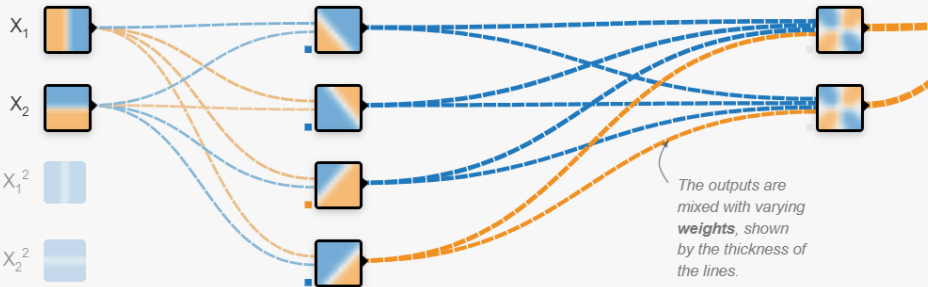
+ - 2 HIDDEN LAYERS



4 neurons



2 neurons



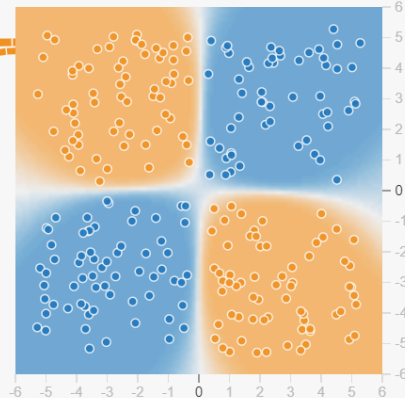
This is the output from one **neuron**. Hover to see it larger.

The outputs are mixed with varying **weights**, shown by the thickness of the lines.

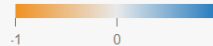
OUTPUT

Test loss 0.006

Training loss 0.003



Colors shows data, neuron and weight values.



☐ Show test data

☐ Discretize output

XGBoost in Particular

There are various implementations of gradient boosted trees. XGBoost combines several important innovations:

- Parallelizes well both across cores and nodes
- Clever cache optimization
- Works well with missing data

The end result is an efficient algorithm that works well enough with non-optimal hyperparameters the beginners can often make quick progress.

The scikit-learn version is probably the most popular, but there is a Spark version (https://xgboost.readthedocs.io/en/latest/jvm/xgboost4j_spark_tutorial.html), and if you want a deeper dive, NVIDIA has this pretty nice taxi fare regression model that uses GPUs with Spark and does a hyperparameter search. Note that I have not tried these myself:

<https://developer.nvidia.com/blog/accelerating-spark-3-0-and-xgboost-end-to-end-training-and-hyperparameter-tuning/>

TensorFlow has a boosted tree API along with a nice walkthrough example in the docs:

https://www.tensorflow.org/tutorials/estimator/boosted_trees

However, note that this is not the XGBoost version (yet).

Other Toolboxes

You have a plethora of alternatives available as well. You are now in a position to appreciate some comparisons.

Package	Applications	Language	Strengths
TensorFlow	Neural Nets	Python, C++	Very popular.
PyTorch	Neural Nets	Python (Lua)	Also very popular. Used to be very different with it's dynamic graphs and eager execution, but lacked simple layers. Now fairly similar in approach.
Spark MLLIB	Classification, Regression, Clustering, etc.	Python, Scala, Java, R	Very scalable. Widely used in serious applications. Lots of plugins to DL frameworks: TensorFrames, TF on Spark, CaffeOnSpark, Keras Elephas.
Scikit-Learn	Classification, Regression, Clustering	Python	Integrates well with TF to create powerful workflows.
Keras	Neural Nets	Python (on top of TF, Theano)	Now completely absorbed into TF.
Jax	Neural Nets	Python	Latest DeepMind (part of Google) framework. Missing pieces, but getting there. Similar to TF & PT.

Draw your number here

2



Downsampled drawing: 2

First guess: 2

Second guess: 0

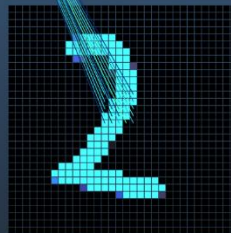
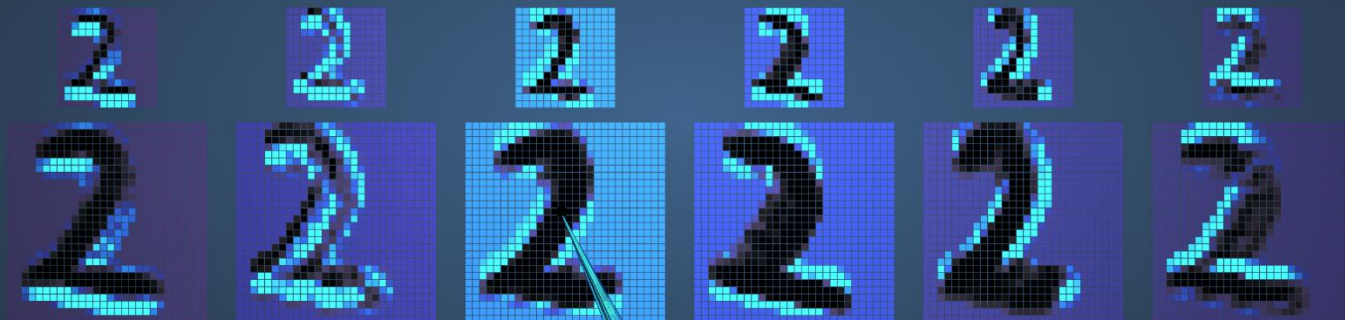
Layer visibility

Input layer	Show
Convolution layer 1	Show
Downsampling layer 1	Show
Convolution layer 2	Show
Downsampling layer 2	Show
Fully-connected layer 1	Show
Fully-connected layer 2	Show
Output layer	Show

A note about hardware.

0123456789

0123456789

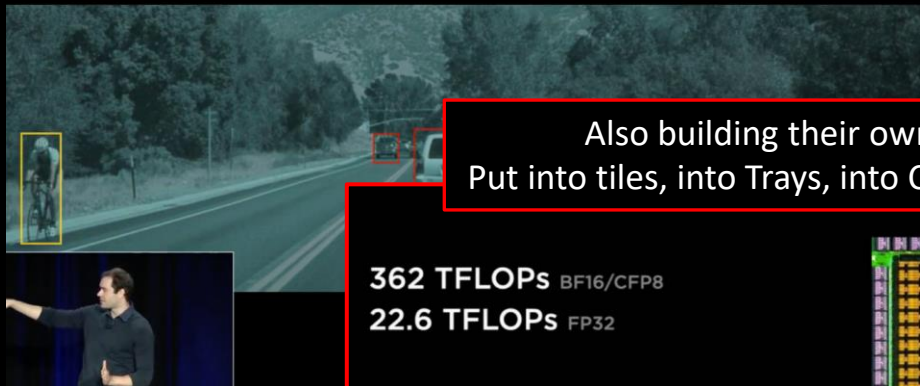


Inference Is Fast

Perceptual Labs



iPhone Demo



Also building their own training chips.
Put into tiles, into Trays, into Cabinets to create Dojo.

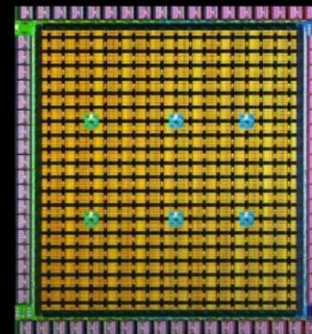
362 TFLOPs BF16/CFP8

22.6 TFLOPs FP32

10TBps/dir. On-Chip Bandwidth

4TBps/edge. Off-Chip Bandwidth

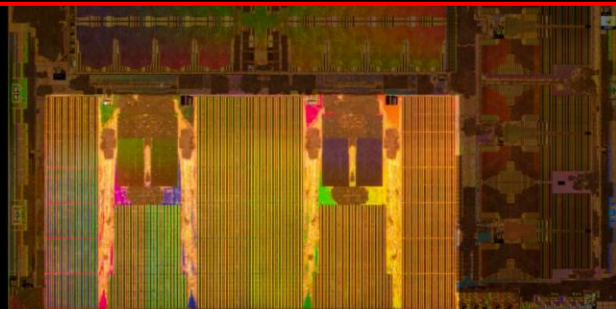
400W TDP



645mm²
7nm Technology

50 Billion
Transistors

11+ Miles
Of Wires



32MB SRAM

96x96 Mul/Add array

ReLU hardware

Pooling hardware

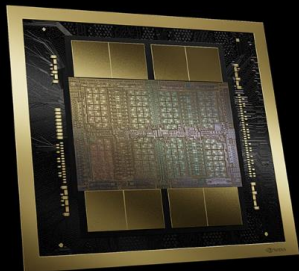
36 TOPS @ 2 GHz

2 per chip, 72 TOPS total

Everyone Doing Specialized Hardware

NVIDIA

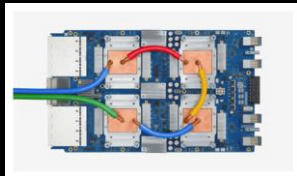
Blackwell



Highlighting 4 bit precision!

Google

TPU



Cloud TPU v6
1.8 exaflops (int 8)
purported...
Due 2024Q4

Intel

Loihi 2



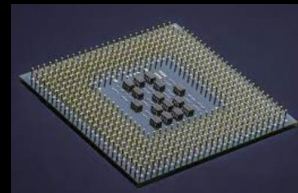
1,048,576 artificial neurons,
and 120 million synapses

Kapoho Point a 4-inch form
factor board featuring eight
Loihi 2 chips

Also new AVX512_VNNI
(Vector Neural Network)
instructions like an FMA
instruction for 8-bit
multiplies with 32-bit
accumulates on new
processors.

Amazon

Inferentia2
Trainium



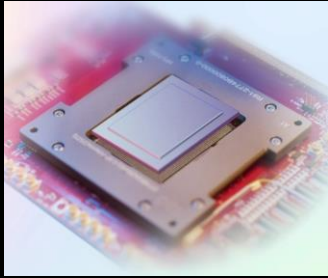
Inferentia for inference and
Trainium for training. Use
standard TensorFlow and
Torch in their EC2 Cloud.

This is where Alexa runs.

Everyone Doing Specialized Hardware

Meta

MTIA 2



708 TFLOPS/s (INT8)

Microsoft

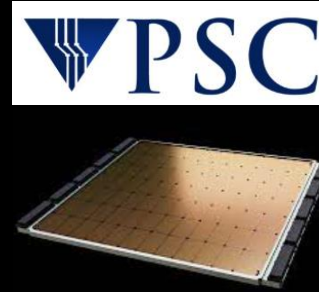
Athena



"Launched" in late 2023, but info still sparse....

Cerebras

CS-2



At PSC!

- 850,000 Sparse Linear Algebra Compute Cores
- 2.6 trillion transistors
- 20 PB/s aggregate memory bandwidth
- 220 Pb/s interconnect bandwidth

Neuromorphic

IBM, ...



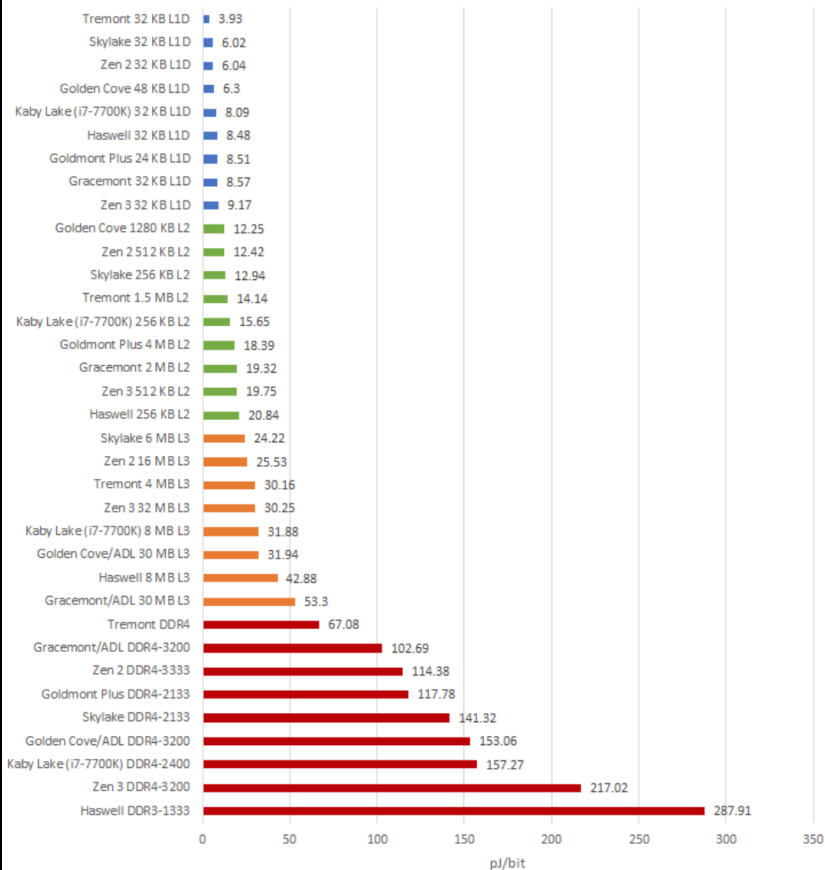
Brain only uses 20W.

Analog, pruning, spiking, lots of new directions.

We are also continuously learning how little we know about how biological mechanisms work.

More Inspiration

Data Transfer Energy Cost



- Laughlin was the first to provide explicit quantities for the energetic cost of processing sensory information. Their findings in blowflies revealed that for visual sensory data, the cost of transmitting one bit of information is around 50 fJ (5×10^{-14} Joules), or equivalently 104 ATP molecules.
- The units on this graph are pJ, 1000X larger. Thus, neural processing efficiency is still far from Landauer's limit of $kT \ln(2)$ J, but still considerably more efficient than a modern computer's near memory. For far (MPI network, or further) accesses it is a huge difference.

Applied Inspiration

Scalable MatMul-free Language Modeling

Rui-Jie Zhu¹, Yu Zhang², Ethan Sifferman¹, Tyler Sheaves³, Yiqiao Wang⁴,
Dustin Richmond¹, Peng Zhou^{1,4}, Jason K. Eshraghian^{1*}

¹University of California, Santa Cruz ²Soochow University

³University of California, Davis ⁴LuxiTech

Abstract

Matrix multiplication (MatMul) typically dominates the overall computational cost of large language models (LLMs). This cost only grows as LLMs scale to larger embedding dimensions and context lengths. In this work, we show that MatMul operations can be completely eliminated from LLMs while maintaining strong performance at billion-parameter scales. Our experiments show that our proposed MatMul-free models achieve performance on-par with state-of-the-art Transformers that require far more memory during inference at a scale up to at least 2.7B parameters. We investigate the scaling laws and find that the performance gap between our MatMul-free models and full precision Transformers narrows as the model size increases. We also provide a GPU-efficient implementation of this model which reduces memory usage by up to 61% over an unoptimized baseline during training. By utilizing an optimized kernel during inference, our model's memory consumption can be reduced by more than 10 $\times$ compared to unoptimized models. To properly quantify the efficiency of our architecture, we build a custom hardware solution on an FPGA which exploits lightweight operations beyond what GPUs are capable of. We processed billion-parameter scale models at 13W beyond human readable throughput, moving LLMs closer to brain-like efficiency. This work not only shows how far LLMs can be stripped back while still performing effectively, but also points at the types of operations future accelerators should be optimized for in processing the next generation of lightweight LLMs. Our code implementation is available at <https://github.com/ridgerchu/matmulfreellm>.

1 Introduction

Matrix Multiplication (MatMul) is the dominant operation in most neural networks, where dense layers involve vector-matrix multiplication (VMM), convolutions can be implemented as block-sparse VMMs with shared weights, and self-attention relies on matrix-matrix multiplication (MMM). The prevalence of MatMul is primarily due to Graphics Processing Units (GPUs) being optimized for MatMul operations. By leveraging Compute Unified Device Architecture (CUDA) and highly optimized linear algebra libraries such as cuBLAS, the MatMul operation can be efficiently parallelized and accelerated. This optimization was a key factor in the victory of AlexNet in the ILSVRC2012 competition and a historic marker for the rise of deep learning [1]. AlexNet notably utilized GPUs to boost training speed beyond CPU capabilities, and as such, deep learning won the 'hardware lottery' [2]. It also helped that both training and inference rely on MatMul.

Despite its prevalence in deep learning, MatMul operations account for the dominant portion of computational expense, often consuming the majority of the execution time and memory access during

The secret sauce is ternary logic, with values of +1, 0 and -1.

The matrix multiplies degenerate into some simpler operations.

This is the may be the finish line of the race to the bottom of precision.

Binary doesn't seem to capture the behaviors we hope for (although it is early days), but ternary does.

This is a bit awkward for our currently binary electronics.

*Corresponding author, email to: pzhou10@ucsc.edu, jsn@ucsc.edu

Read, read, read!

One of my major goals is to leave you with the ability to understand many of the latest publications in *applied, scientific* AI.

Of course, a month is not enough for you to become an expert, but you might be surprised how much of the literature you can understand. You should be well-positioned to fill in the gaps.

If you want to test your knowledge, this "state of the field" lecture by the foremost pioneers of deep learning is an excellent summation of the current leading edge. It is targeted at practitioners of the art, so don't feel intimidated by any unknown references. But if you do get the gist of it, congratulations, you are holding your own with current researchers.

<https://dl.acm.org/doi/pdf/10.1145/3448250>

turing lecture

DOI:10.1145/3448250

How can neural networks learn the rich internal representations required for difficult tasks such as recognizing objects or understanding language?

BY YOSHUA BENGIO, YANN LECUN, AND GEOFFREY HINTON

Deep Learning for AI

TURING LECTURE

Yoshua Bengio, Yann LeCun, and Geoffrey Hinton are recipients of the 2018 ACM A.M. Turing Award for breakthroughs that have made deep neural networks a critical component of computing.

RESEARCH ON ARTIFICIAL neural networks was motivated by the observation that human intelligence emerges from highly parallel networks of relatively simple, non-linear neurons that learn by adjusting the strengths of their connections. This observation leads to a central computational question: How is it possible for networks of this general kind to learn the complicated internal representations that are required for difficult tasks such as recognizing

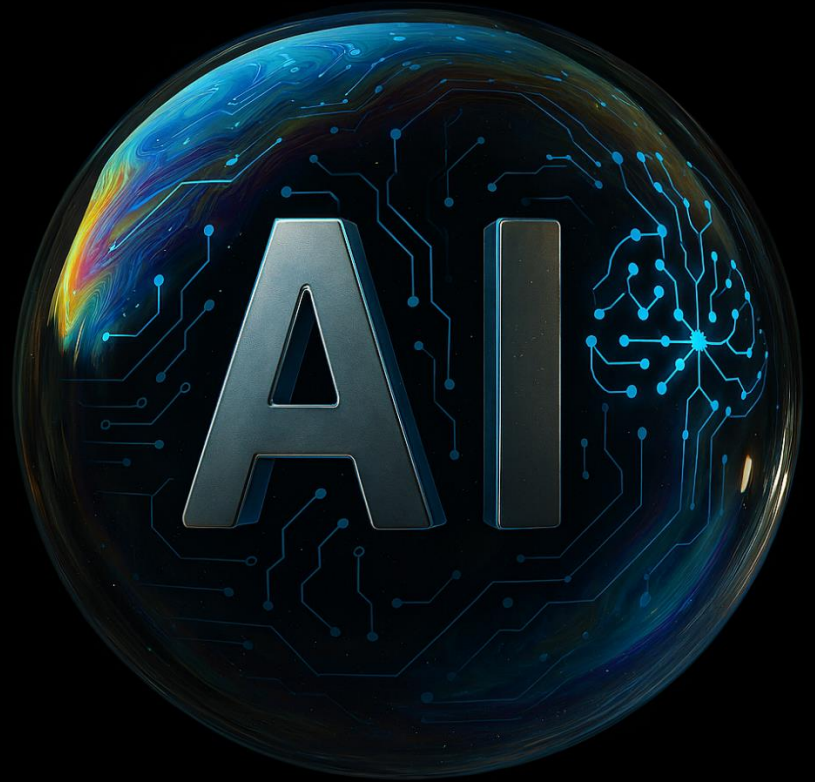
objects or understanding language? Deep learning seeks to answer this question by using many layers of activity vectors as representations and learning the connection strengths that give rise to these vectors by following the stochastic gradient of an objective function that measures how well the network is performing. It is very surprising that such a conceptually simple approach has proved to be so effective when applied to large training sets using huge amounts of computation and it appears that a key ingredient is depth: shallow networks simply do not work as well.

We reviewed the basic concepts and some of the breakthrough achievements of deep learning several years ago.¹⁴ Here we briefly describe the origins of deep learning, describe a few of the more recent advances, and discuss some of the future challenges. These challenges include learning with little or no external supervision, coping with test examples that come from a different distribution than the training examples, and using the deep learning approach for tasks that humans solve by using a deliberate sequence of steps which we attend to consciously—tasks that Kahneman¹⁵ calls *system 2* tasks as opposed to *system 1* tasks like object recognition or immediate natural language understanding, which generally feel effortless.

From Hand-Coded Symbolic Expressions to Learned Distributed Representations

There are two quite different paradigms for AI. Put simply, the logic-inspired paradigm views sequential reasoning as the essence of intelligence and aims to implement reasoning in computers using hand-designed rules of inference that operate on hand-designed symbolic expressions that formalize knowledge. The brain-inspired paradigm views learning representations from data as the essence of intelligence and aims to implement learning by hand-designing or evolving rules for modifying the connec-

Big Question:
Is it a bubble?



Wrap Up

John Urbanic

Parallel Computing Scientist
Pittsburgh Supercomputing Center

Distinguished Service Professor
Carnegie Mellon University

Assignment 6 Review:

Newton vs. The Machine



Just the basics.

```
import pandas as pd
import tensorflow as tf
import numpy as np
from tensorflow.keras import layers

inputf = pd.read_csv("train_X.csv").values
target = pd.read_csv("train_Y.csv").values
```

```
newton_model = tf.keras.Sequential([
    layers.Dense(128, activation='relu'),
    layers.Dense(128, activation='relu'),
    layers.Dense(128, activation='relu'),
    layers.Dense(128, activation='relu'),
    layers.Dense(128, activation='relu'),
    layers.Dense(128, activation='relu'),
    layers.Dense(128, activation='relu'),
    layers.Dense(128, activation='relu'),
    layers.Dense(128, activation='relu'),
    layers.Dense(128, activation='relu'),
    layers.Dense(4)
])
```

```
newton_model.compile(loss = tf.losses.MeanAbsoluteError(), optimizer = tf.optimizers.Adam(learning_rate=0.001), metrics=['accuracy'])
```

```
history = newton_model.fit(inputf, target, epochs=1000, batch_size=5000, validation_split=0.01)
```

```
...
...
```

```
Epoch 999/1000
```

```
8755270/8755270 [=====] - 5s 1us/sample - loss: 0.0088 - accuracy: 0.9793 - val_loss: 0.0114 - val_accuracy: 0.9695
```

```
Epoch 1000/1000
```

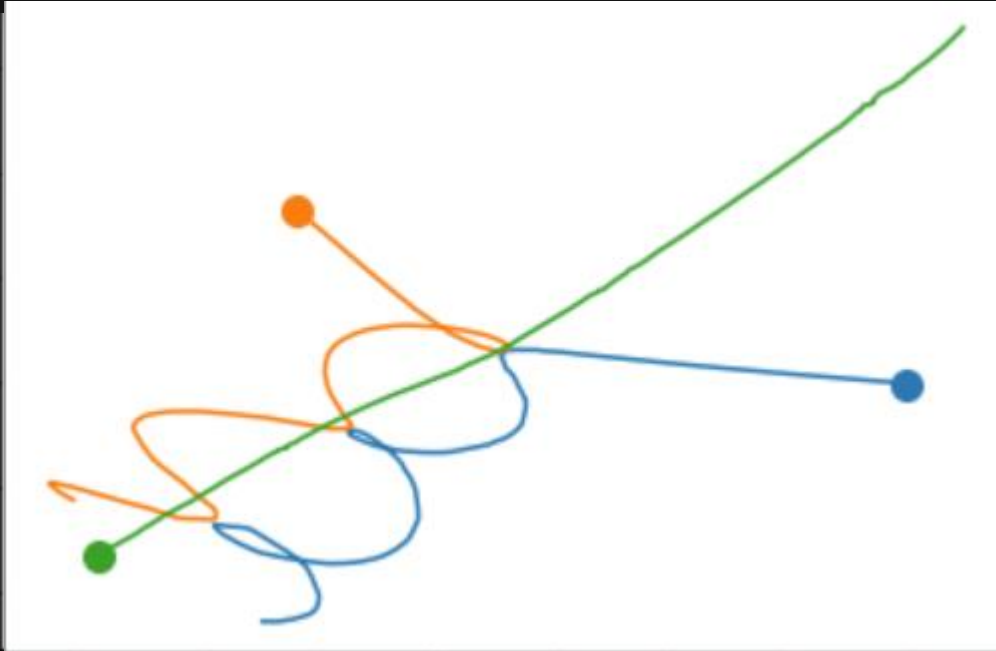
```
8755270/8755270 [=====] - 5s 1us/sample - loss: 0.0087 - accuracy: 0.9796 - val_loss: 0.0135 - val_accuracy: 0.9674
```

validation_split

The validation data is selected from the last samples in the x and y data provided, before shuffling.

Does it work?

The MAE vs. Epoch curve is very similar to the paper (remember there is some randomness and "versioning" involved), but the visual test is even more convincing.



Training Trajectory vs. Generated Trajectory

A great early submission.

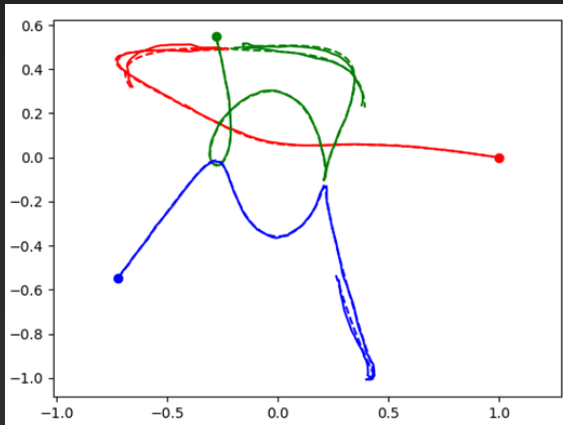
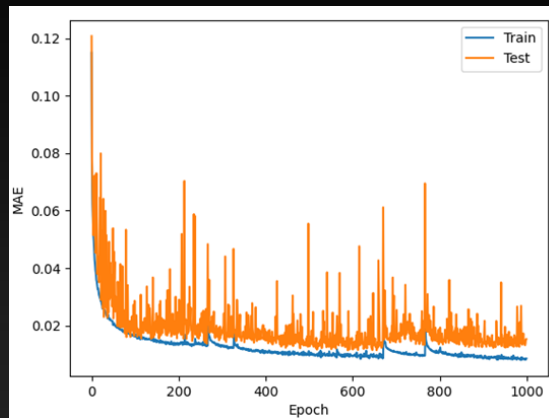
```
import pandas as pd
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt
```

```
inputf = pd.read_csv('train_X.csv').values
target = pd.read_csv('train_Y.csv').values
```

```
model = tf.keras.Sequential([
    tf.keras.layers.Dense(128, activation='relu', input_shape=(3,)),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dense(4)
])
model.compile(optimizer='adam', loss='mean_absolute_error')
history = model.fit(inputf, target, batch_size=5000, epochs=1000, verbose=1, validation_split=0.01)
```

```
# Plot Mean Absolute Error over time (epochs)
plt.figure(0)
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.xlabel('Epoch')
plt.ylabel('MAE')
plt.legend(['Train', 'Test'], loc='upper right')
plt.savefig('mae.png')
```

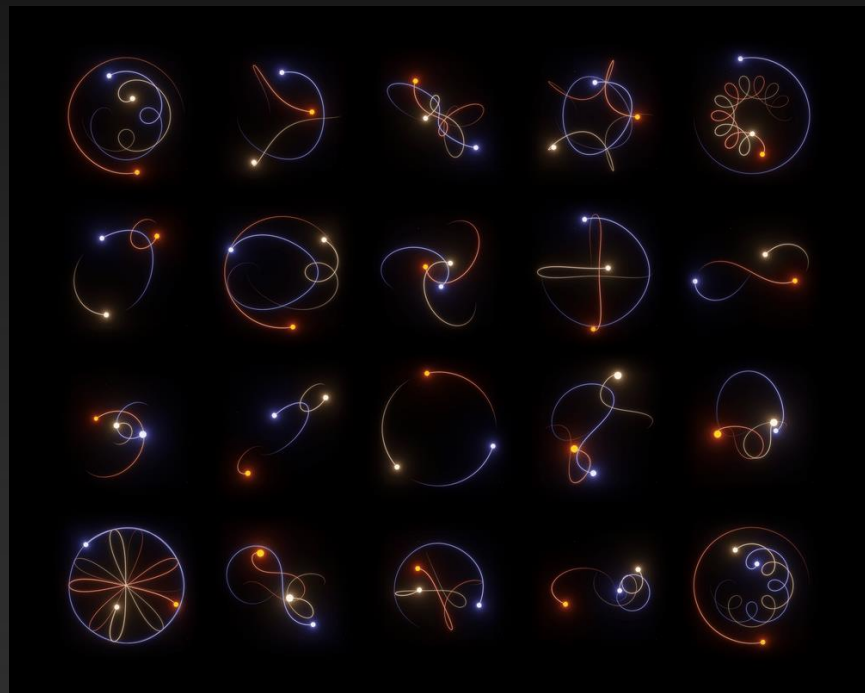
```
# Plot random trajectory in the validation set (solid: prediction, dashed: actual)
# "The validation data is selected from the last samples in the x and y data provided, before shuffling."
plt.figure(1)
idx = np.random.randint(int(inputf.shape[0]*0.99)+1, inputf.shape[0])
data = np.hstack((inputf,target))
data = data[(data[:,1] == data[idx,1]) & (data[:,2] == data[idx,2])]
data = data[data[:,0].argsort()]
traj_in = data[:,0:3]
traj_out = data[:,3:7]
pred_out = model.predict(traj_in)
plt.plot(traj_out[0,0], traj_out[0,1], 'ro')
plt.plot(traj_out[:,0], traj_out[:,1], 'r--')
plt.plot(pred_out[:,0], pred_out[:,1], 'r-')
plt.plot(traj_out[0,2], traj_out[0,3], 'go')
plt.plot(traj_out[:,2], traj_out[:,3], 'g--')
plt.plot(pred_out[:,2], pred_out[:,3], 'g-')
plt.plot(-traj_out[0,0]-traj_out[0,2], -traj_out[0,1]-traj_out[0,3], 'bo')
plt.plot(-traj_out[:,0]-traj_out[:,2], -traj_out[:,1]-traj_out[:,3], 'b--')
plt.plot(-pred_out[:,0]-pred_out[:,2], -pred_out[:,1]-pred_out[:,3], 'b-')
plt.axis('equal')
plt.savefig('traj.png')
```



Useful and deep.

We now have an application that can solve general solutions in milliseconds, instead of hours. This is a breakthrough in certain areas of astrophysics.

Most of these surprising solutions were only discovered recently, and via computer simulation.



Congratulations!

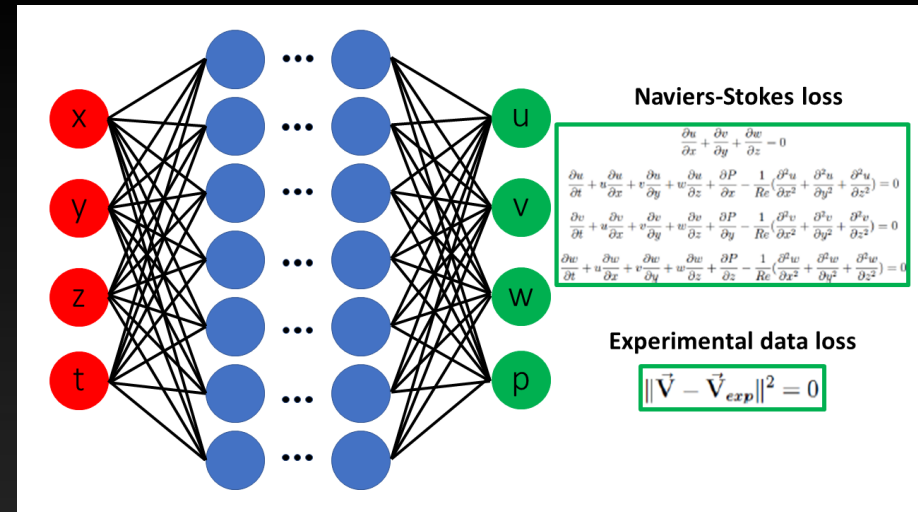
You have now reproduced a non-trivial result that would have been considered science-fiction not that long ago. I hope you can also evaluate the limitations.

If we had another week, we could have taken this problem right up to the forefront of physics-based deep learning, something that the Appendix to this paper was just suggesting...

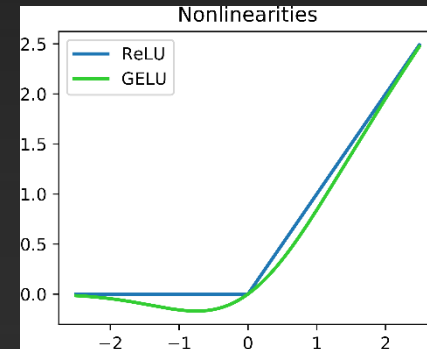
Physics Informed Neural Networks

But maybe we can include our *a priori* knowledge. These types of networks (PINNs) are rapidly gaining interest in the world of physical modeling.

They are also no magic bullet. We now have two competing loss functions, for the data and the physics. And if the gradients in our network now have physical significance, we have to be more rigorous in our treatment of them. No ReLU activation functions, but instead something like the Gaussian Error Linear Unit (GELU).



Wikipedia



Final Exam

- It will be the same format as the mid-term: written section followed by programming.
- Same total time: 80 minutes. The schedule gave us 3 hours, but that seemed inhumane to me.
- The time and location should be

Tuesday, December 9, 1:00pm - 2:20pm WEH 5409

- Again, show up on time with your laptop and a pencil.
- Material will be drawn from entire semester!

And now our
journey is ended.



Grades

I will do my best to get this last assignment graded quickly. And then the final exams.

You should all have final grades posted well before the deadline.

Don't get ahead of me on the final curve. I don't know what the adjustment will be yet. No one should be unpleasantly surprised.

I am happy to discuss grades, or anything, with any of you and will maintain availability until the semester is over.

Course Evaluations

Course Evaluations are due soon.

This is a pretty new course, with no real peers. Your feedback is critical to help me decide if we continue in this direction.

Please take a few moments and let us know what you thought.

Farewell

As I said, this was a very ambitious syllabus. I think we did very well, and I congratulate you for making that possible.

It was a pleasure and I very much enjoyed it.

I hope you all find this a valuable asset in your future endeavors, and most importantly, I hope that this has enhanced your enjoyment and appreciation of modern scientific computing at scale.

I would be delighted to hear from any of you in the future and to help in any way I can.*

**That includes references and recommendations.*

GPUs?
Up to you.



Huang's Law

An observation/claim made by Jensen Huang, CEO of Nvidia, at its 2018 GPU Technology Conference.

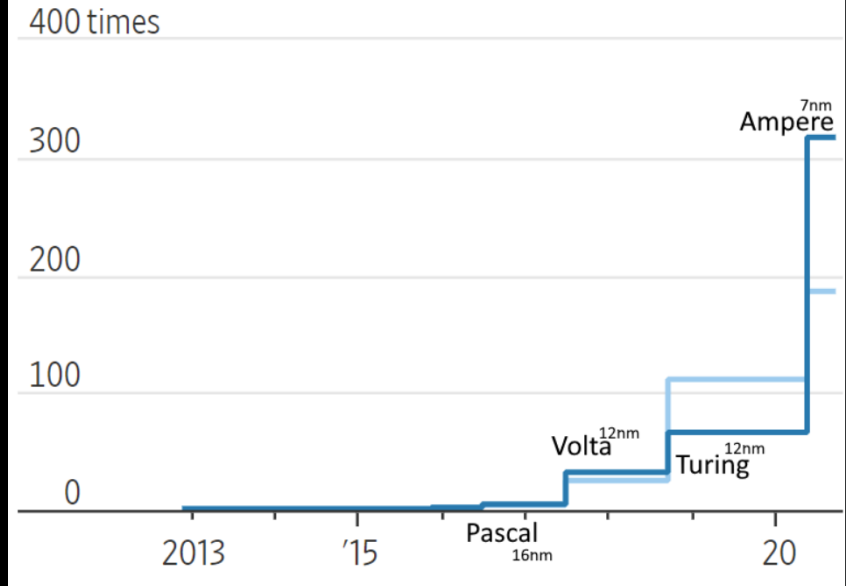
He observed that Nvidia's GPUs were "25 times faster than five years ago" whereas Moore's law would have expected only a ten-fold increase.

In 2006 Nvidia's GPU had a 4x performance advantage over other CPUs. In 2018 the Nvidia GPU was 20 times faster than a comparable CPU node: the GPUs were 1.7x faster each year. Moore's law would predict a doubling every two years, however Nvidia's GPU performance was more than tripled every two years fulfilling Huang's law.

It is a little premature, and there are confounding factors at play, so most people haven't yet elevated this to the status of Moore's Law.

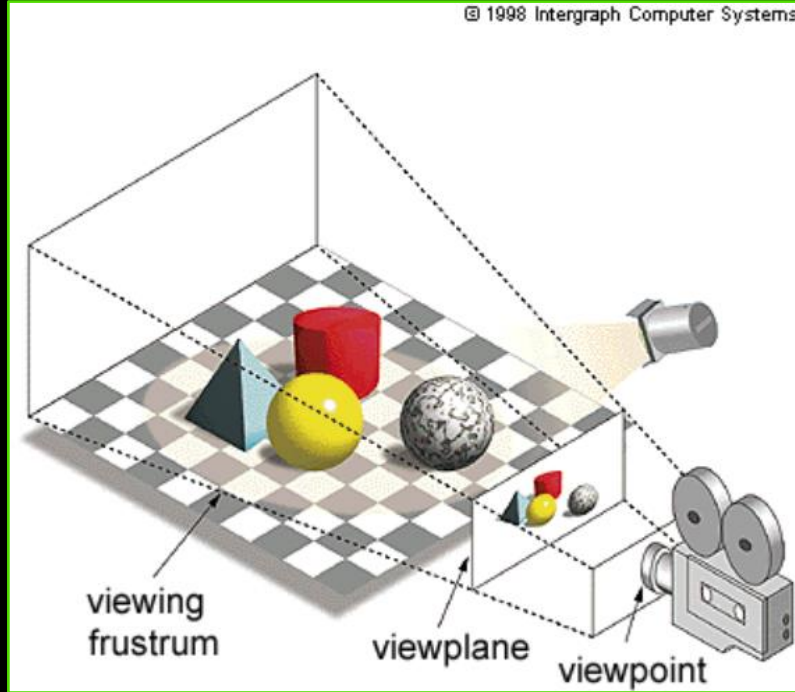
Speed and energy efficiency of Nvidia's chips as a multiple of performance in 2012

- Operations per second
- Operations per second per watt



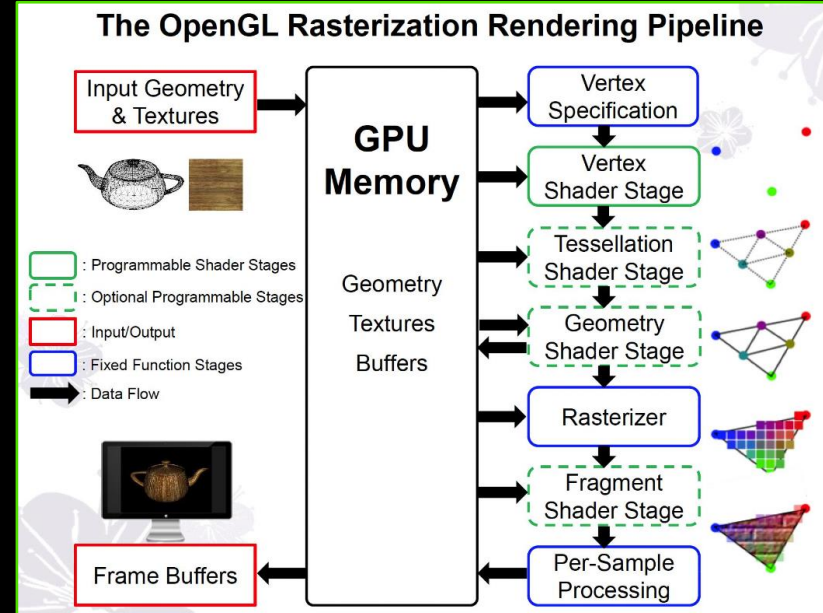
Source: NVIDIA

Why Video Gaming Cards?



By the turn of the century, the video gaming market has already standardized around a few APIs for rendering 3D video games in real-time.

None of these looked anything like scientific computing.





An API in 2004 first demonstrated the potential use of this latent floating point ability.

By 2007 NVIDIA supported a dedicated API for their own hardware.

Note that these early devices were not at all engineered for scientific computing and lacked several very fundamental capabilities. In particular EEC and double precision.

Heroic Efforts

Brook for GPUs: Stream Computing on Graphics Hardware

Ian Buck Tim Foley Daniel Horn Jeremy Sugerman Kayvon Fatahalian Mike Houston Pat Hanrahan

Stanford University

Abstract

In this paper, we present Brook for GPUs, a system for general-purpose computation on programmable graphics hardware. Brook extends C to include simple data-parallel constructs, enabling the use of the GPU as a streaming coprocessor. We present a compiler and runtime system that abstracts and virtualizes many aspects of graphics hardware. In addition, we present an analysis of the effectiveness of the GPU as a compute engine compared to the CPU, to determine when the GPU can outperform the CPU for a particular algorithm. We evaluate our system with five applications, the SAXPY and SGEMV BLAS operators, image segmentation, FFT, and ray tracing. For these applications, we demonstrate that our Brook implementations perform comparably to hand-written GPU code and up to seven times faster than their CPU counterparts.

CR Categories: I.3.1 [Computer Graphics]: Hardware Architecture—Graphics processors D.3.2 [Programming Languages]: Language Classifications—Parallel Languages

Keywords: Programmable Graphics Hardware, Data Parallel Computing, Stream Computing, GPU Computing, Brook

1 Introduction

In recent years, commodity graphics hardware has rapidly evolved from being a fixed-function pipeline into having programmable vertex and fragment processors. While this new

modern hardware. In addition, the user is forced to express their algorithm in terms of graphics primitives, such as textures and triangles. As a result, general-purpose GPU computing is limited to only the most advanced graphics developers.

This paper presents *Brook*, a programming environment that provides developers with a view of the GPU as a streaming coprocessor. The main contributions of this paper are:

- The presentation of the Brook stream programming model for general-purpose GPU computing. Through the use of streams, kernels and reduction operators, Brook abstracts the GPU as a streaming processor.
- The demonstration of how various GPU hardware limitations can be virtualized or extended using our compiler and runtime system; specifically, the GPU memory system, the number of supported shader outputs, and support for user-defined data structures.
- The presentation of a cost model for comparing GPU vs. CPU performance tradeoffs to better understand under what circumstances the GPU outperforms the CPU.

2 Background

2.1 Evolution of Streaming Hardware

Programmable graphics hardware dates back to the original programmable framebuffer architectures [England 1986]. One of the most influential programmable graphics systems

What should you do? What can you do?

Everything widely used to enable Python on GPUs, such as immensely important libraries like *TensorFlow*, *JAX*, *scikit-Cuda* and *PyTorch*, is written with C++/CUDA and then interfaced as Python libraries. If you want to contribute to the Python ecosystem, this is the way.

This shouldn't be surprising to you. Most of the important computational libraries you use are really just C and Fortran that the community has created wrappers around. That works well.

Reasonable exceptions are things like Numba and CuPy, because they rely on particular templates or routines that you can invoke at a high level. They can get great results without wrecking your codebase. Of course, they aren't as flexible as low level coding, so you may be limited.

Let's take a brief survey of the most popular methods for attempting this.

Don't use Python to program GPUs. Use GPUs for Python.

Trying to GPU code in Python is a terrible idea. It becomes an unmaintainable collision of very high level and very low level coding paradigms.

As you can now appreciate, the foremost issue in practical GPU programming is memory management. Many introductory problems, of trivial size, overlook this issue. That is misleading.

One of Python's biggest advantages is hiding memory management with its wonderful garbage collector, and also via dynamic typing.

Trying to program GPUs with Python is like trying to teach relativity without mathematics. You know it isn't the right way, but you may have had the request.

Nevertheless...

Undergraduate Lecture Notes in Physics

José Natário

General Relativity Without Calculus

A Concise Introduction to the Geometry
of Relativity

CUDA Python

Preferred by NVIDIA.

A great place to learn this is

<https://nvidia.github.io/cuda-python/overview.html>

Requires all kinds of low level detail that is completely incongruous with your Python code. What will other developers do with this?

The fact that the CUDA code itself must be a big **text string** is a strong clue that this is not a good idea for any substantial algorithm. I don't even dignify this by using the term "software engineering".

```
from cuda import cuda, nvrtc
import numpy as n

def ASSERT_DRV(err):
    if isinstance(err, cuda.CUresult):
        if err != cuda.CUresult.CUDA_SUCCESS:
            raise RuntimeError("Cuda Error: {}".format(err))
    elif isinstance(err, nvrtc.nvrtcResult):
        if err != nvrtc.nvrtcResult.NVRTC_SUCCESS:
            raise RuntimeError("Nvrtc Error: {}".format(err))
    else:
        raise RuntimeError("Unknown error type: {}".format(err))

saxpy = """\
extern "C" __global__
void saxpy(float a, float *x, float *y, float *out, size_t n)
{
    size_t tid = blockIdx.x * blockDim.x + threadIdx.x;
    if (tid < n) {
        out[tid] = a * x[tid] + y[tid];
    }
}
"""
```

SAXPY Routine Code

CUDA Python

But this is just the CUDA side. Now we have to compile it to make it available.

```
err, prog = nvrtc.nvrtcCreateProgram(str.encode(saxpy), b"saxpy.cu", 0, [],
[])

# Compile program
opts = [b"--fmad=false", b"--gpu-architecture=compute_75"]
err, = nvrtc.nvrtcCompileProgram(prog, 2, opts)

# Get PTX from compilation
err, ptxSize = nvrtc.nvrtcGetPTXSize(prog)
ptx = b" " * ptxSize
err, = nvrtc.nvrtcGetPTX(prog, ptx)
```

Compiling SAXPY Code

Now you can load the load the routine into your application.

```
# Initialize CUDA Driver API
err, = cuda.cuInit(0)
# Retrieve handle for device 0
err, cuDevice = cuda.cuDeviceGet(0)
# Create context
err, context = cuda.cuCtxCreate(0, cuDevice)
# Load PTX as module data and retrieve function
ptx = np.char.array(ptx)
# Note: Incompatible --gpu-architecture would be detected here
err, module = cuda.cuModuleLoadData(ptx.ctypes.data)
ASSERT_DRV(err)
err, kernel = cuda.cuModuleGetFunction(module, b"saxpy")
ASSERT_DRV(err)
```

CUDA Python

Now we have to set up our data.

There is a lot going on here just to set up the data for one kernel call.

It doesn't get any simpler if we actually have to move data dynamically. Which we will for larger problems.

Note that the [comment](#) is not mine. It is right from this official introductory example.

```
NUM_THREADS = 512 # Threads per block
NUM_BLOCKS = 32768 # Blocks per grid

a = np.array([2.0], dtype=np.float32)
n = np.array(NUM_THREADS * NUM_BLOCKS, dtype=np.uint32)
bufferSize = n * a.itemsize

hX = np.random.rand(n).astype(dtype=np.float32)
hY = np.random.rand(n).astype(dtype=np.float32)
hout = np.zeros(n).astype(dtype=np.float32)

err, dxclass = cuda.cuMemAlloc(bufferSize)
err, dyclass = cuda.cuMemAlloc(bufferSize)
err, doutclass = cuda.cuMemAlloc(bufferSize)

err, stream = cuda.cuStreamCreate(0)

err, = cuda.cuMemcpyHtoDAsync(
    dxclass, hX.ctypes.data, bufferSize, stream
)
err, = cuda.cuMemcpyHtoDAsync(
    dyclass, hY.ctypes.data, bufferSize, stream
)

# The following code example is not intuitive
# Subject to change in a future release
dX = np.array([int(dxclass)], dtype=np.uint64)
dY = np.array([int(dyclass)], dtype=np.uint64)
dout = np.array([int(doutclass)], dtype=np.uint64)

args = [a, dX, dY, dout, n]
args = np.array([arg.ctypes.data for arg in args], dtype=np.uint64)
```

CUDA Python

Finally, we can call the kernel.

There is some more cleanup code we should call after, but we'll just ignore that.

There you go, all you need to do SAXP!

```
err, = cuda.cuLaunchKernel(  
    kernel,  
    NUM_BLOCKS, # grid x dim  
    1, # grid y dim  
    1, # grid z dim  
    NUM_THREADS, # block x dim  
    1, # block y dim  
    1, # block z dim  
    0, # dynamic shared memory  
    stream, # stream  
    args.ctype.data, # kernel arguments  
    0, # extra (ignore)  
)  
  
err, = cuda.cuMemcpyDtoHAsync(  
    hout.ctype.data, doutclass, bufferSize, stream  
)  
err, = cuda.cuStreamSynchronize(stream)
```

PyCUDA

Before NVIDIA introduced CUDA Python, there was PyCUDA.

NVIDIA has been very polite in saying that PyCUDA has a place in their CUDA Python framework, but I have no idea what that means. It appears to just be superseded by CUDA Python.

However, PyCUDA has been around for a long time, so you may find reference to it. It is a similar idea. Here is their "hello_gpu" example.

This is from:

<https://document.tician.de/pycuda/>

```
import pycuda.autoinit
import pycuda.driver as drv
import numpy

from pycuda.compiler import SourceModule
mod = SourceModule("""
__global__ void multiply_them(float *dest, float *a, float *b)
{
    const int i = threadIdx.x;
    dest[i] = a[i] * b[i];
}
""")

multiply_them = mod.get_function("multiply_them")

a = numpy.random.randn(400).astype(numpy.float32)
b = numpy.random.randn(400).astype(numpy.float32)

dest = numpy.zeros_like(a)
multiply_them(
    drv.Out(dest), drv.In(a), drv.In(b),
    block=(400,1,1), grid=(1,1))

print(dest-a*b)
```


Numba

Numba is great for those concerned with Python performance. It has the capability to greatly optimize your code in a "Pythonic" way.

The general idea is that Numba uses decorators to enable just in time compilation. There are many qualifications and limitations, so it is not just a "Python compiler".

We don't have time to go into its many capabilities here, so we will just focus on its CUDA extensions. If you find this interesting, I encourage you to learn more about Numba in general.

Here is a variation on the common `@vectorize` decorator that adds in CUDA capability.

```
import math
# Note that for the CUDA target, we need to use the scalar functions
# from the math module, not NumPy

SQRT_2PI = np.float32((2*math.pi)**0.5)

@vectorize(['float32(float32, float32, float32)'], target='cuda')
def gaussian_pdf(x, mean, sigma):
    '''Compute value of Gaussian probability density function at x with given mean and sigma.'''
    return math.exp(-0.5 * ((x - mean) / sigma)**2) / (sigma * SQRT_2PI)
```

Numba with CUDA

Easy as that, we have a subroutine that runs on the GPU.

```
import numpy as np

# Evaluate the Gaussian a million times
x = np.random.uniform(-3, 3, size=1000000).astype(np.float32)
mean = np.float32(0.0)
sigma = np.float32(1.0)

gaussian_pdf(x, mean, sigma)
```

Note that once again we have ducked any memory management with our simple example. However, Numba does provide that capability as well. Of course, our code starts to look a lot less "Pythonic" the more low-level detail we roll in. At least it remains actual Python.

Numba even allows up to take it all the way down to CUDA kernels. But at some point it becomes silly.

Perhaps you will disagree, in which case I recommend the following detailed 5-part walk-through:

<https://www.vincent-lunot.com/post/an-introduction-to-cuda-in-python-part-1/>

```
@cuda.jit
def cudakernel1(array):
    thread_position = cuda.grid(1)
    array[thread_position] += 0.5
```

CuPy

CuPy is another option to allow drop-in high-level access to the GPU. It can be as simple as replacing some NumPy and SciPy calls:

```
import cupy as cp

x = cp.arange(6).reshape(2, 3).astype('f')

x.sum(axis=1)
```

There is good documentation at <https://github.com/cupy/cupy>

And you can also find some more misleadingly simple examples of how you could use this to write your own CUDA kernels:

```
>>> import cupy as cp
>>> kern = cp.RawKernel(r'''
... extern "C" __global__
... void multiply_elemwise(const float* in1, const float* in2, float* out) {
...     int tid = blockDim.x * blockIdx.x + threadIdx.x;
...     out[tid] = in1[tid] * in2[tid];
... }
... ''', 'multiply_elemwise')
>>> in1 = cp.arange(16, dtype=cp.float32).reshape(4, 4)
>>> in2 = cp.arange(16, dtype=cp.float32).reshape(4, 4)
>>> out = cp.zeros((4, 4), dtype=cp.float32)
>>> kern((4,), (4,), (in1, in2, out)) # grid, block and arguments
>>> out
array([[ 0.,  1.,  4.,  9.],
       [16., 25., 36., 49.],
       [64., 81., 100., 121.],
       [144., 169., 196., 225.]], dtype=float32)
```

PySummary

Use the well-known and capable GPU-enabled Python libraries.

Maybe try a little Numba as it is low effort, and low risk.

If you are determined to intermix CUDA and Python at source level, I've shown you the only options that aren't scary experimental.

If you really find GPUs interesting, you can learn enough C to use them effectively in a day. I do this with my own Advanced Computational Physics students.

At which point you can use the standard Python C API to create useful *Python* libraries for yourself and the community. Not some hybrid abomination.